CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Design Description

for

TREVORA

Change History Signature

Version	Date	Description	Author(s)
1.0	15 May 2026	Initial SDD draft for Trevora	Lauro, Jhon Gil V. Unabia, Brent Jelson D. Gamallo, Benz Leo Sy, Jullian Philip S. Potane, Kate Marielle C.

Preface

This Software Design Description (SDD) document presents the design of Trevora, a web-based vehicle service history system for service understanding and mechanic handoff. It describes how the requirements defined in the Software Requirements Specification (SRS) will be translated into the system's architecture, modules, user interface components, backend components, object-oriented design, and data design.

This document is intended for the development team, project adviser, and stakeholders who need to understand how Trevora will be structured and implemented. It serves as a technical guide for designing the system components that support service record input, service data validation and correction, unified vehicle service history consolidation, and AI-assisted service understanding and mechanic handoff.

The design decisions in this document are based on the approved SRS of Trevora, including its functional requirements, non-functional requirements, constraints, assumptions, and module-based use cases. Any future changes in the system design, database structure, or implementation details should remain aligned with the approved requirements and be documented through proper revision updates.

Table of Contents

1. Introduction

1.1. Purpose

The purpose of this Software Design Description is to describe how the approved requirements of Trevora will be translated into the system's design and implementation structure. This document presents the system architecture, user interface design, frontend and backend components, object-oriented components, and data design for each module and transaction.

It serves as a technical guide for the development team, project adviser, and stakeholders to understand how Trevora will be structured, implemented, and maintained during development.

1.2. Scope

Trevora is a web-based vehicle service history system designed for vehicle owners. The system helps vehicle owners capture, validate, consolidate, understand, and share vehicle maintenance and repair records.

The system allows vehicle owners to submit service records through receipt image upload, voice input, or manual entry. The system extracts or captures key service details such as service date, service type, parts replaced, shop/mechanic name, cost, and remarks. Before saving, the owner can review, correct, and confirm the service details.

Validated records are stored under the correct registered vehicle profile and organized into one chronological service history. The system also shows AI-generated explanations beside service records so vehicle owners can understand what was done and why it matters.

For mechanic handoff, the owner can generate a one-time QR access request for the service history of the selected registered vehicle. The mechanic can request access, but the owner must approve the request first. Once approved, the mechanic can view the approved vehicle service history through temporary read-only access and use AI-assisted search to find related past service information. Access automatically expires after a system-set time limit.

The MVP does not include dealership integration, manufacturer database integration, vehicle diagnostic hardware, predictive maintenance, legal vehicle ownership transfer, or mechanic editing of owner records.

1.3. Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification.
Trevora	The official project name for the vehicle service history system.
Vehicle Owner	The primary user who submits, validates, views, manages, and shares the service history of their registered vehicle.
Vehicle Profile	A system record that stores information about a registered vehicle and links service records to that vehicle.

Service Record	A structured record of maintenance or repair activity.
Receipt Image	A submitted image of a service receipt used as an input source for extracting service details.
Voice Input	Spoken service information submitted by the user and converted into text-based service data.
Manual Entry	A backup input method where the vehicle owner directly enters service details into system fields.
Validated Service Record	A service record reviewed, corrected, and confirmed by the owner before storage.
QR Access Request	A one-time QR code generated by the vehicle owner to allow a mechanic to request temporary access to the service history of the selected registered vehicle.
Temporary Read-Only Access	Limited mechanic access through a temporary page where the service history of the approved vehicle can be viewed but not edited or deleted, and access expires after a system-set time limit.
AI-Assisted Search	A feature that helps mechanics find relevant approved records using natural-language search.
Use Case	A user-centered transaction or system function required by the SRS.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
AI-Generated Explanation	A simple explanation shown beside a service record to help the vehicle owner understand what was done and why it matters.
Owner Approval	The required confirmation from the vehicle owner before a mechanic can view shared service records.
System-Set Time Limit	The fixed access duration set by the system for temporary mechanic access.
Mechanic / Service Personnel	A secondary user who can request temporary read-only access to the service history of a vehicle approved by the vehicle owner.

1.4. References

The following references support the system requirements related to vehicle service records, maintenance data quality, receipt image extraction, speech-to-text input, AI-generated explanations, QR security, and service history sharing.

- ***Chen, J., Ruan, Y., Guo, L., & Lu, H. (2020).*** *BCVehis: A blockchain-based service prototype of vehicle history tracking for used-car trades in China. IEEE Access, 8, 214842–214851.*
  Use for: vehicle history tracking, traceable records, service history sharing.
- ***Dua, M., Akanksha, & Dua, S. (2023).*** *Noise robust automatic speech recognition: Review and analysis. International Journal of Speech Technology, 26(2), 475–519.*
  Use for: speech-to-text / voice input.

- ***Focardi, R., Luccio, F. L., & Wahsheh, H. A. M. (2019).*** *Usable security for QR code. Journal of Information Security and Applications, 48, 102369.*
  Use for: QR access, QR security, one-time access concerns.
- ***Ha, H. T., & Horak, A. (2022).*** *Information extraction from scanned invoice images using text analysis and layout features. Signal Processing: Image Communication, 102, 116601.*
  Use for: receipt image extraction / OCR.
- ***Hemphill, T. A., Longstreet, P., & Banerjee, S. (2022).*** *Automotive repairs, data accessibility, and privacy and security challenges: A stakeholder analysis and proposed policy solutions. Technology in Society, 71, 102090.*
  Use for: automotive repair data access, privacy, sharing issues.
- ***Hodkiewicz, M., & Ho, M. T. (2016).*** *Cleaning historical maintenance work order data for reliability analysis. Journal of Quality in Maintenance Engineering, 22(2), 146–163.*
  Use for: cleaning maintenance records, reliability of historical records.
- ***Hu, Z., Zhang, X., & Xiong, H. (2025).*** *A topic joint model for knowledge extraction from unstructured maintenance records. Engineering Applications of Artificial Intelligence, 142, 109743.*
  Use for: AI/knowledge extraction from unstructured maintenance records.
- ***Liang, J. S. (2020).*** *A process-based automotive troubleshooting service and knowledge management system in collaborative environment. Robotics and Computer-Integrated Manufacturing, 61, 101836.*
  Use for: automotive service knowledge management.
- ***Lin, C.-J., Liu, Y.-C., & Lee, C.-L. (2022).*** *Automatic receipt recognition system based on artificial intelligence technology. Applied Sciences, 12(2), 853.*
  Use for: AI receipt recognition.
- ***Madhikermi, M., Kubler, S., Robert, J., Buda, A., & Framling, K. (2016).*** *Data quality assessment of maintenance reporting procedures. Expert Systems with Applications, 63, 145–164.*
  Use for: maintenance data quality and reporting reliability.
- ***Rong, Y., Leemann, T., Nguyen, T.-T., Fiedler, L., Qian, P., Unhelkar, V., Seidel, T., Kasneci, G., & Kasneci, E. (2024).*** *Towards human-centered explainable AI: A survey of user studies for model explanations. IEEE Transactions on Pattern Analysis and Machine Intelligence, 46(4), 2104–2122.*
  Use for: AI-generated explanations / user understanding.
- ***Sala, R., Bertoni, M., Pirola, F., & Pezzotta, G. (2023).*** *Improvement of maintenance-based product-service system offering through field data: A case study. Production & Manufacturing Research, 11(1).*
  Use for: using field maintenance data and service improvement.
- ***Sala, R., Pirola, F., Pezzotta, G., & Cavalieri, S. (2022).*** *Data-driven decision making in maintenance service delivery process: A case study. Applied Sciences, 12(15), 7395.*
  Use for: maintenance service data and decision-making.
- ***Wang, R. Y., & Strong, D. M. (1996).*** *Beyond accuracy: What data quality means to data consumers. Journal of Management Information Systems, 12(4), 5–33.*
  Use for: data quality, completeness, accessibility, usefulness.
- ***Webster, B. M., Hare, C. E., & McLeod, J. (1999).*** *Records management practices in small and medium-sized enterprises: A study in North-East England. Journal of Information Science, 25(4), 283–294.*
  Use for: records management and documentation practices.

2. Architectural Design

Trevora follows a web-based client-server architecture using a layered backend design. The system uses React and JavaScript for the frontend, Spring Boot for the backend, and Supabase as the database and file storage provider. Vehicle owners, mechanics/service personnel, and administrators access the system through a modern web browser. The frontend handles the user interface and sends requests to the backend through secure HTTPS API calls. The backend processes system logic, validates user actions, manages service records, controls QR-based mechanic access, communicates with external services, and stores or retrieves data from Supabase.

The frontend is responsible for displaying vehicle profile pages, service record input screens, receipt upload interface, voice input interface, editable review forms, service history page, AI-generated explanation panel, QR sharing screen, owner approval screen, mechanic temporary read-only access page, and AI-assisted search interface.

The Spring Boot backend is organized into controllers, service classes, repository classes, and domain model/entity classes. Controllers handle API requests from the frontend. Service classes implement the main business rules, such as service record processing, service draft validation, service history retrieval, AI-generated explanation processing, QR access generation, owner approval, temporary mechanic access control, notification handling, and audit logging. Repository classes handle database operations. Domain model/entity classes represent core system objects such as User, VehicleProfile, ServiceRecord, ServiceDraft, QRAccessRequest, MechanicAccessSession, and AIExplanation.

Supabase is used as the database and file storage provider. The backend communicates with Supabase to store and retrieve user accounts, vehicle profiles, service drafts, validated service records, AI explanation results, QR access requests, mechanic access sessions, mechanic search logs, notifications, and audit logs. Supabase storage may also be used to store uploaded receipt images and voice files, while the database stores their related metadata and file references.

External services and libraries may be used for OCR, speech-to-text, AI-generated explanations, AI-assisted search, and QR code generation. OCR is used to extract text from uploaded receipt images. Speech-to-text is used to convert voice input into text-based service data. AI services are used to generate simple explanations for service records and to search approved shared vehicle service history during mechanic handoff. A QR code generation library is used to create one-time QR access requests for the selected registered vehicle.

The system communicates through secure HTTPS API requests. The React frontend sends requests to the Spring Boot backend. The Spring Boot backend applies authentication, role-based permission checks, validation, and business rules before storing or retrieving data from Supabase. Mechanics can only access shared vehicle service history after scanning a QR code, submitting an access request, and receiving owner approval. Their access is temporary, read-only, and limited only to the approved vehicle service history.

If OCR, speech-to-text, or AI services are unavailable, the system still supports manual entry, editable review, saving of validated service records, and basic service history viewing when possible.

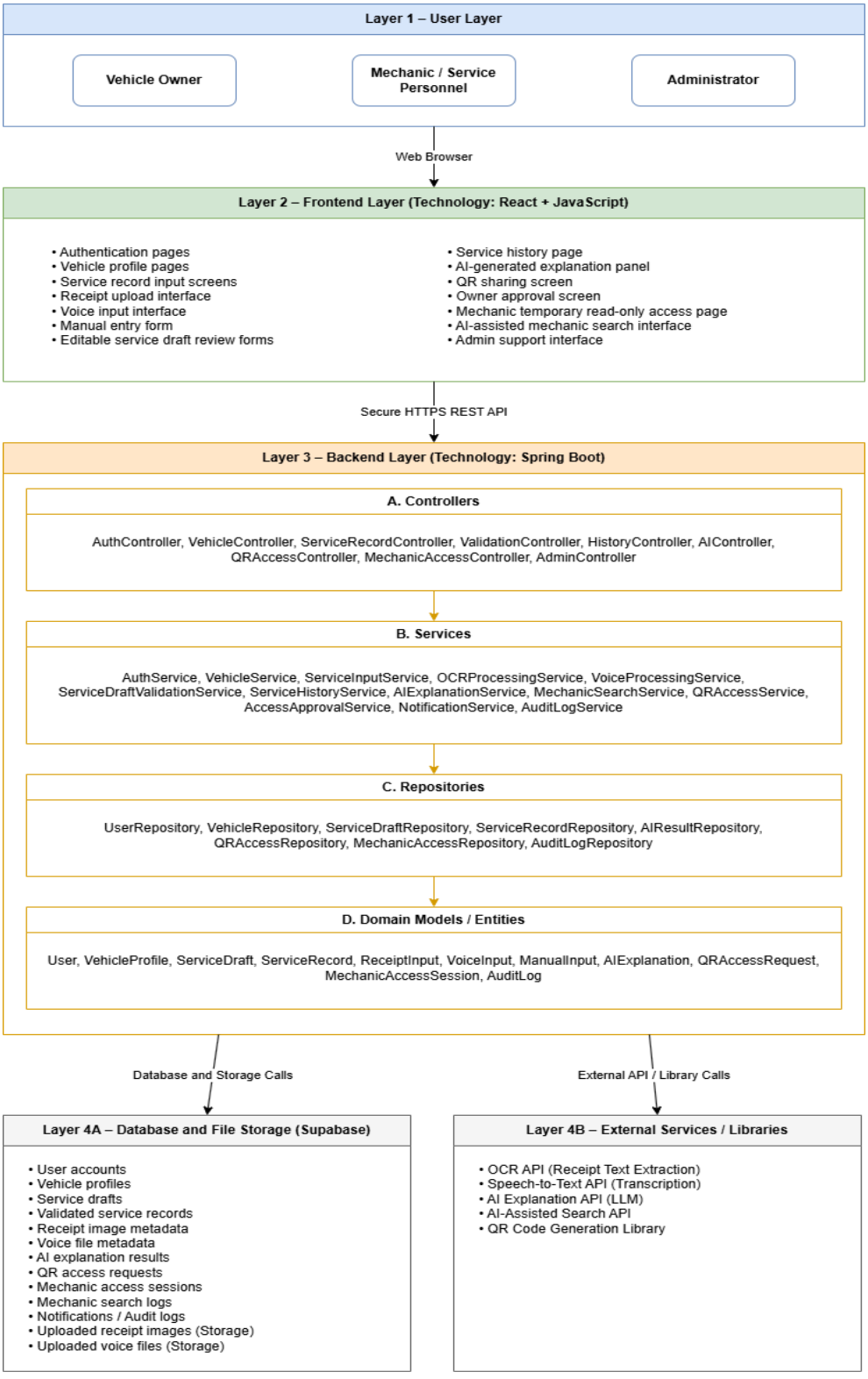

Figure 2.1 - shows the layered architecture of Trevora, including the React frontend, Spring Boot backend, Supabase database and storage layer, and external processing services. The backend is organized into controllers, services, repositories, and domain models to support object-oriented implementation.

3. Detailed Design

Module 1: Service Record Input

1.1 Create or Select Registered Vehicle Profile

- ***User Interface Design***

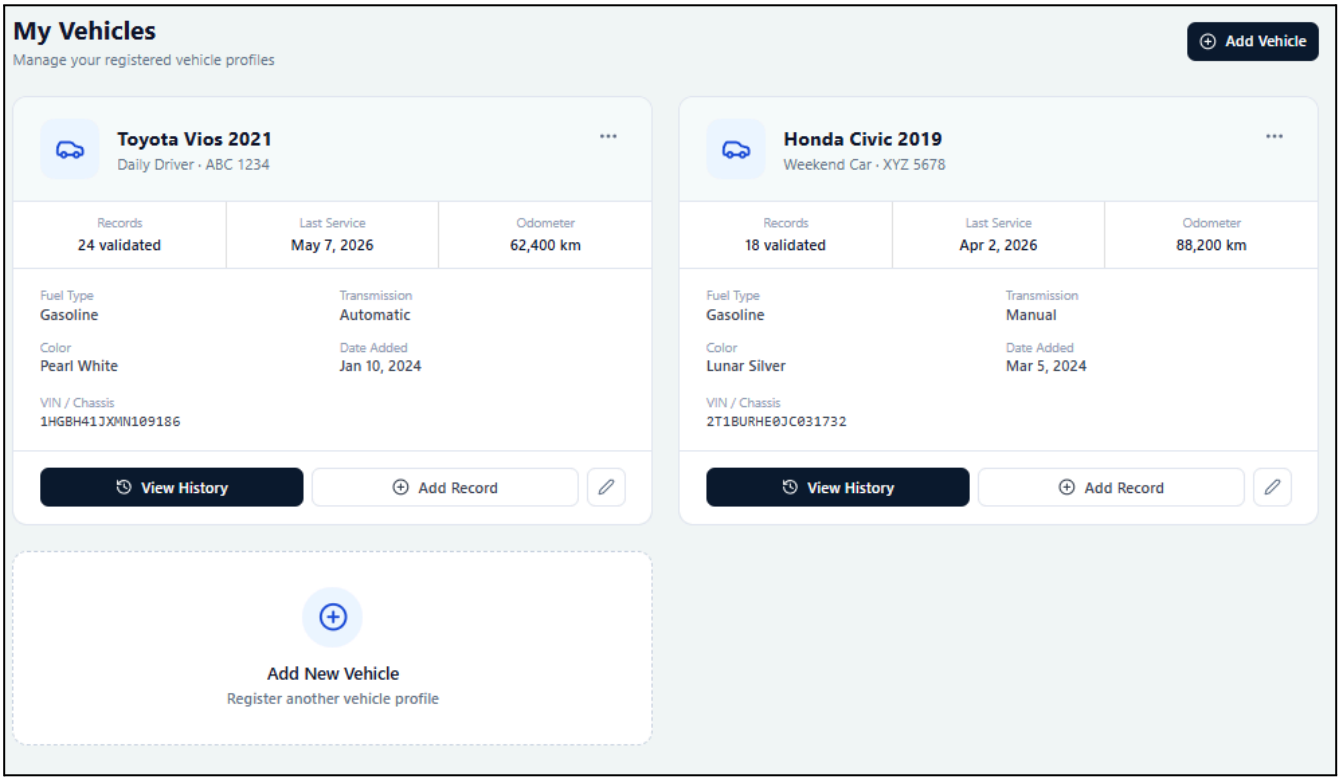

Figure - User Interface Design for Create or Select Registered Vehicle Profile

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
VehicleProfileSelectionPage	Displays the owner's existing registered vehicles and allows the owner to select a vehicle before submitting a service record.	React page component
VehicleCard	Shows summary details of each vehicle, such as make, model, year, plate number, nickname, and odometer.	Reusable React UI component
AddVehicleProfileForm	Allows the owner to enter vehicle details when creating a new vehicle profile.	React form component
VehicleProfileValidator	Checks required vehicle profile fields before allowing the owner to continue.	Frontend validation utility

ContinueToService InputButton	Allows the owner to proceed to service record input after selecting or creating a valid vehicle profile.	Button/navigation component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
VehicleController	Receives API requests for retrieving, creating, and selecting vehicle profiles.	Spring Boot REST controller
VehicleService	Handles business rules for vehicle profile creation, selection, and ownership verification.	Spring service class
VehicleRepository	Retrieves and stores vehicle profile records in the database.	Repository/data access component
VehicleProfile	Represents the registered vehicle profile linked to a vehicle owner.	Domain model/entity class
User	Represents the authenticated vehicle owner who owns one or more vehicle profiles.	Domain model/entity class

- ***Object-Oriented Components***

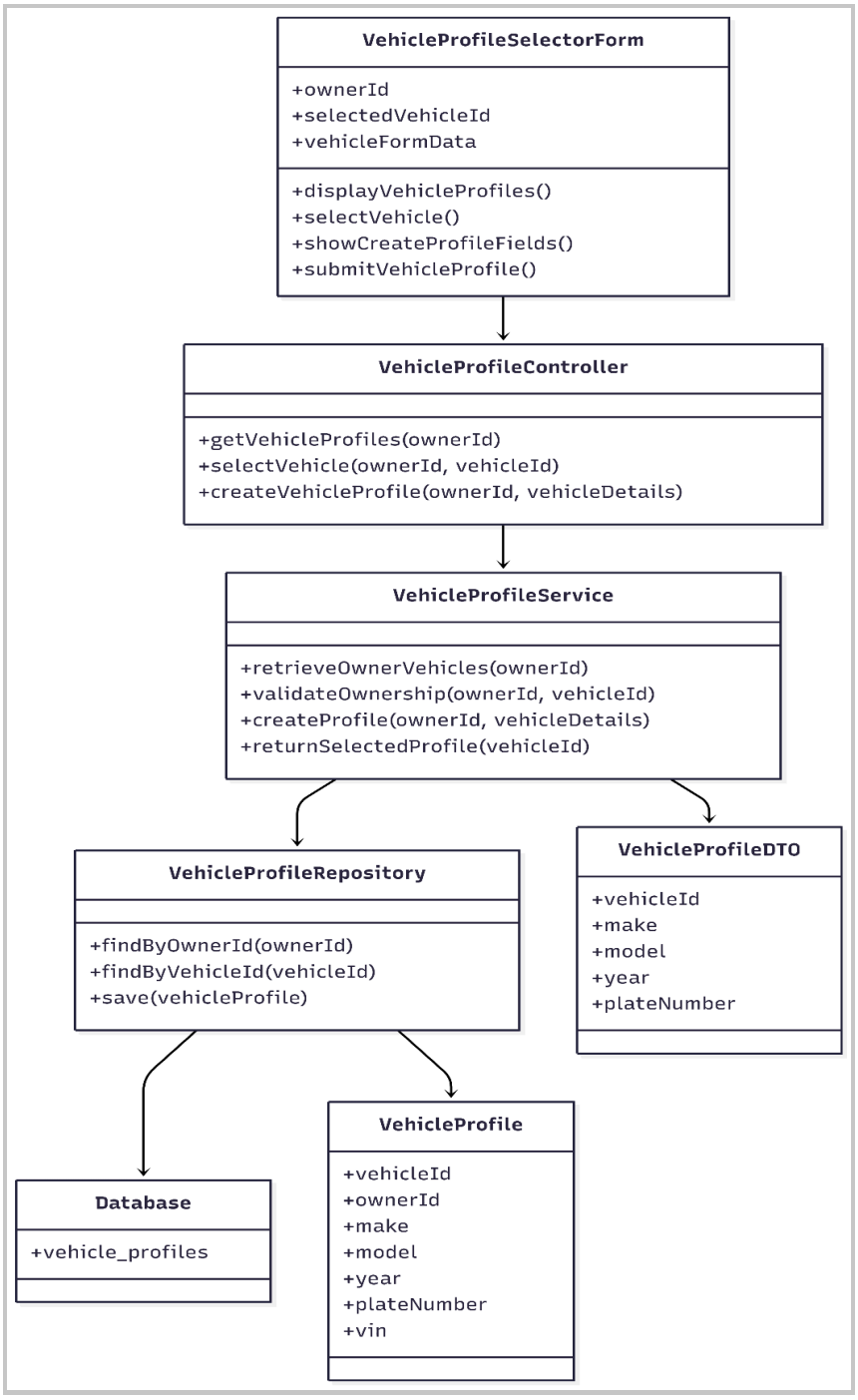

Figure - Class Diagram for Create or Select Registered Vehicle Profile

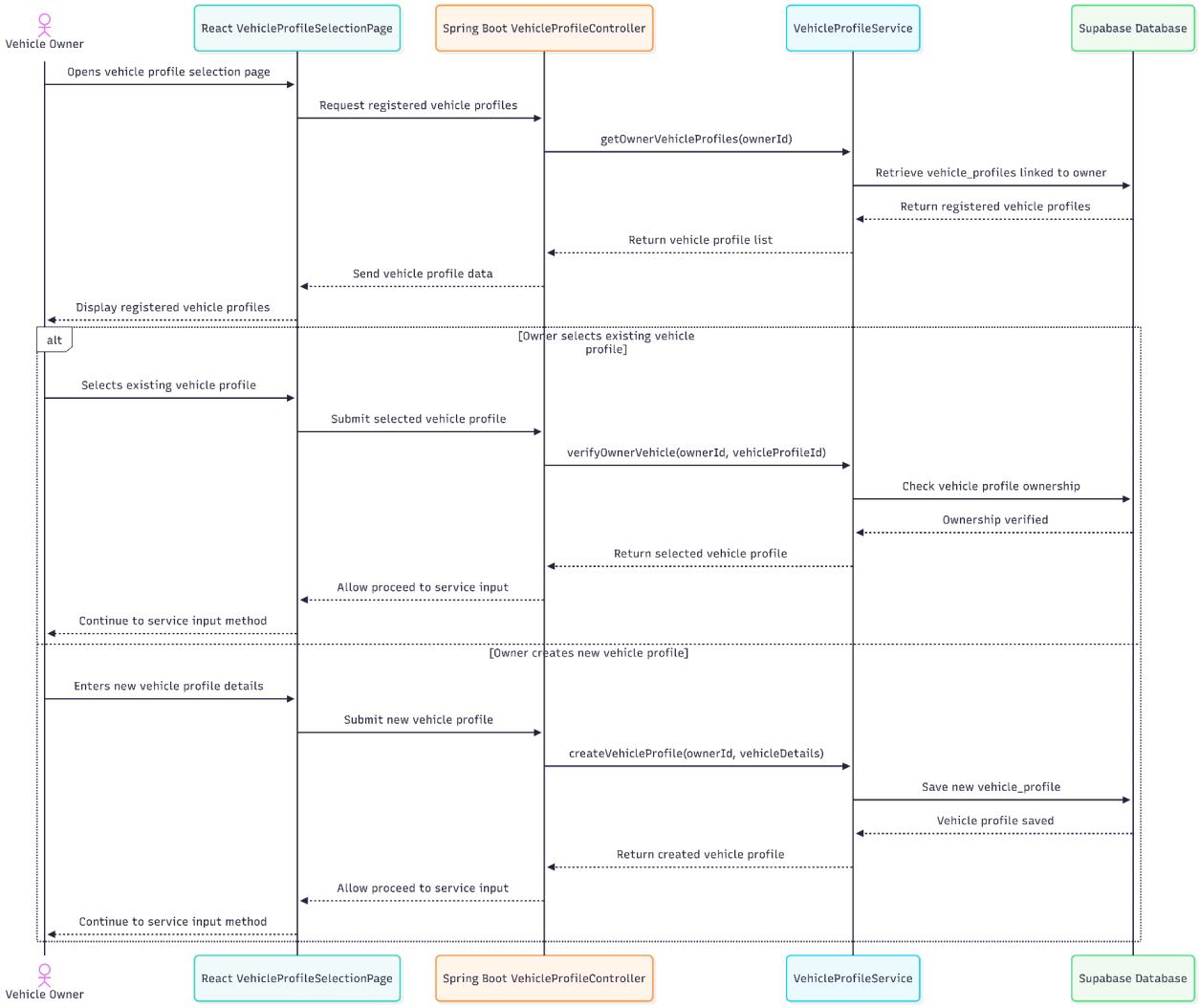

Figure - Sequence Diagram for Create or Select Registered Vehicle Profile

- ***Data Design***

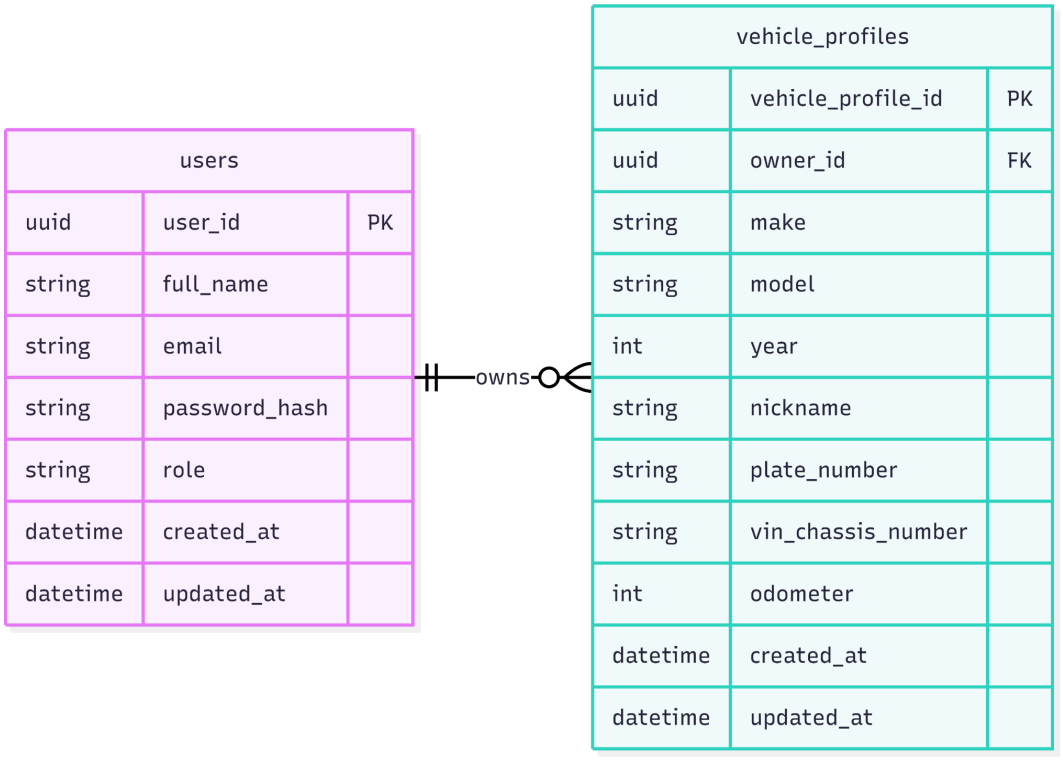

Figure - Entity Relationship Diagram for Create or Select Registered Vehicle Profile

1.2 Upload Receipt and Extract Details

- ***User Interface Design***

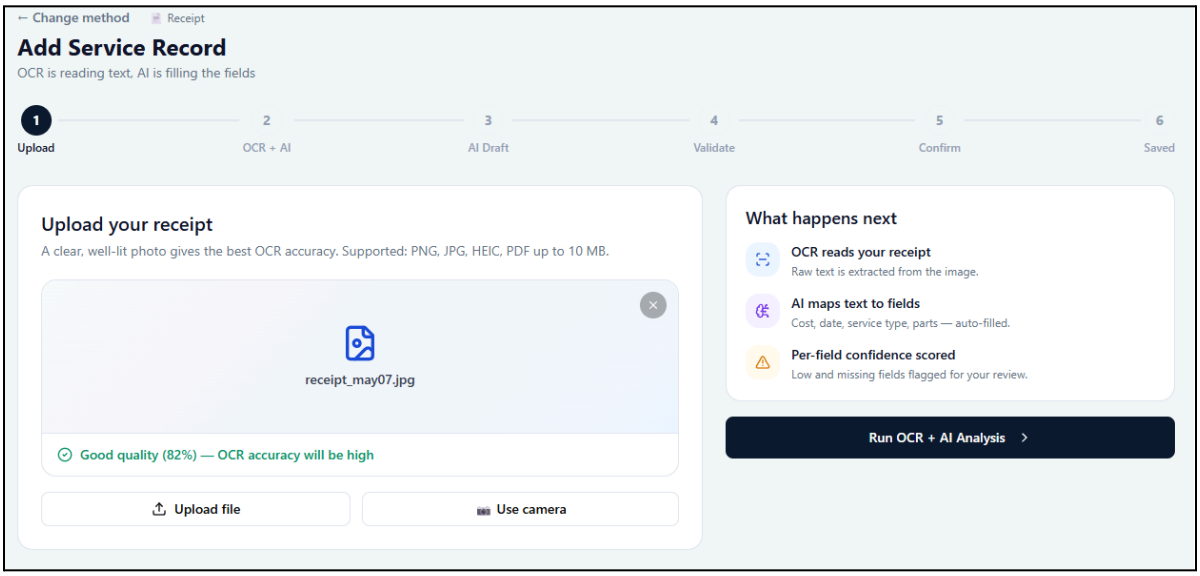

Figure - User Interface Design for Upload Receipt and Extract Details

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ReceiptUploadPage	Displays the receipt upload interface for creating a service record from an image.	React page component
ReceiptUploadBox	Allows the owner to upload a receipt image or capture one using the device camera.	Reusable upload component
ReceiptPreviewPanel	Displays a preview of the uploaded receipt image before processing.	React UI component
UploadInstructionPanel	Shows accepted file formats, upload reminders, and image quality guidance.	React UI component
OCRProcessingStatus	Displays the processing status while OCR and field extraction are being performed.	React status component
ExtractedDraftRedirect	Redirects the owner to the structured draft review screen after extraction is completed.	Navigation component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceRecordController	Receives the uploaded receipt image and starts the receipt-based service input process.	Spring Boot REST controller
ServiceInputService	Coordinates the receipt input process and converts extracted receipt data into a structured service draft.	Spring service class
OCRProcessingService	Checks image quality, extracts text from the receipt image, and returns OCR confidence information.	Spring service/external API handler
ReceiptInput	Represents receipt-based input, including image path, extracted text, and OCR confidence.	Domain model/entity or DTO class
ServiceDraft	Represents the structured service entry created from the extracted receipt information.	Domain model/entity class
FieldConfidence	Stores source and confidence status for extracted fields that may require review.	Domain model/value object
ServiceDraftRepository	Saves the generated service draft for later review and validation.	Repository/data access component

- ***Object-Oriented Components***

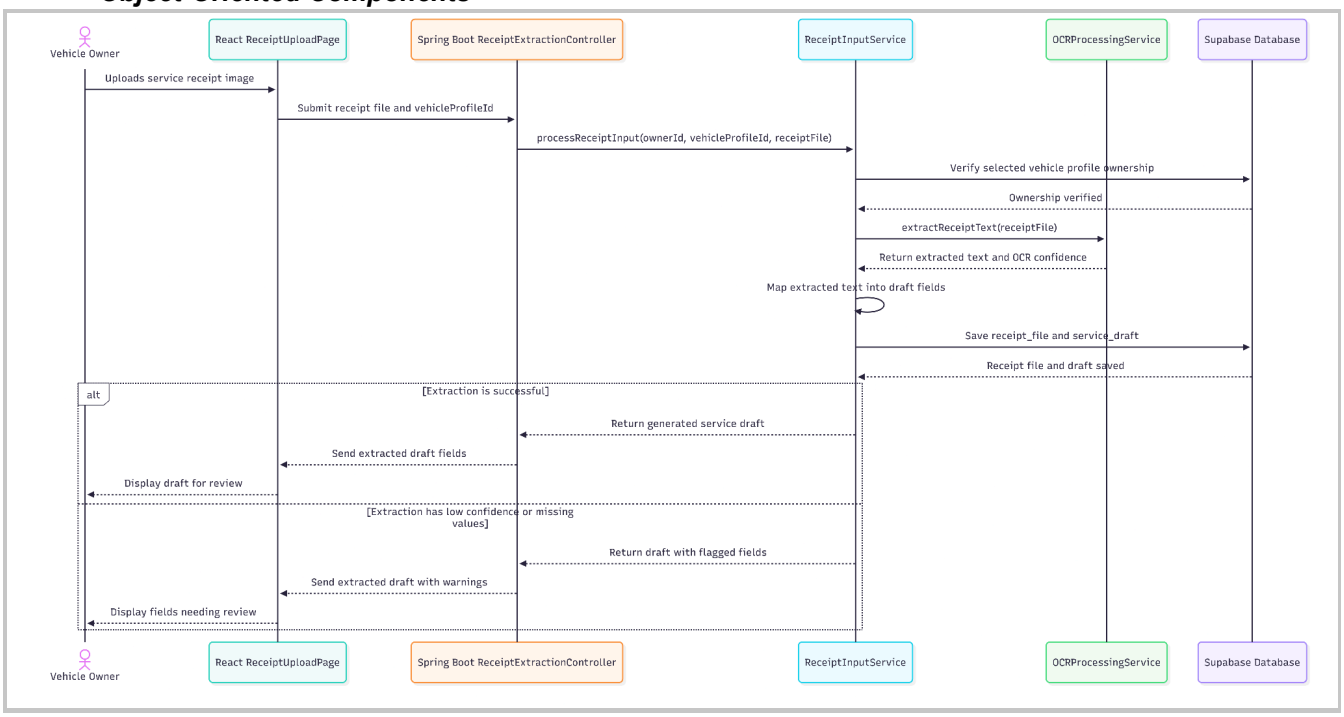

Figure - Sequence Diagram for Upload Receipt and Extract Details

- ***Data Design***

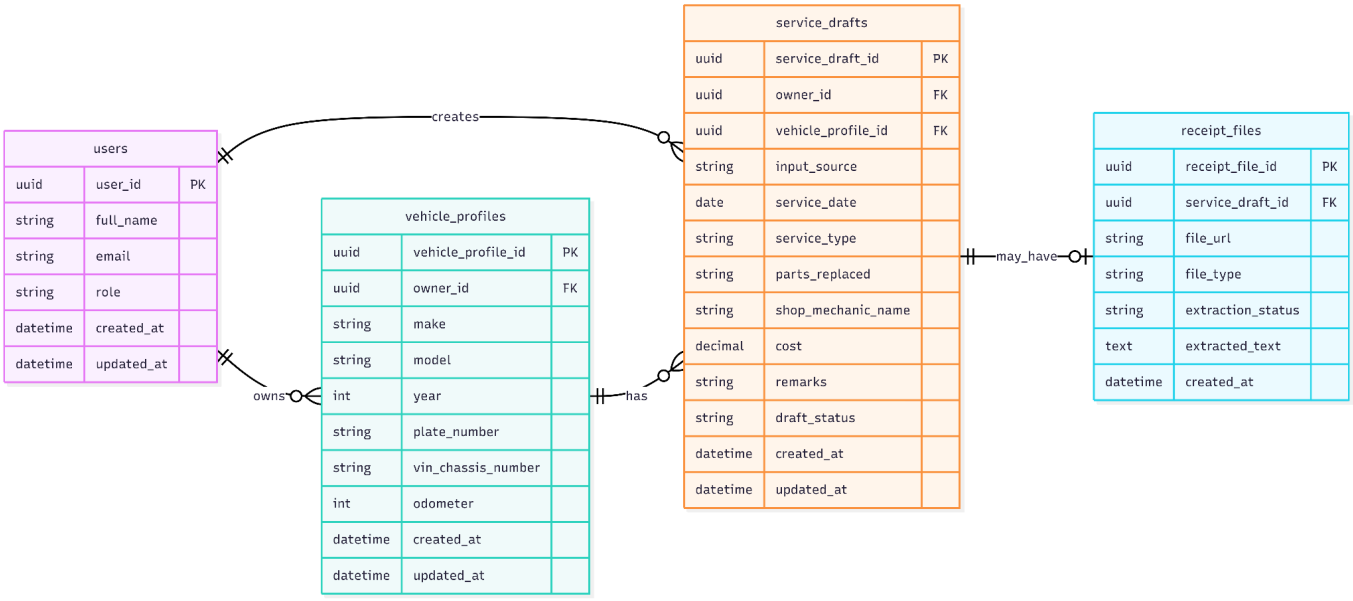

Figure - Entity Relationship Diagram for Upload Receipt and Extract Details

1.3 Record Voice Service Information

- ***User Interface Design***

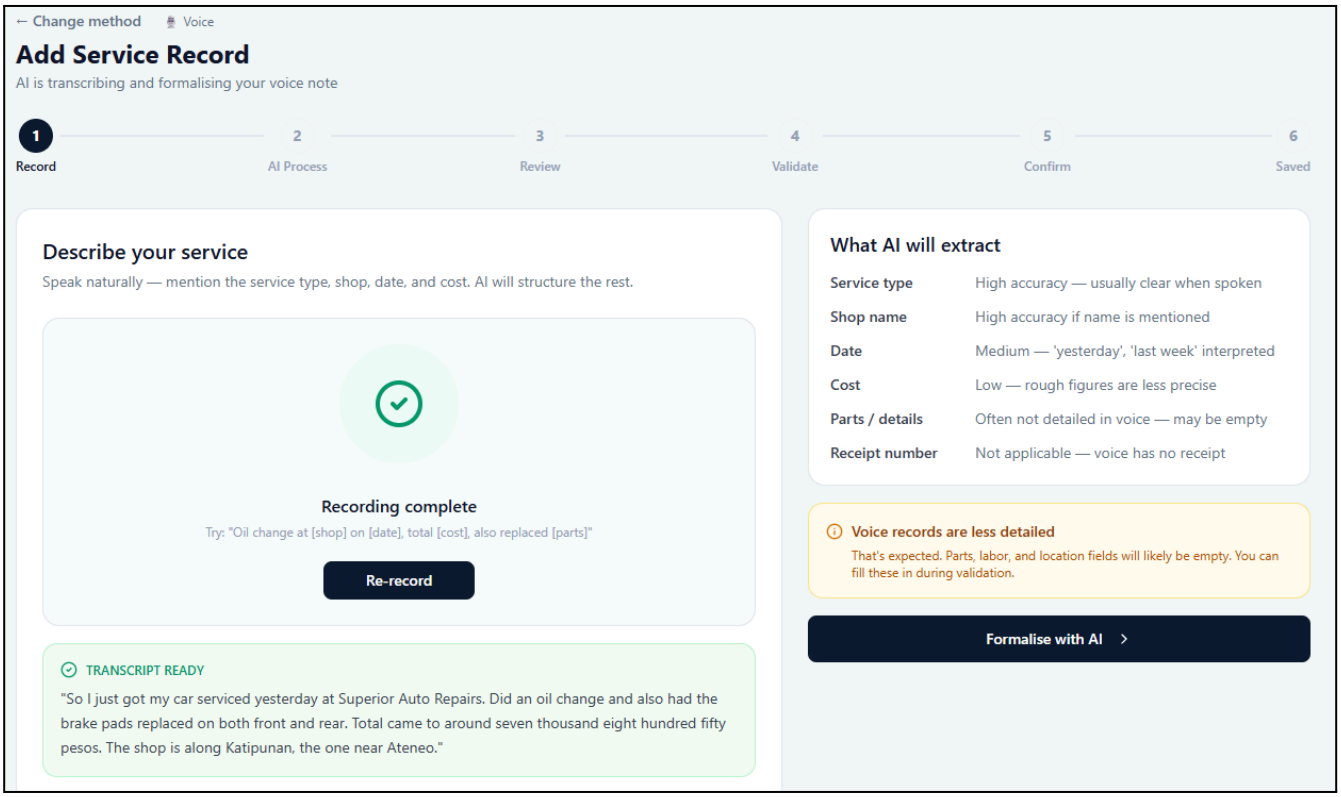

Figure - User Interface Design for Record Voice Service Information

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
VoiceInputPage	Displays the voice input interface for recording spoken service information.	React page component
VoiceRecorderControl	Allows the owner to start, pause, stop, retry, or submit a voice recording.	React media control component
VoicePromptPanel	Shows suggested details the owner should mention, such as service date, service type, cost, odometer, shop name, and remarks.	React instruction component
AudioPreviewPanel	Allows the owner to review or replay the recorded audio before submission.	React UI component
TranscriptionStatus	Displays transcription progress and processing status.	React status component

VoiceDraftRedirect	Redirects the owner to the structured draft review screen after transcription and field mapping.	Navigation component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceRecordController	Receives the submitted voice recording and starts the voice-based service input process.	Spring Boot REST controller
ServiceInputService	Coordinates the voice input process and converts transcribed speech into a structured service draft.	Spring service class
VoiceProcessingService	Converts voice input into text and calculates transcription confidence when available.	Spring service/external API handler
VoiceInput	Represents voice-based input, including audio path, transcribed text, and transcription confidence.	Domain model/entity or DTO class
ServiceDraft	Represents the structured service entry created from the extracted receipt information.	Domain model/entity class
FieldConfidence	Stores source and confidence status for extracted fields that may require review.	Domain model/value object
ServiceDraftRepository	Saves the generated service draft for later review and validation.	Repository/data access component

- ***Object-Oriented Components***

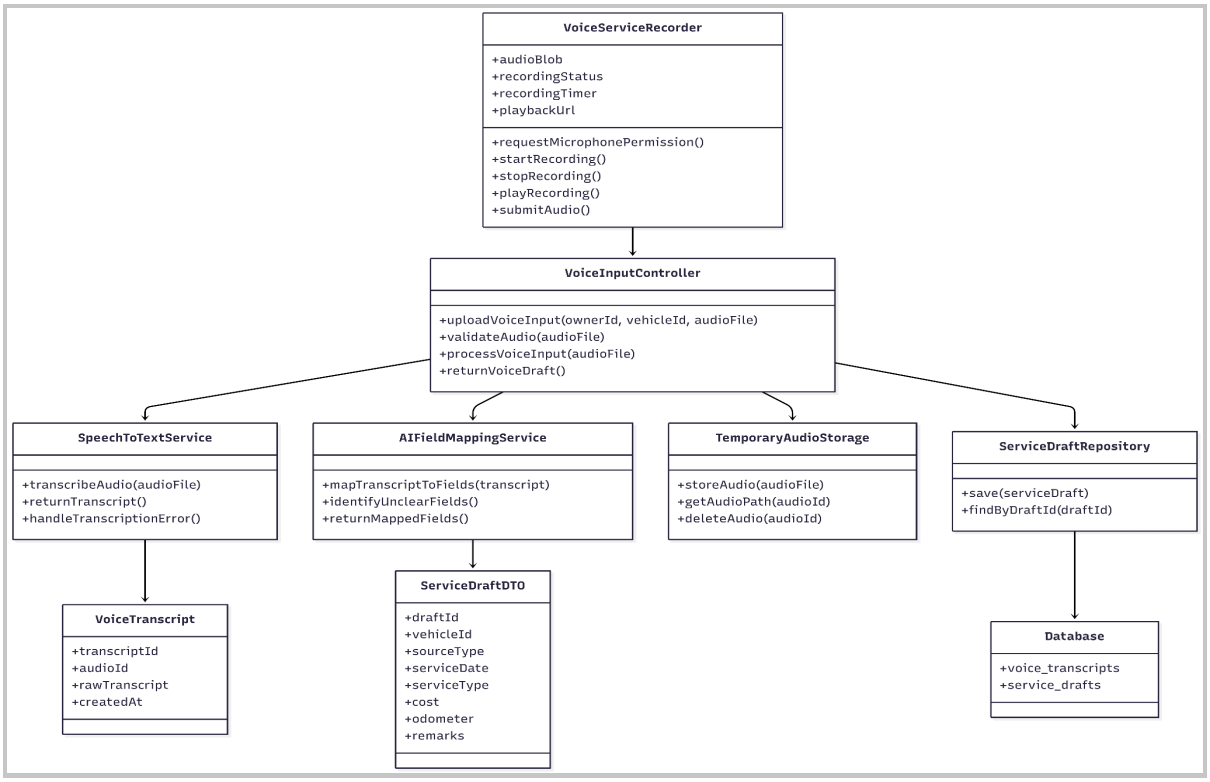

Figure - Class Diagram for Record Voice Service Information

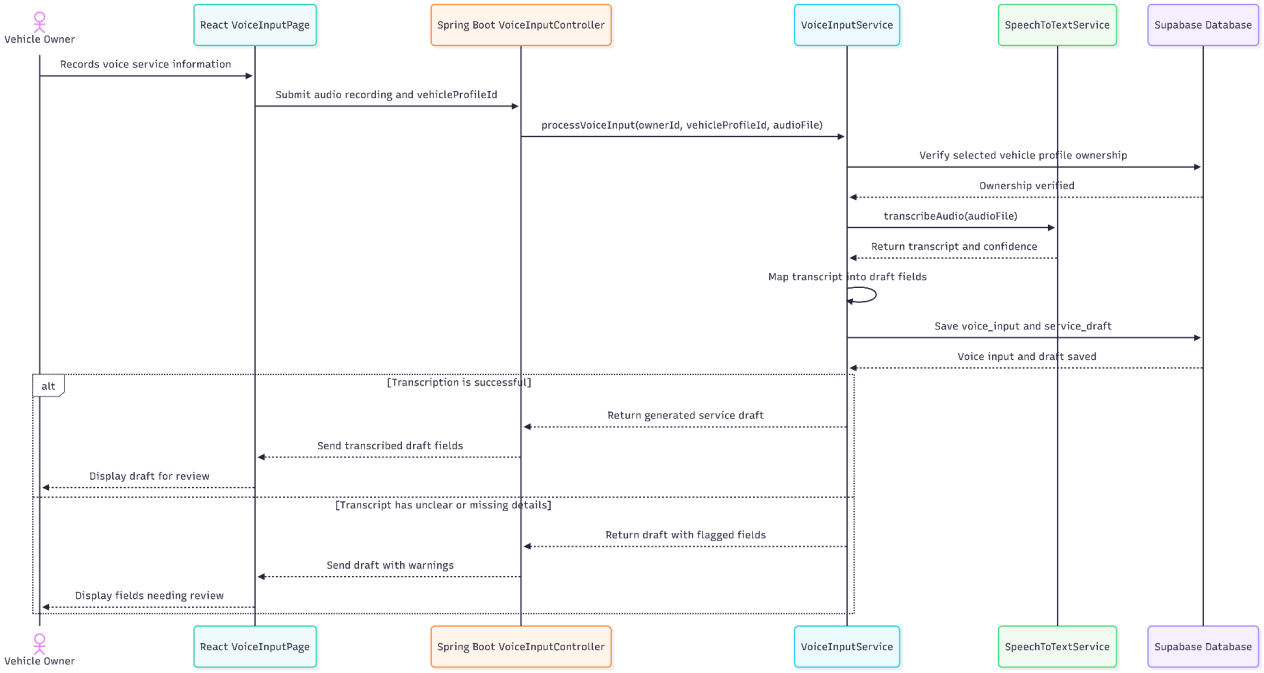

Figure - Sequence Diagram for Record Voice Service Information

- ***Data Design***

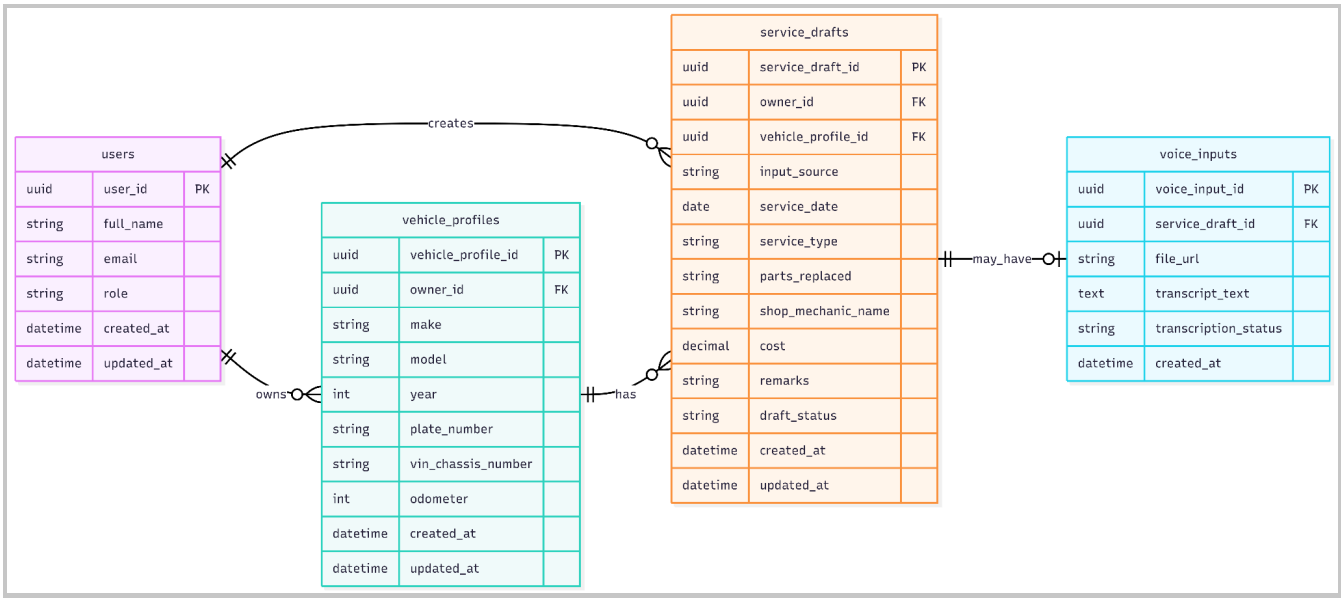

Figure - Entity Relationship for Record Voice Service Information

1.4 Enter Service Details Manually

- ***User Interface Design***

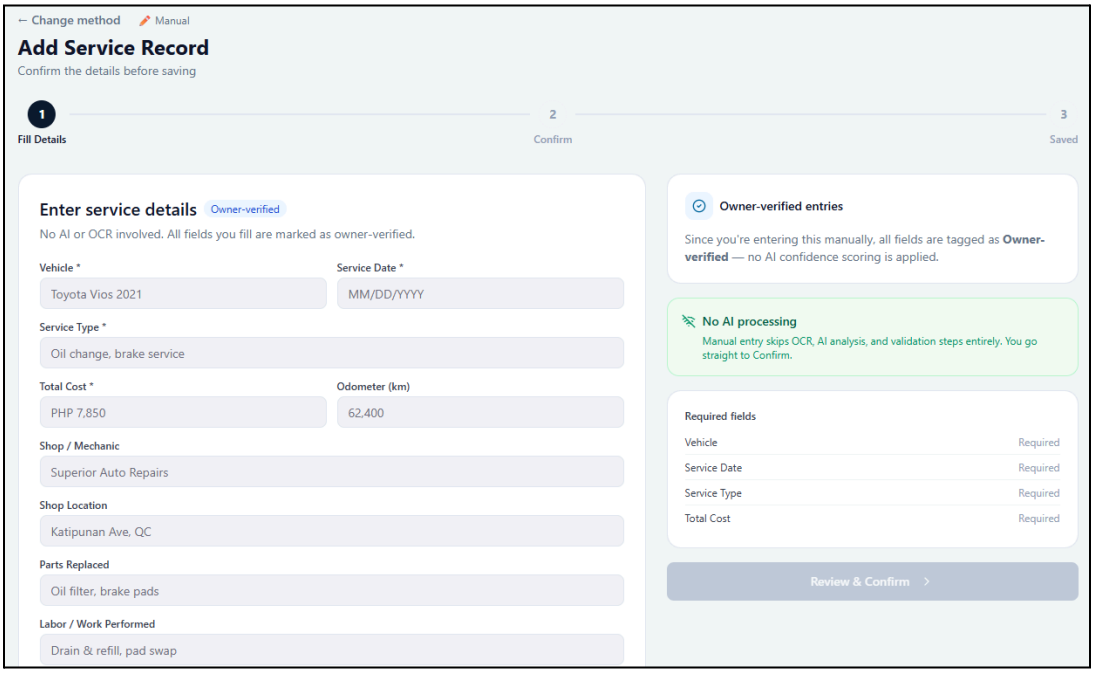

Figure - User Interface Design for Enter Service Details Manually

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ManualEntryPage	Displays the manual service record form for owner-entered service details.	React page component
ManualServiceForm	Allows the owner to enter service date, service type, odometer, cost, shop name, parts replaced, labor performed, location, and remarks.	React form component
RequiredFieldIndicator	Marks required fields that must be completed before proceeding.	React UI component
ManualInputValidator	Checks required fields and basic input format before submission.	Frontend validation utility
SaveDraftButton	Submits the manually entered details as a structured service draft.	Button/action component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceRecordController	Receives manually entered service details from the frontend.	Spring Boot REST controller
ServiceInputService	Converts owner-entered manual fields into a structured service draft.	Spring service class
ManualInput	Represents manually entered service information submitted by the owner.	Domain model/entity or DTO class
ServiceDraft	Represents the structured service entry created from manual input.	Domain model/entity class
ServiceDraftRepository	Saves the manual service draft for review and confirmation.	Repository/data access component
VehicleService	Verifies that the selected vehicle profile belongs to the authenticated owner.	Spring service class

- ***Object-Oriented Components***

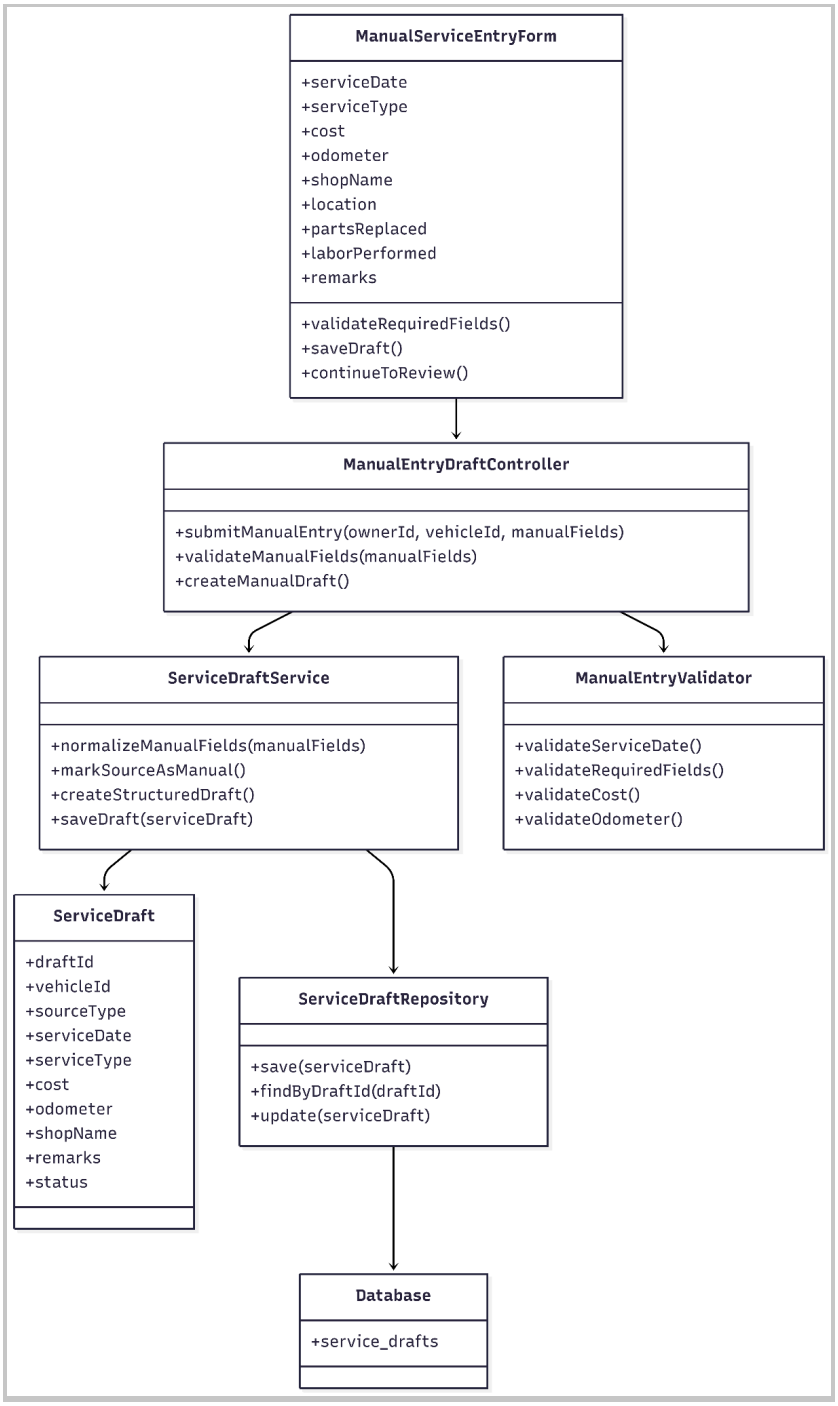

Figure - Class Diagram for Enter Service Details Manually

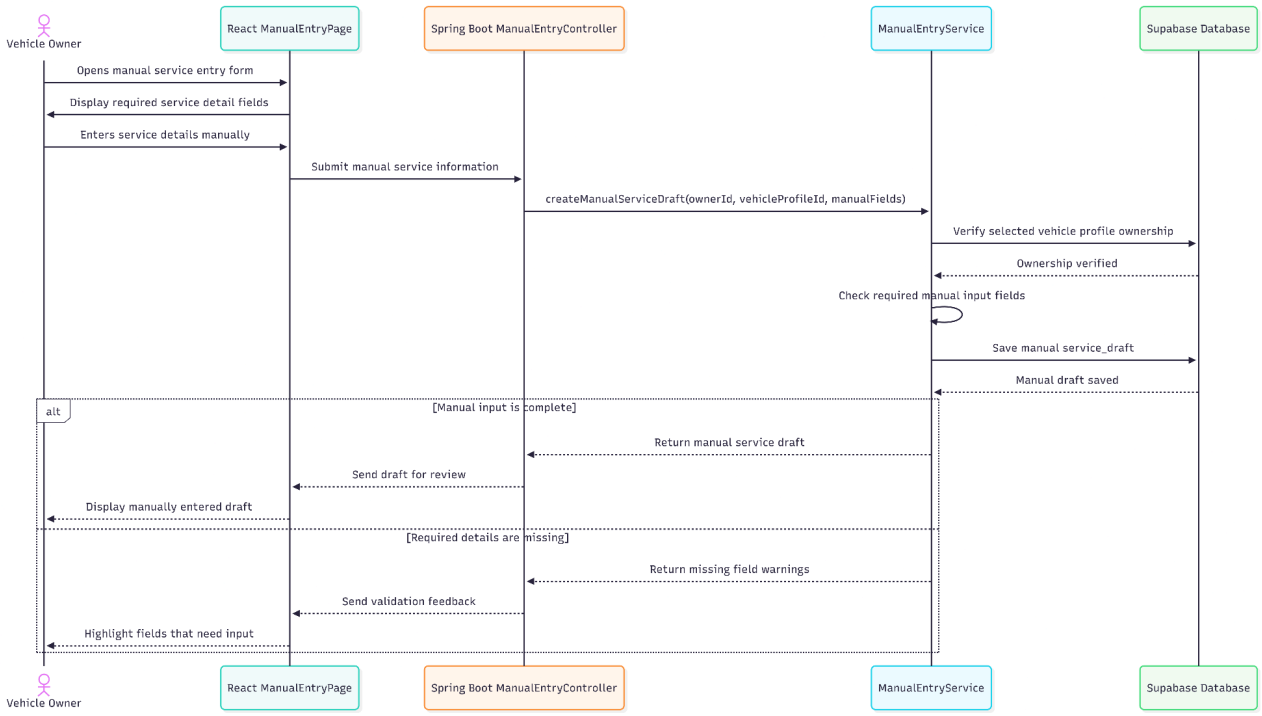

Figure - Sequence Diagram for Enter Service Details Manually

- ***Data Design***

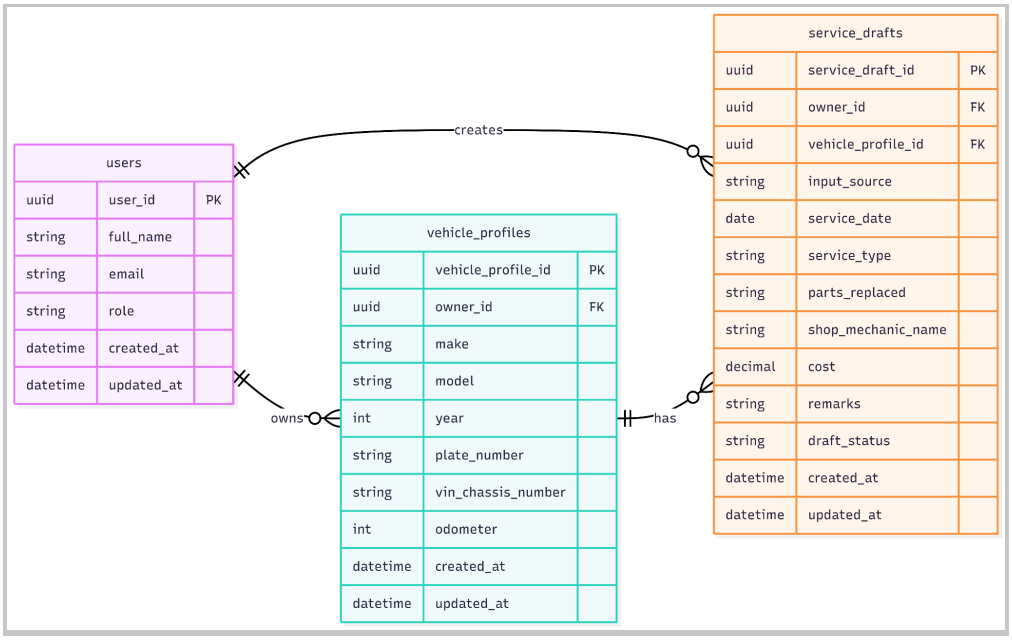

Figure - Entity Relationship Diagram for Enter Service Details Manually

1.5 Convert Input to Structured Service Entry

- ***User Interface Design***

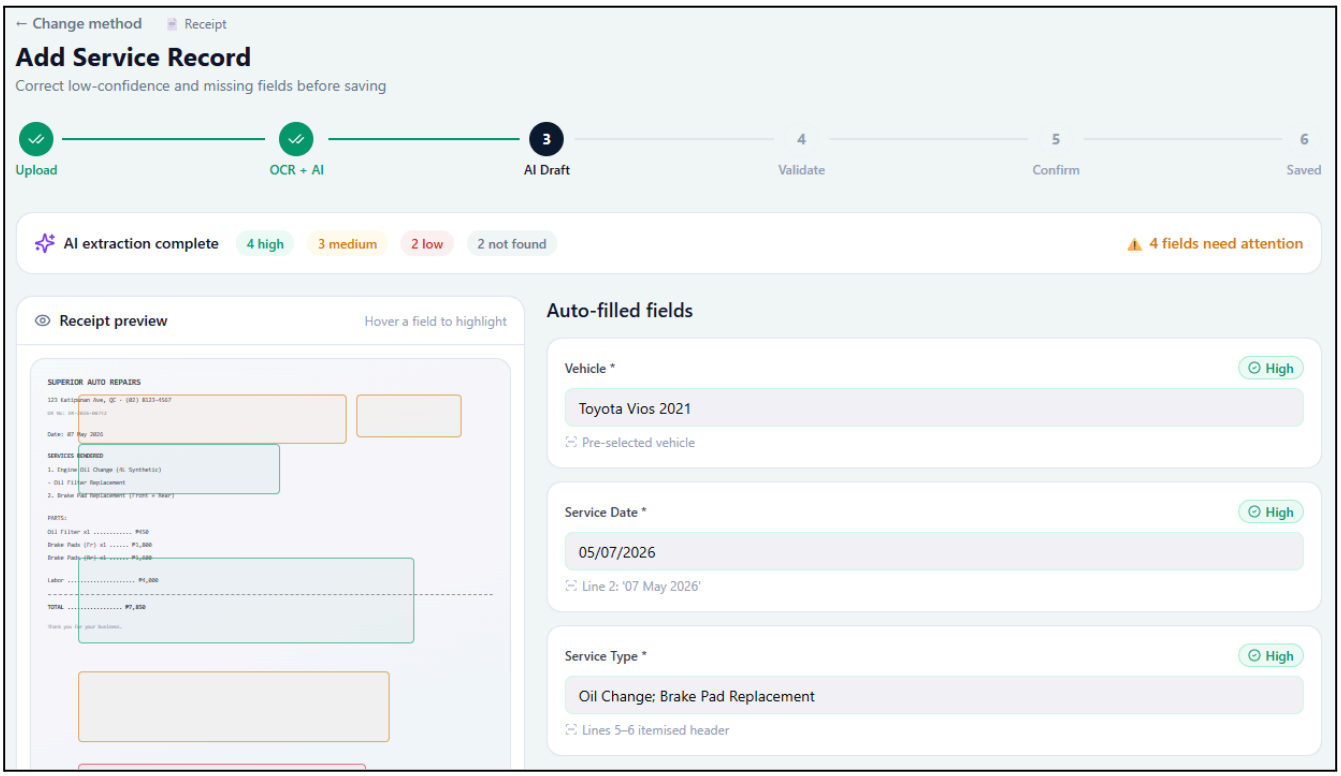

Figure - User Interface Design for Convert Input to Structured Service Entry

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
StructuredServiceDraftPage	Displays the generated structured service draft regardless of whether the source is receipt, voice, or manual input.	React page component
StructuredFieldList	Shows service fields such as service date, service type, odometer, cost, shop name, parts replaced, labor performed, and remarks.	React UI component
FieldSourceBadge	Indicates whether a field came from OCR, voice transcription, AI mapping, or owner entry.	Reusable React UI component
ConfidenceStatusBadge	Displays low-confidence, missing, or confirmed status for fields when applicable.	Reusable React UI component

ProceedToReview Button	Sends the structured draft to the review and validation process.	Button/navigation component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceRecordController	Handles the request to create or retrieve the structured service draft.	Spring Boot REST controller
ServiceInputService	Standardizes receipt, voice, and manual input into one structured service draft format.	Spring service class
OCRProcessingService	Provides extracted receipt text and confidence data for receipt-based drafts.	Spring service/external API handler
VoiceProcessingService	Provides transcribed text and confidence data for voice-based drafts.	Spring service/external API handler
ServiceDraft	Represents the unified structured service draft created from any input method.	Domain model/entity class
FieldConfidence	Represents source and confidence status for each mapped field.	Domain model/value object
ServiceDraftRepository	Stores and retrieves the structured service draft.	Repository/data access component

- ***Object-Oriented Components***

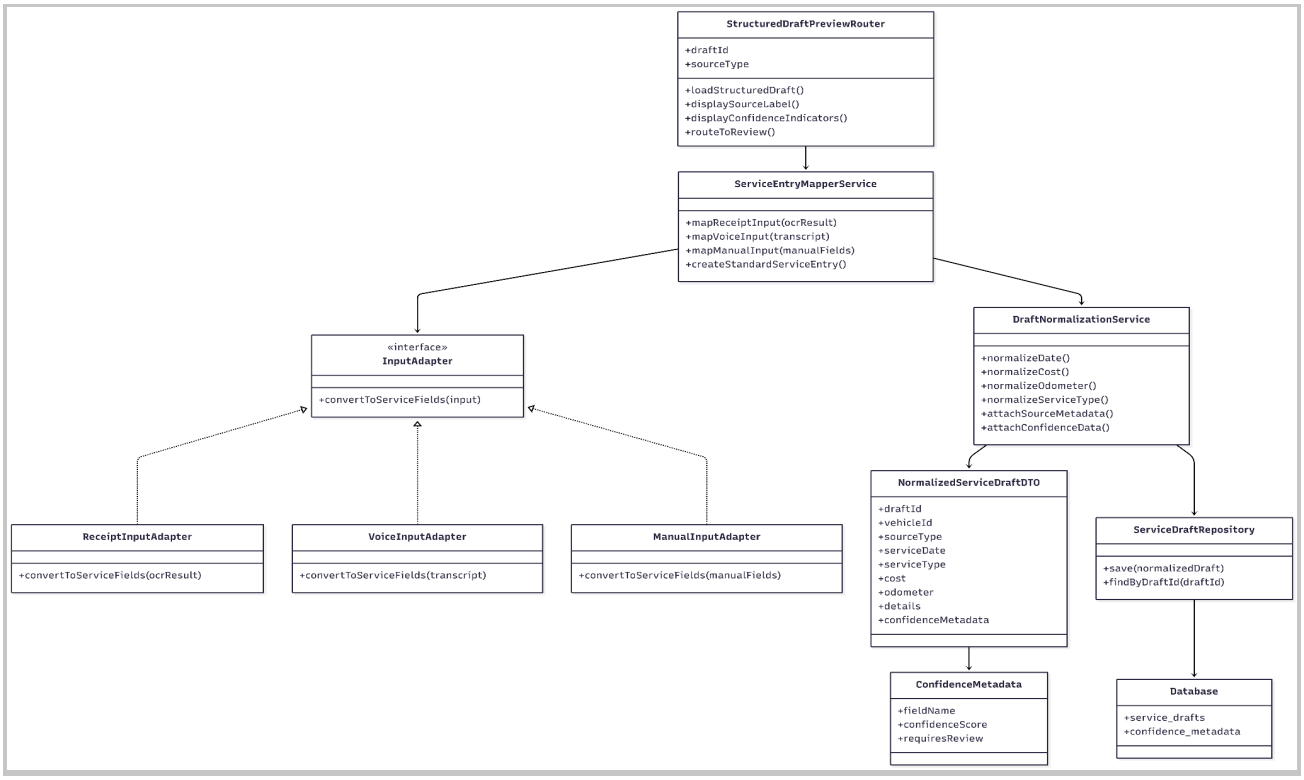

Figure - Class Diagram for Convert Input to Structured Service Entry

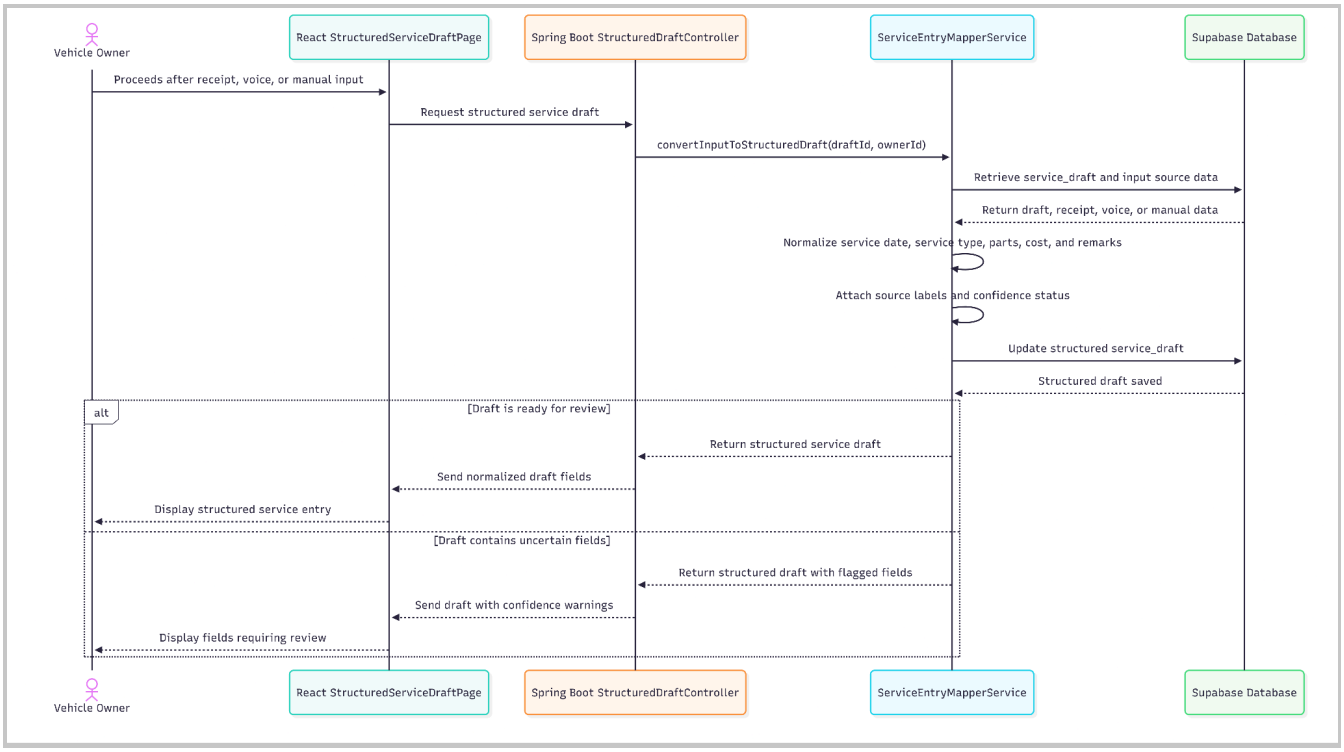

Figure - Sequence Diagram for Convert Input to Structured Service Entry

- ***Data Design***

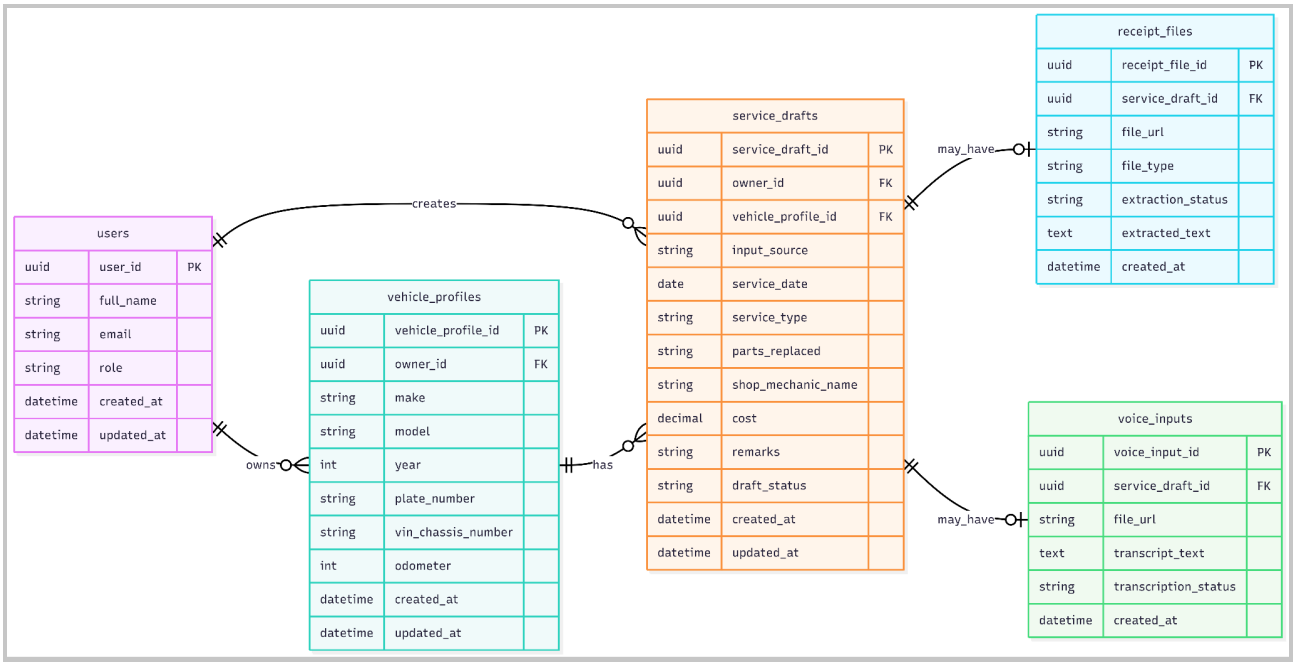

Figure - User Interface Design for Convert Input to Structured Service Entry

Module 2: Service Data Validation and Correction

2.1 Review Service Draft

- ***User Interface Design***

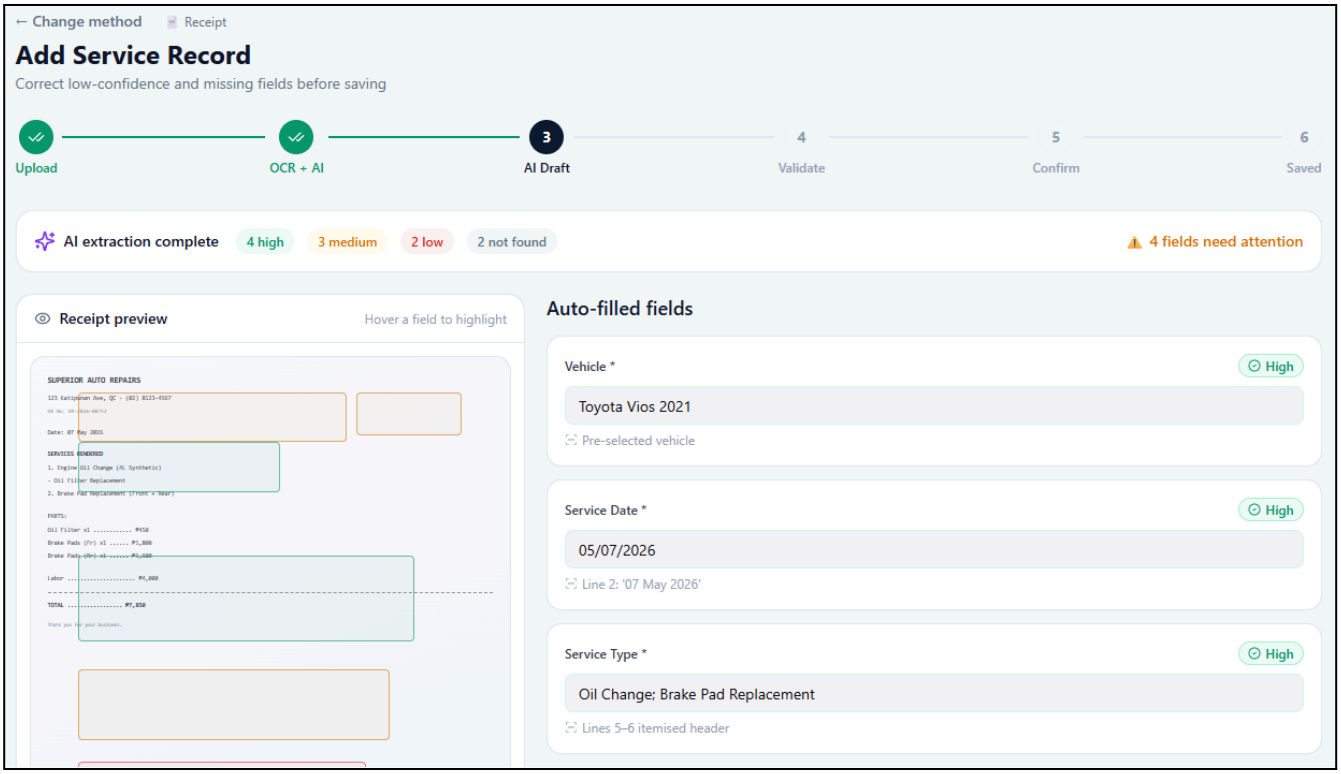

Figure - User Interface Design for Review Service Draft

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceDraftReviewPage	Displays the structured service draft generated from Module 1 and allows the owner to review the captured service details before saving.	React page component
EditableServiceDraftForm	Shows editable fields such as service date, service type, odometer, cost, shop name, parts replaced, labor performed, location, and remarks.	React form component
FieldSourceBadge	Shows the source of each field, such as OCR, voice transcription, AI mapping, or owner-entered input.	Reusable React UI component
ConfidenceStatusBadge	Displays field status such as confirmed, low-confidence, missing, or not found when applicable.	Reusable React UI component
DraftPreviewPanel	Displays the original input reference when available, such as receipt preview, extracted text, or voice transcript.	React UI component
ProceedToValidationButton	Allows the owner to proceed to required-field and confidence checking.	Button/navigation component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ValidationController	Receives requests to retrieve and display a service draft for review.	Spring Boot REST controller
ServiceDraftValidationService	Prepares the service draft for review and checks whether confidence and source data should be displayed.	Spring service class
ServiceDraftRepository	Retrieves the existing service draft created from Module 1.	Repository/data access component
ServiceDraft	Represents the structured draft created from receipt, voice, or manual input.	Domain model/entity class
FieldConfidence	Stores the source and confidence status of draft fields from OCR, voice transcription, or AI mapping.	Domain model/value object
VehicleService	Verifies that the selected draft belongs to a vehicle owned by the authenticated user.	Spring service class

- ***Object-Oriented Components***

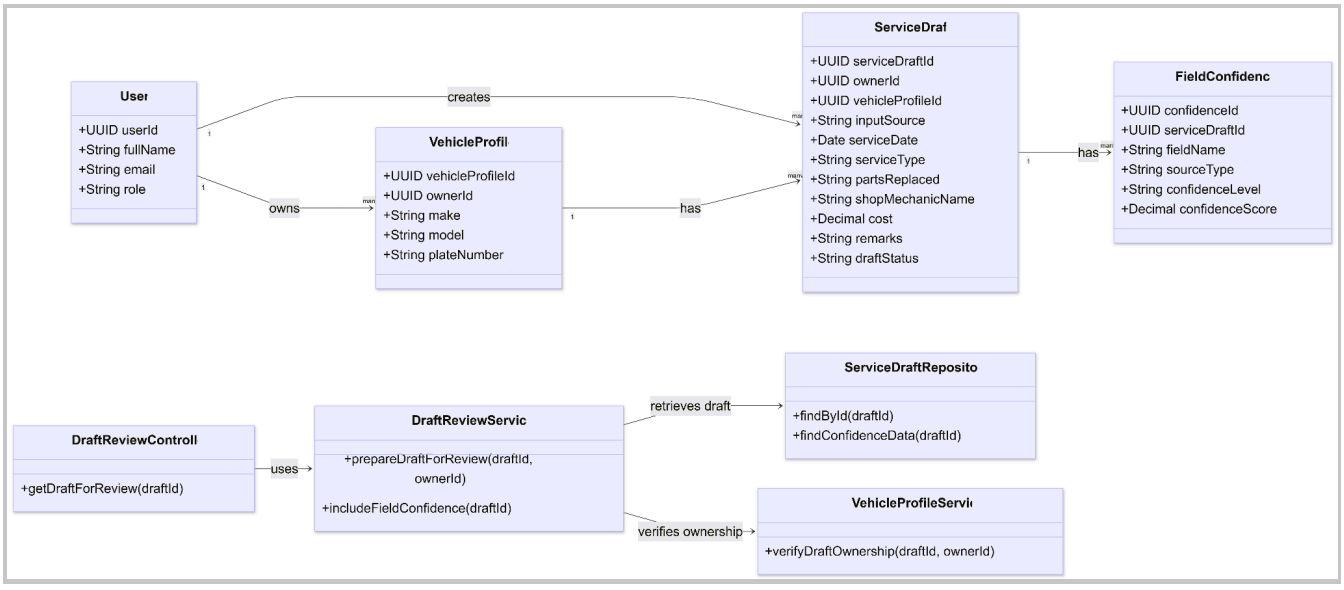

Figure - Class Diagram for Review Service Draft

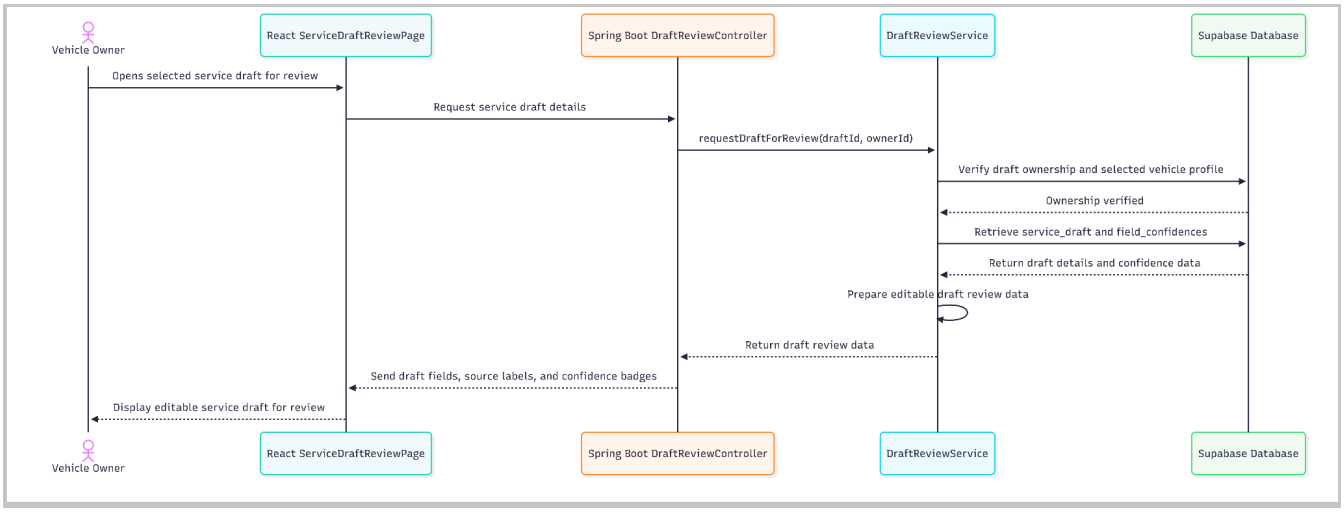

Figure - Sequence Diagram for Review Service Draft

- ***Data Design***

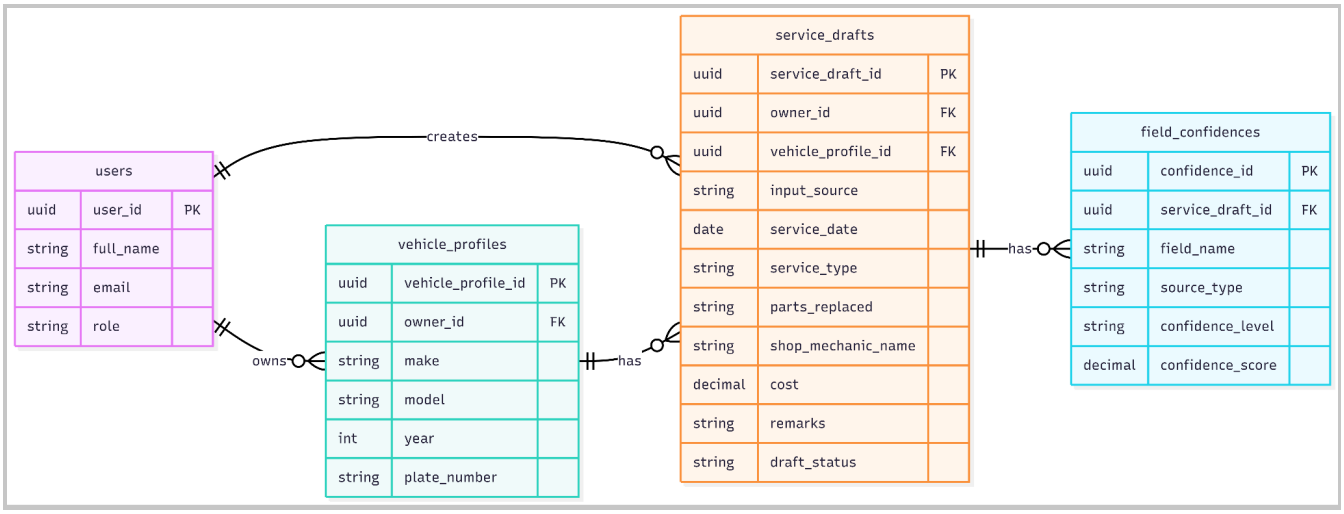

Figure - Entity Relationship Diagram for Review Service Draft

2.2 Identify Missing Required and Flagged Fields

- ***User Interface Design***

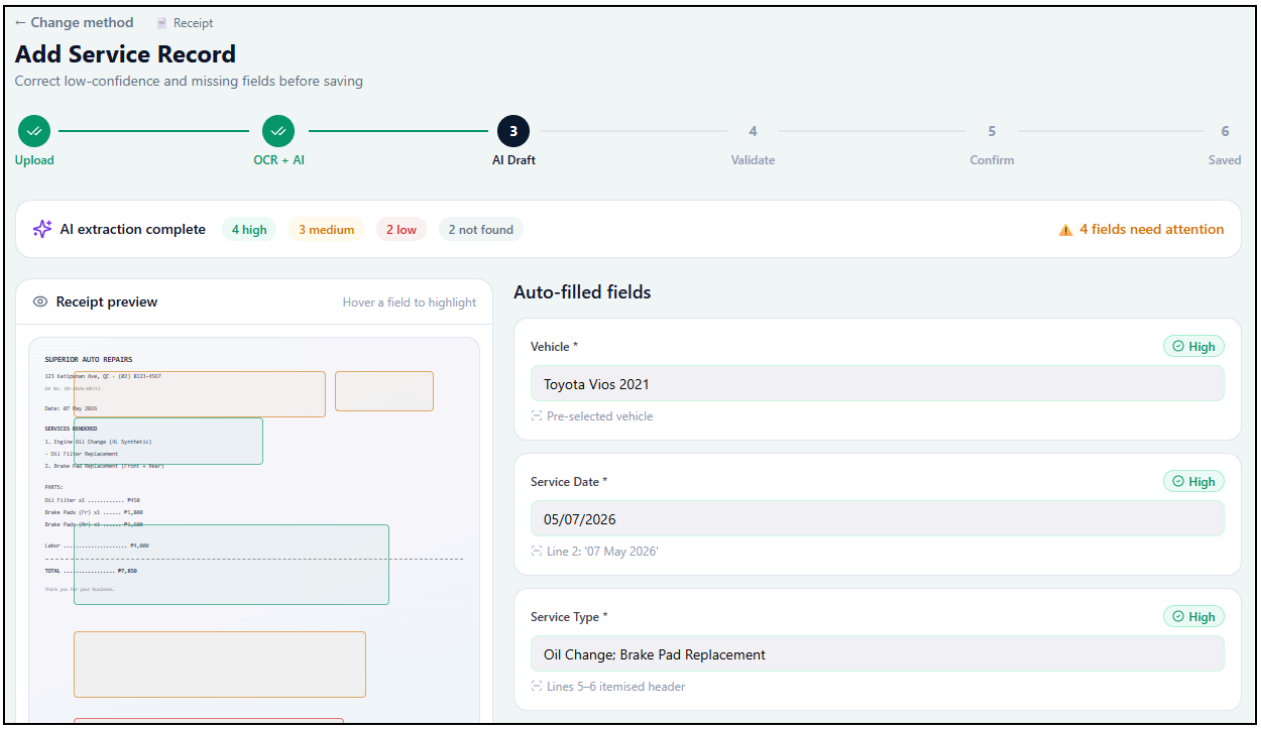

Figure - User Interface Diagram for Identify Missing Required and Flagged Fields

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
DraftValidationPage	Displays validation results for the service draft after the system checks required and flagged fields.	React page component
MissingFieldAlert	Shows required fields that are missing, such as vehicle profile, service date, service type, or cost.	Alert/message component
FlaggedFieldList	Displays fields marked as low-confidence, not found, uncertain, or incomplete.	React list component
FieldValidationIndicator	Highlights each field based on validation status, such as valid, missing, low-confidence, or needs review.	Reusable React UI component
ReturnToEditButton	Allows the owner to return to the editable review form and correct the highlighted fields.	Button/navigation component
ContinueToConfirmationButton	Allows the owner to proceed only when required fields are complete and validation rules are satisfied.	Button/navigation component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ValidationController	Receives the request to check missing required fields and flagged fields in the draft.	Spring Boot REST controller
ServiceDraftValidationService	Applies validation rules for required fields and checks low-confidence or not-found field statuses.	Spring service class
ValidationResult	Represents the result of validation, including missing fields, flagged fields, and whether the draft can proceed.	DTO/value object
FieldValidationRule	Defines required-field and confidence-checking rules used during draft validation.	Helper/service class or rule object
ServiceDraftRepository	Retrieves and updates the validation status of the service draft.	Repository/data access component
ServiceDraft	Represents the draft being checked before confirmation.	Domain model/entity class
FieldConfidence	Provides confidence and source data used to determine whether a field should be flagged.	Domain model/value object

- ***Object-Oriented Components***

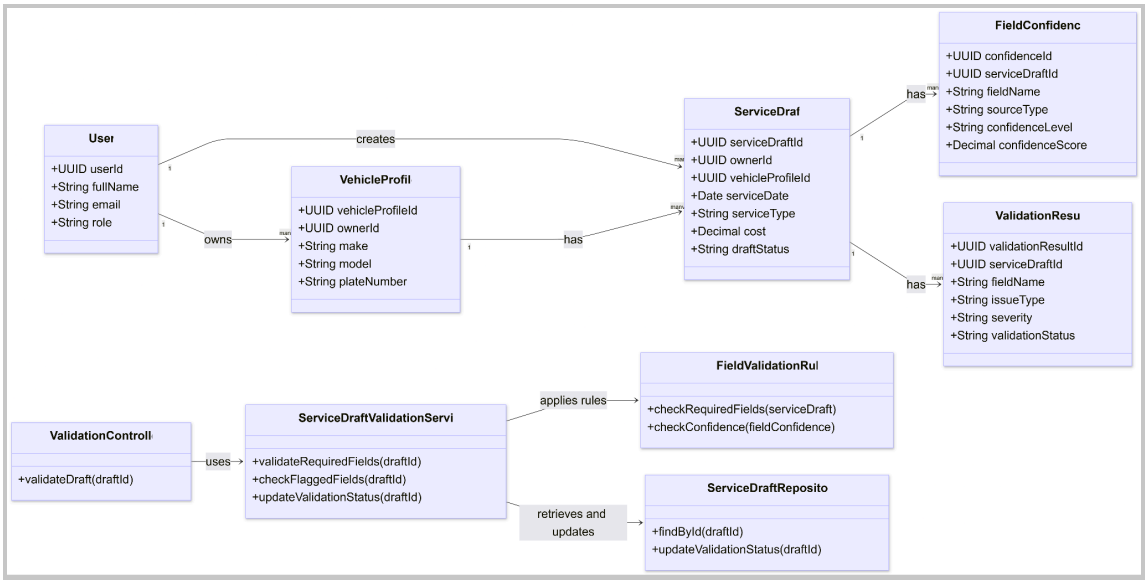

Figure - Class Diagram for Identify Missing Required and Flagged Fields

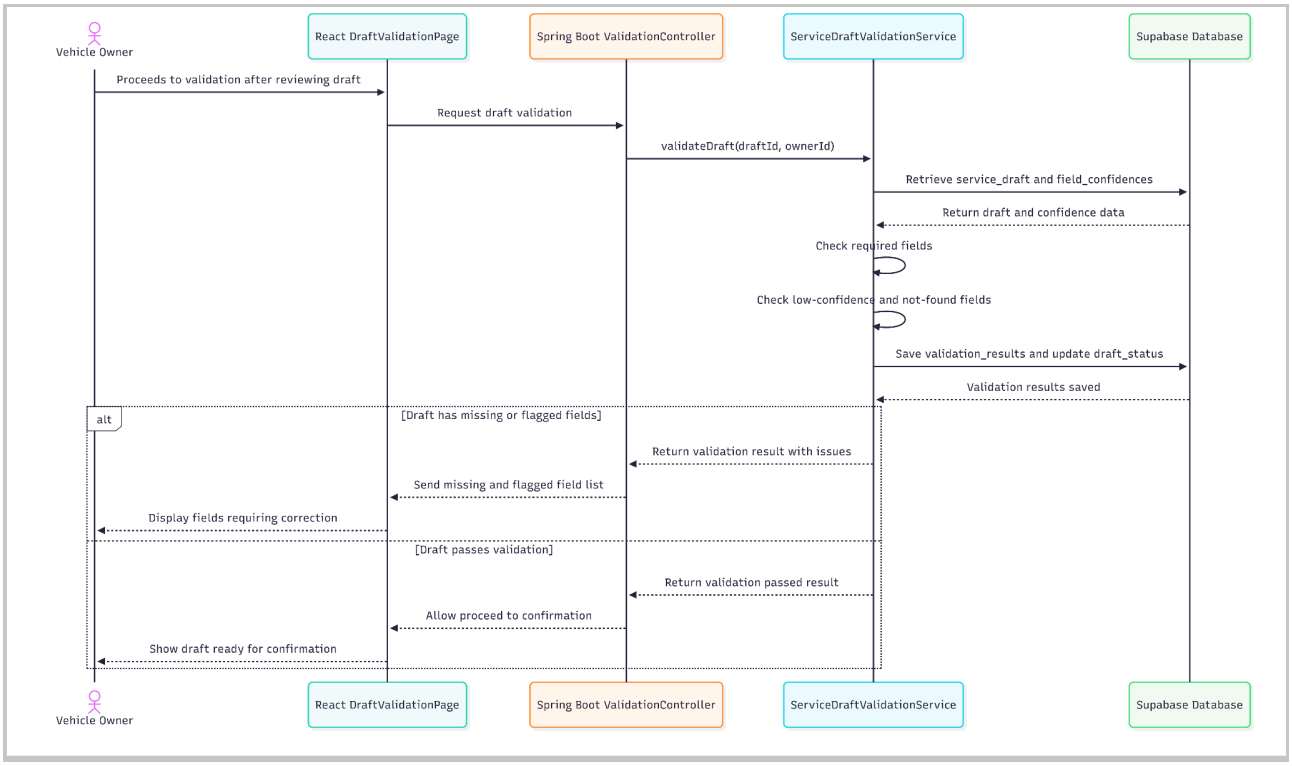

Figure - Sequence Diagram for Identify Missing Required and Flagged Fields

- ***Data Design***

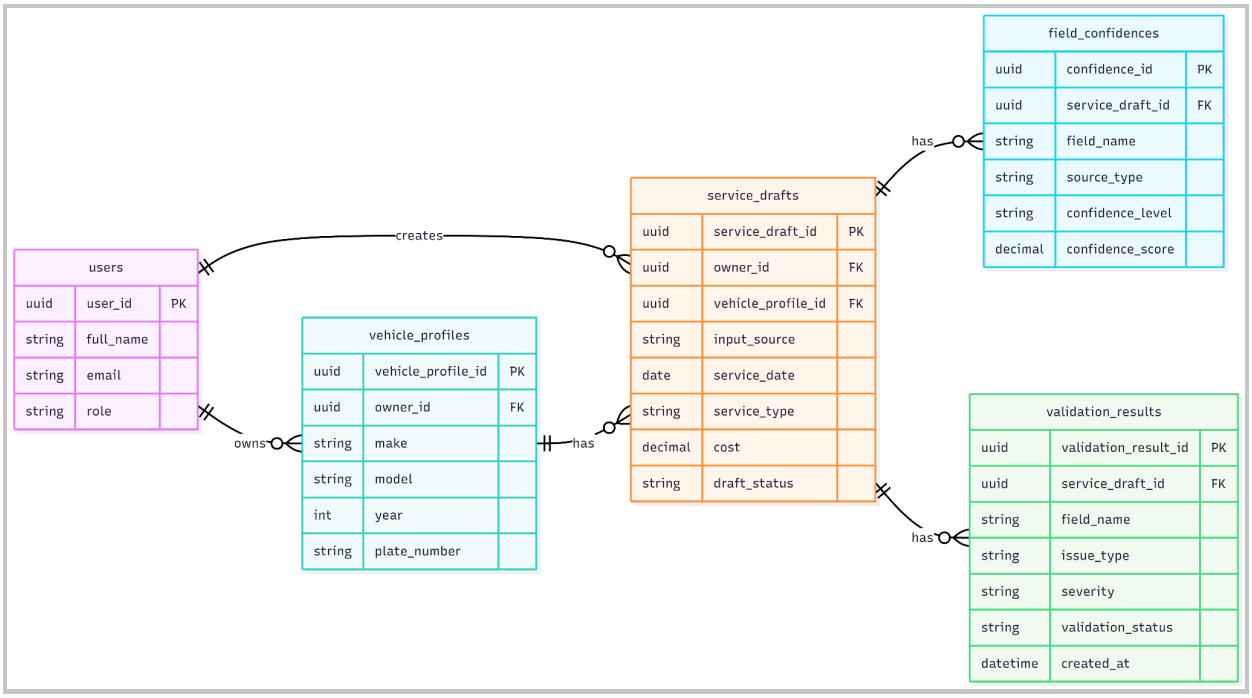

Figure - Entity Relationship Diagram for Identify Missing Required and Flagged Fields

2.3 Correct Flagged or Incomplete Service Details

- ***User Interface Design***

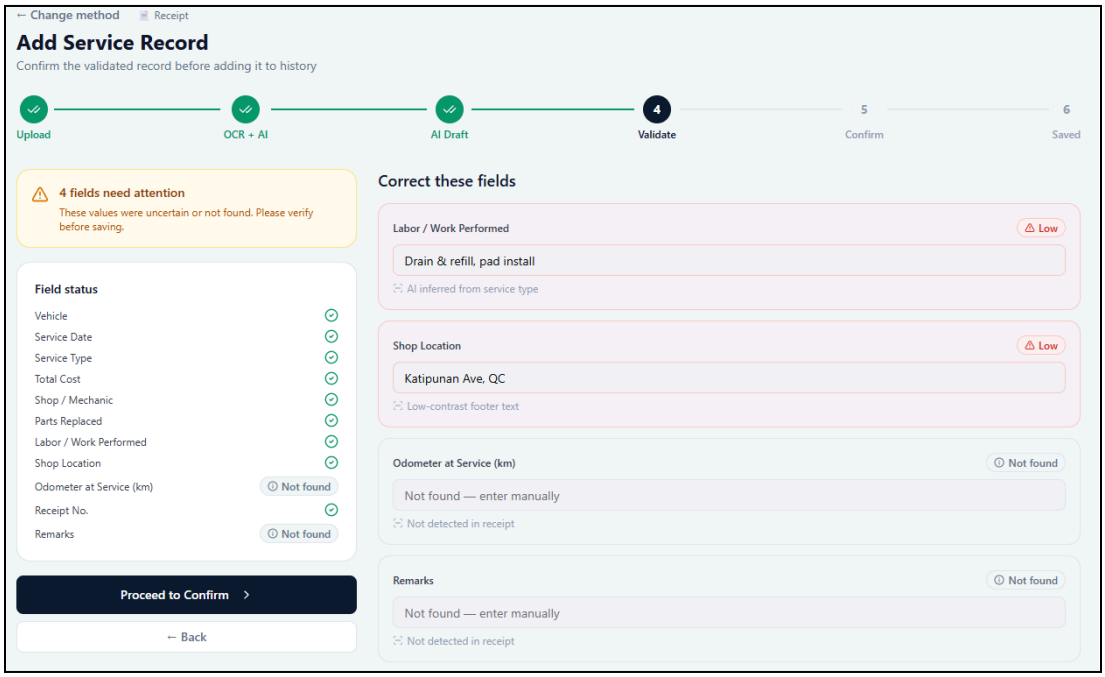

Figure - User Interface Design for Correct Flagged or Incomplete Service Details

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceDraftCorrectionPage	Displays the editable service draft with highlighted fields that require correction.	React page component
CorrectableFieldInput	Allows the owner to edit missing, inaccurate, incomplete, low-confidence, or not-found fields.	Reusable form input component
FieldCorrectionForm	Groups all editable service draft fields and submits corrected values to the backend.	React form component
CorrectionStatusBadge	Shows whether a corrected field is now owner-reviewed or owner-confirmed.	Reusable React UI component
RevalidateDraftButton	Sends the corrected draft back to the system for validation checking.	Button/action component
SaveCorrectionButton	Saves corrected draft values before proceeding to confirmation.	Button/action component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ValidationController	Receives corrected field values from the frontend and returns updated validation results.	Spring Boot REST controller
ServiceDraftValidationService	Rechecks corrected fields and updates their validation status.	Spring service class
ServiceDraftCorrectionService	Applies owner corrections to the service draft and marks corrected values as owner-reviewed or owner-confirmed.	Spring service class
ServiceDraftRepository	Saves updated draft values after correction.	Repository/data access component
ServiceDraft	Represents the service draft being corrected.	Domain model/entity class
FieldConfidence	Updates field status after the owner corrects low-confidence, missing, or not-found fields.	Domain model/value object
ValidationResult	Returns updated validation feedback after correction.	DTO/value object

- ***Object-Oriented Components***

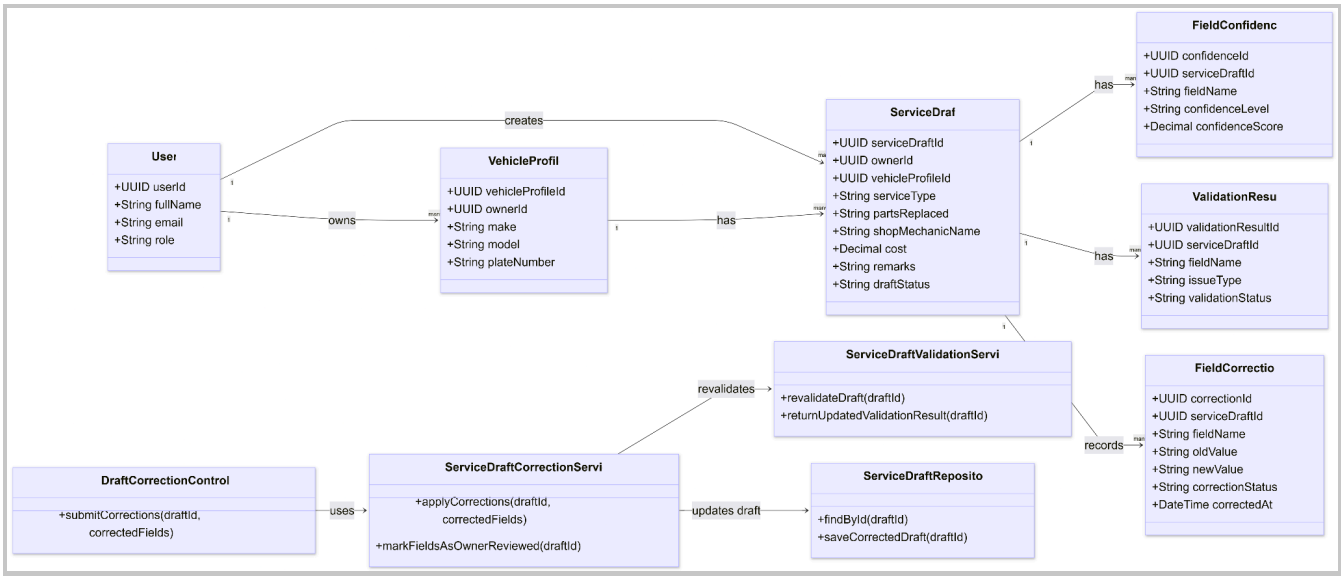

Figure - Class Diagram for Correct Flagged or Incomplete Service Details

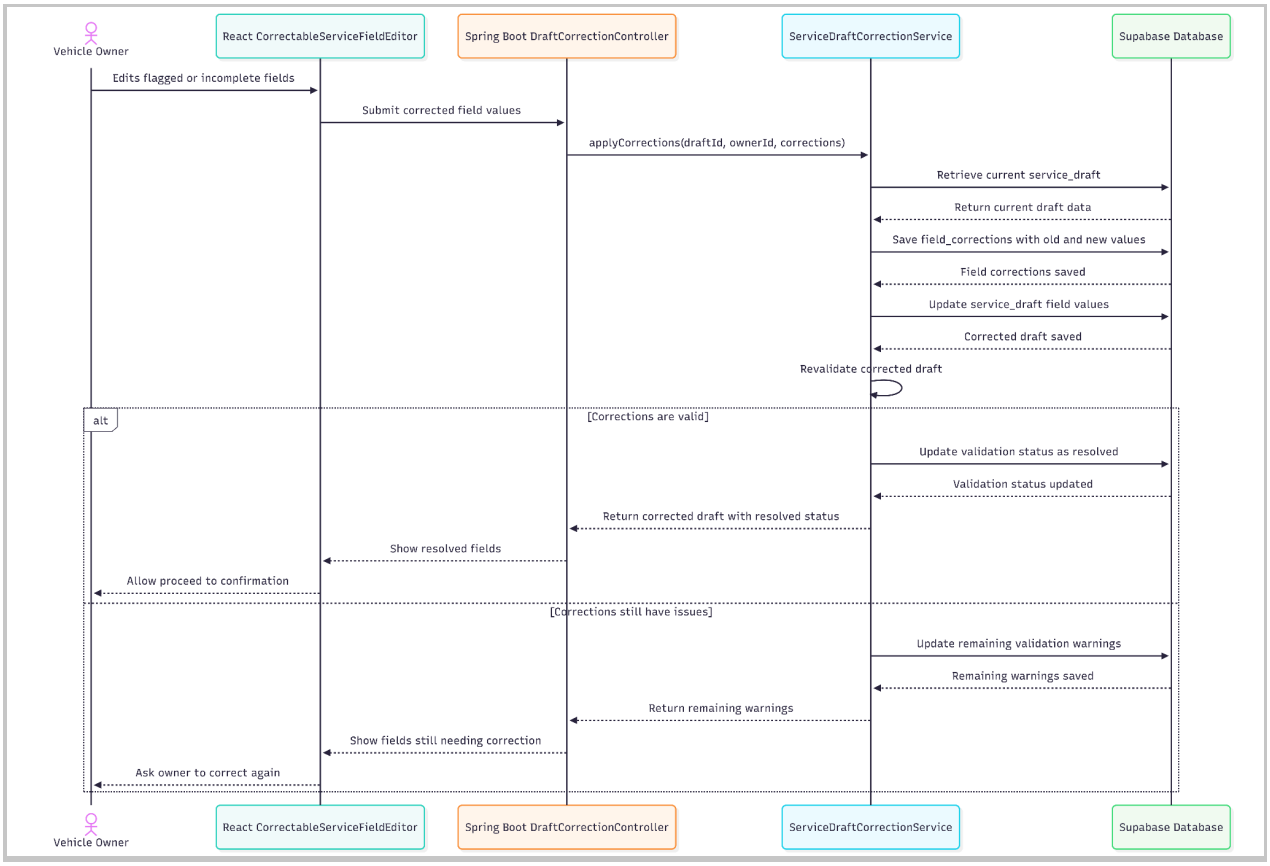

Figure - Sequence Diagram for Correct Flagged or Incomplete Service Details

- ***Data Design***

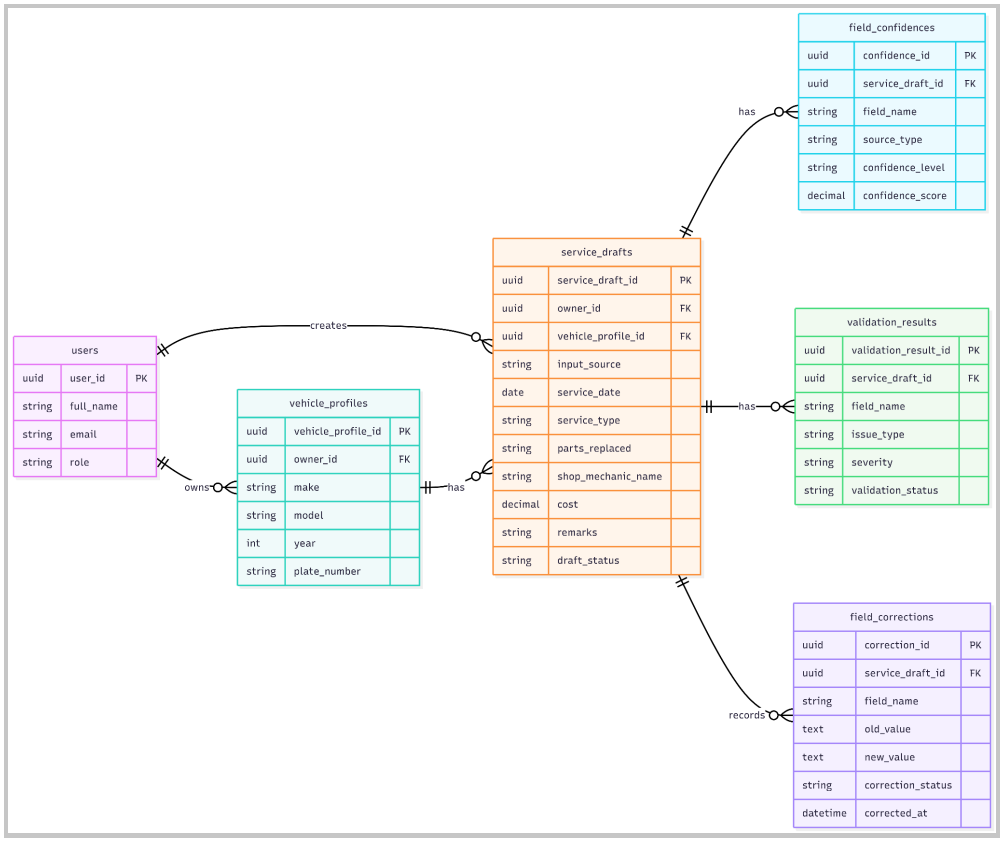

Figure - Entity Relationship Diagram for Correct Flagged or Incomplete Service Details

2.4 Confirm and Save Validated Record

- ***User Interface Design***

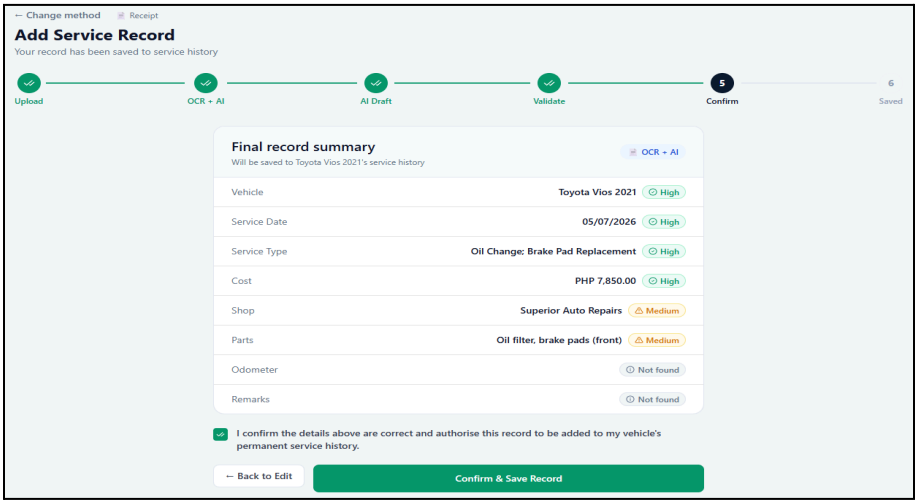

Figure - User Interface Design for Confirm and Save Validated Record

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceRecordConfirmationPage	Displays the final read-only summary of the validated service record before saving.	React page component
FinalServiceRecordSummary	Shows final service details such as vehicle profile, service date, service type, odometer, cost, shop name, parts replaced, labor performed, and remarks.	React UI component
RecordSourceSummary	Shows whether the record came from OCR + AI, Voice + AI, or owner-entered manual input.	React UI component
ConfirmSaveButton	Allows the owner to confirm that the validated record is correct and ready to be saved.	Button/action component
BackToEditButton	Allows the owner to return to the correction screen before saving.	Button/navigation component
SaveStatusMessage	Displays success or failure message after attempting to save the validated service record.	Alert/status component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ValidationController	Receives the final confirmation request from the frontend.	Spring Boot REST controller
ServiceRecordService	Converts a validated ServiceDraft into a saved ServiceRecord.	Spring service class
ServiceDraftValidationService	Performs the final validation check before saving.	Spring service class
ServiceDraftRepository	Retrieves the confirmed service draft and updates its status after saving.	Repository/data access component
ServiceRecordRepository	Stores the finalized validated service record under the selected vehicle profile.	Repository/data access component
VehicleService	Verifies that the vehicle profile belongs to the authenticated owner before saving.	Spring service class
ServiceDraft	Represents the validated draft before it becomes a permanent service record.	Domain model/entity class

ServiceRecord	Represents the final saved service record under the correct vehicle profile.	Domain model/entity class
AuditLogService	Records the save action for traceability and data integrity.	Spring service class

- ***Object-Oriented Components***

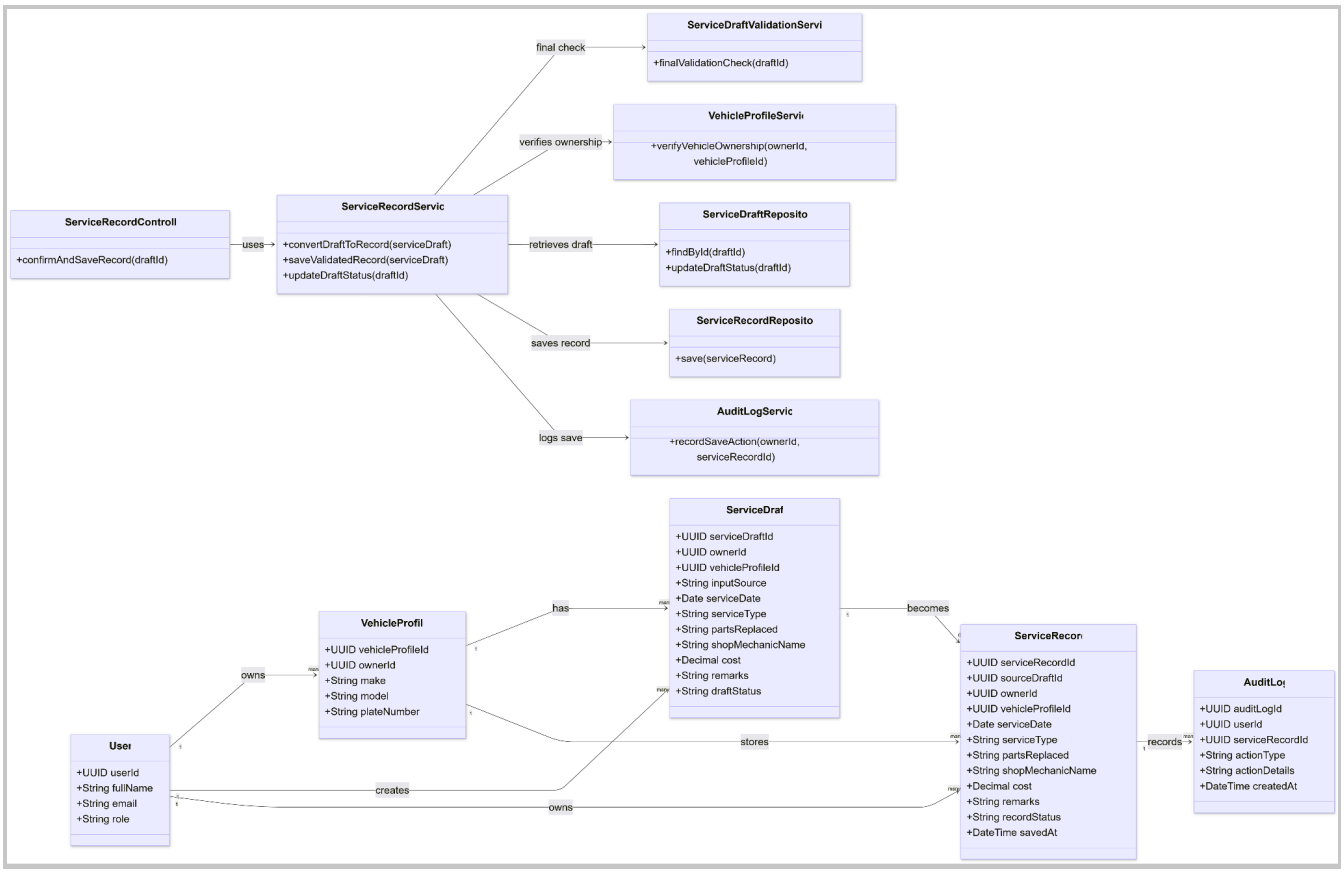

Figure - Class Diagram for Confirm and Save Validated Record

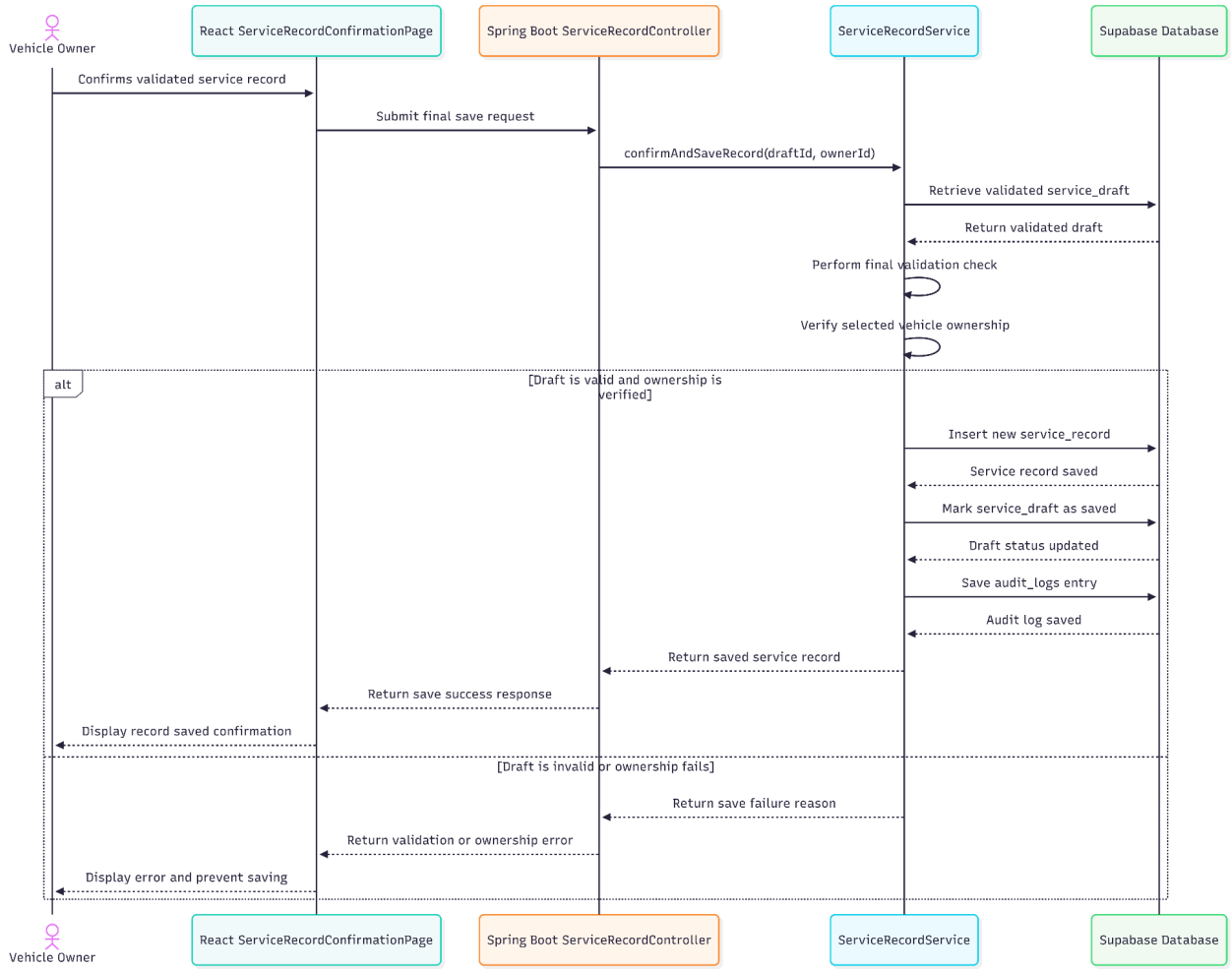

Figure - Sequence Diagram for Confirm and Save Validated Record

- ***Data Design***

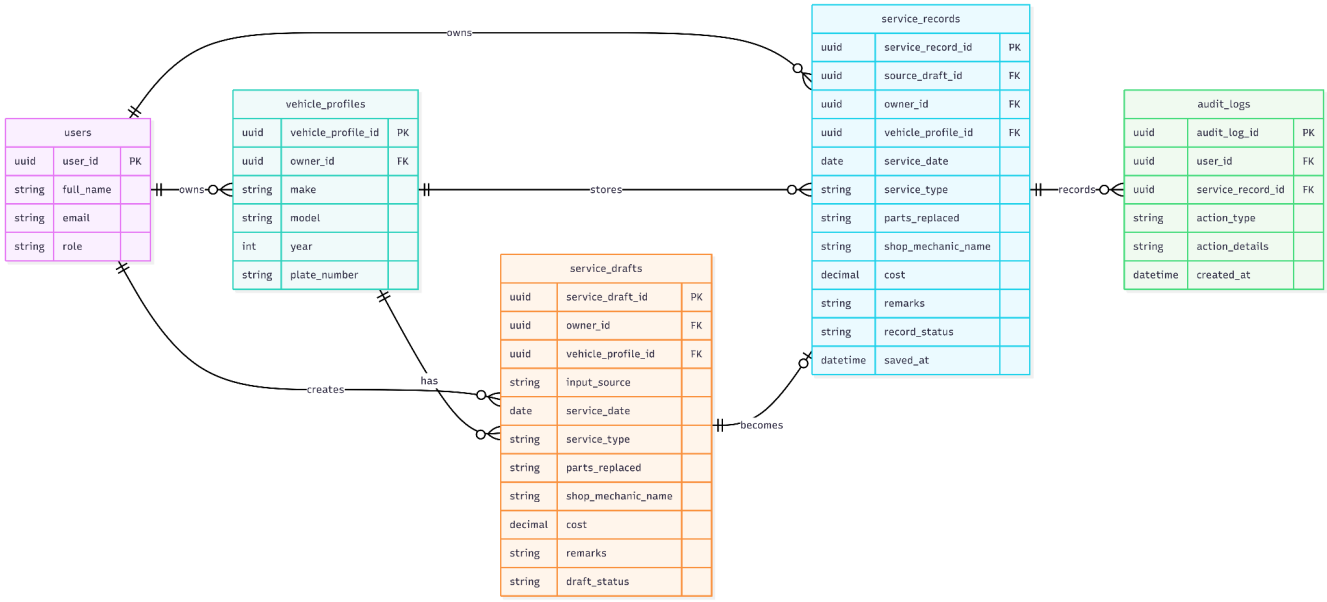

Figure - Entity Relationship Diagram for Confirm and Save Validated Record

Module 3: Unified Vehicle Service History Consolidation

3.1 Link Validated Record to Vehicle Profile

- ***User Interface Design***

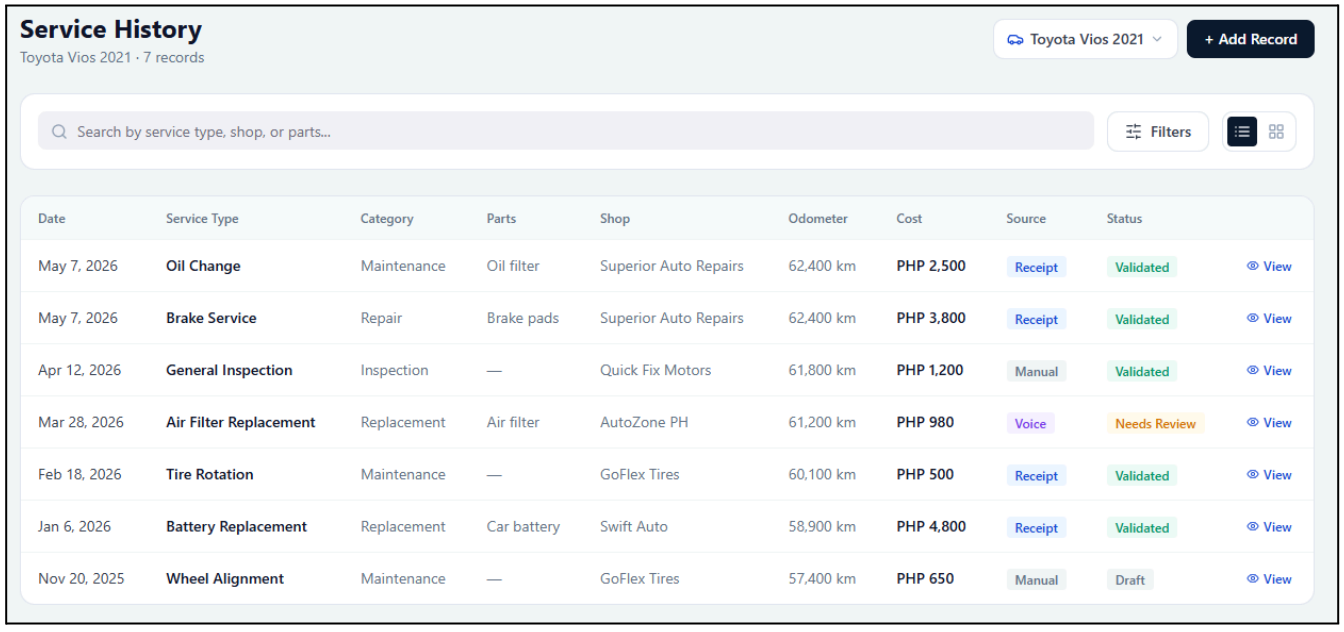

Figure - User Interface for Link Validated Record to Vehicle Profile

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
VehicleRecordLinkSummaryPage	Shows the validated service record together with the selected vehicle before the record is saved to the vehicle history.	React page component
SelectedVehicleContextCard	Displays selected vehicle details such as make, model, year, plate number, nickname, and odometer so the owner can confirm the correct vehicle.	Reusable React UI component
ValidatedRecordPreviewCard	Shows the service date, service type, cost, shop/mechanic name, parts replaced, and remarks from the validated draft before linking.	Reusable React preview component
ChangeVehicleSelectionModal	Allows the owner to switch to another owned vehicle if the currently selected vehicle is incorrect before saving.	React modal component
ConfirmLinkRecordButton	Submits the owner confirmation that the validated record should be linked to the selected vehicle profile.	Button/action component
RecordLinkStatusMessage	Shows loading, success, retry, or error messages when the system links the validated record to the vehicle profile.	Frontend feedback/status component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceRecordController	Receives the API request to confirm a validated draft and link it to the selected registered vehicle profile.	Spring Boot REST controller
ServiceHistoryService	Handles the rule that a saved service record must be stored under the correct vehicle profile and included in that vehicle's unified history.	Spring service class
VehicleService	Verifies that the selected vehicle profile exists and belongs to the authenticated vehicle owner before linking the record.	Spring service class
ServiceRecordRepository	Stores the linked service record in Supabase and retrieves saved records by vehicle profile.	Repository/data access component
VehicleRepository	Retrieves the selected vehicle profile from Supabase and supports ownership verification.	Repository/data access component
ServiceRecord	Represents the validated maintenance or repair record that is saved under a vehicle profile.	Domain model/entity class

VehicleProfile	Represents the registered vehicle profile that owns one or more service records.	Domain model/entity class
AuditLogService	Records important save/link actions for traceability when vehicle history data is created or updated.	Spring service class

- ***Object-Oriented Components***

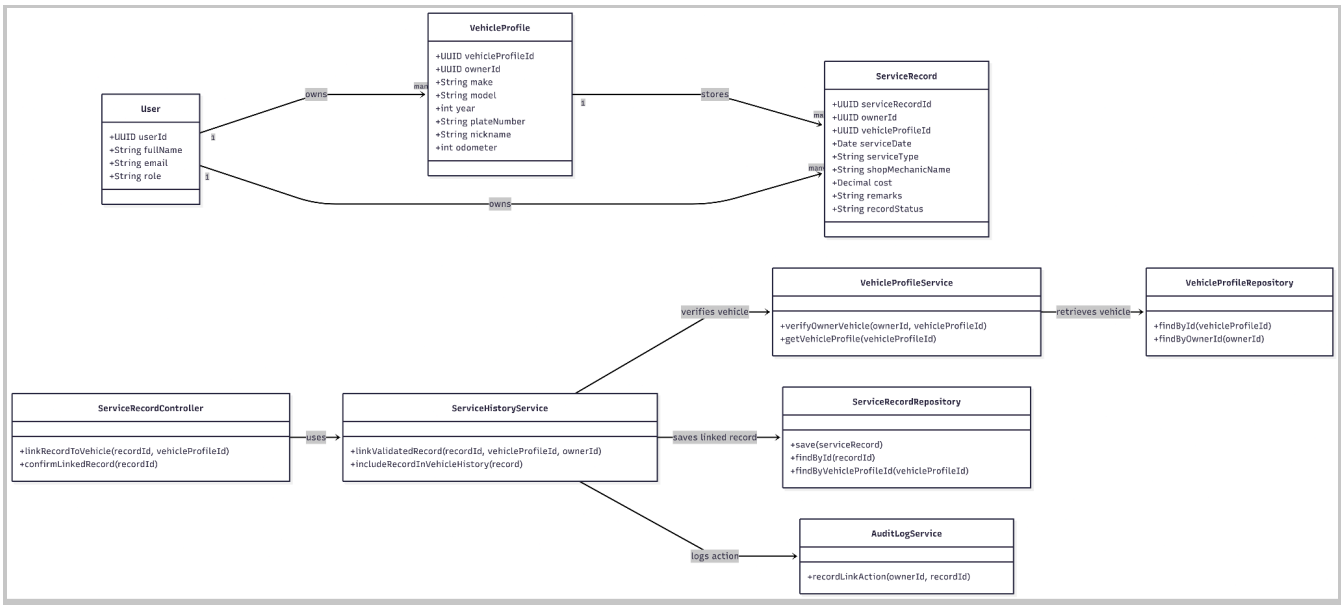

Figure - Class Diagram for Link Validated Record to Vehicle Profile

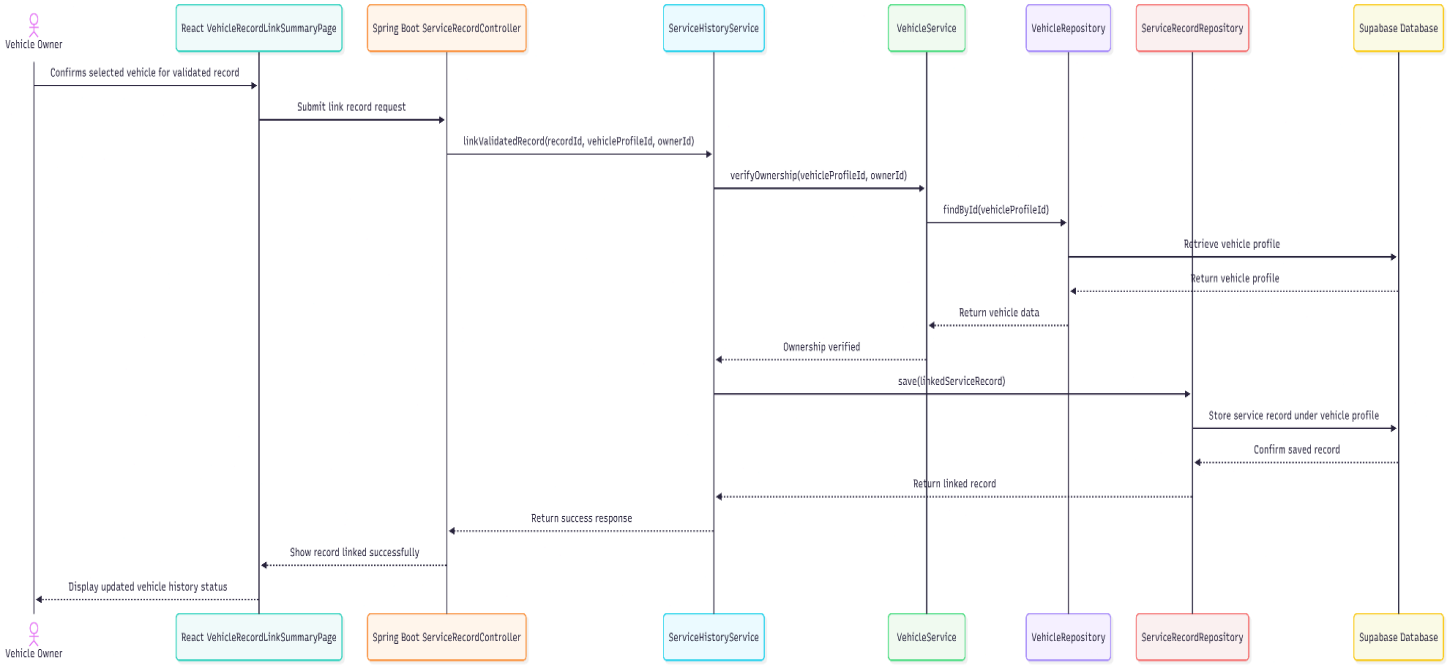

Figure - Sequence Diagram for Link Validated Record to Vehicle Profile

- ***Data Design***

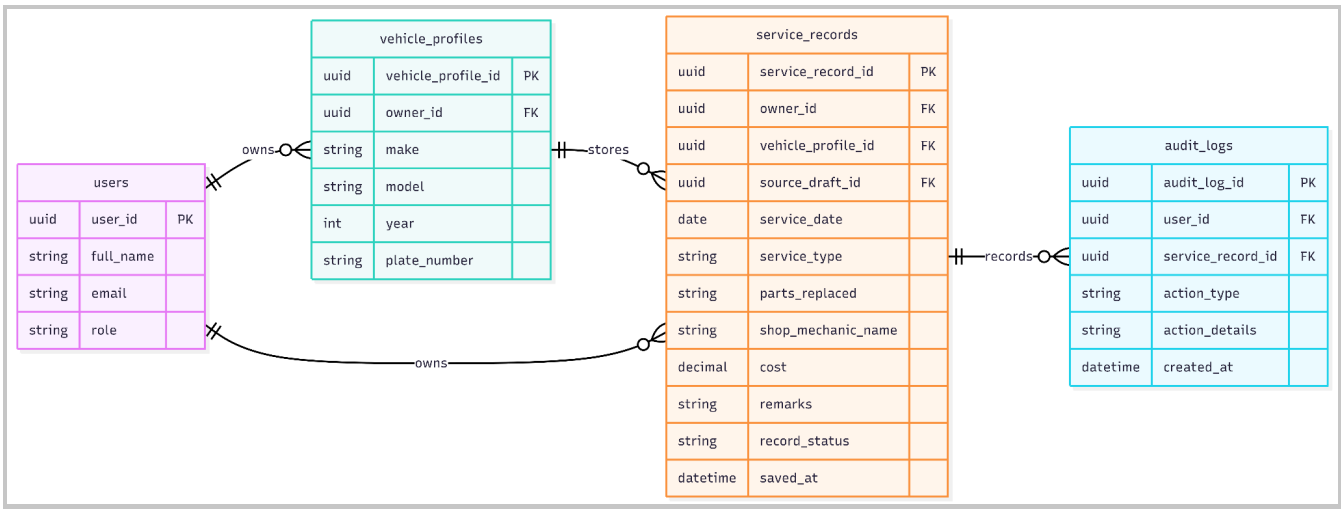

Figure - Entity Relationship Diagram for Link Validated Record to Vehicle Profile

3.2 Organize Records Chronologically

- ***User Interface Design***

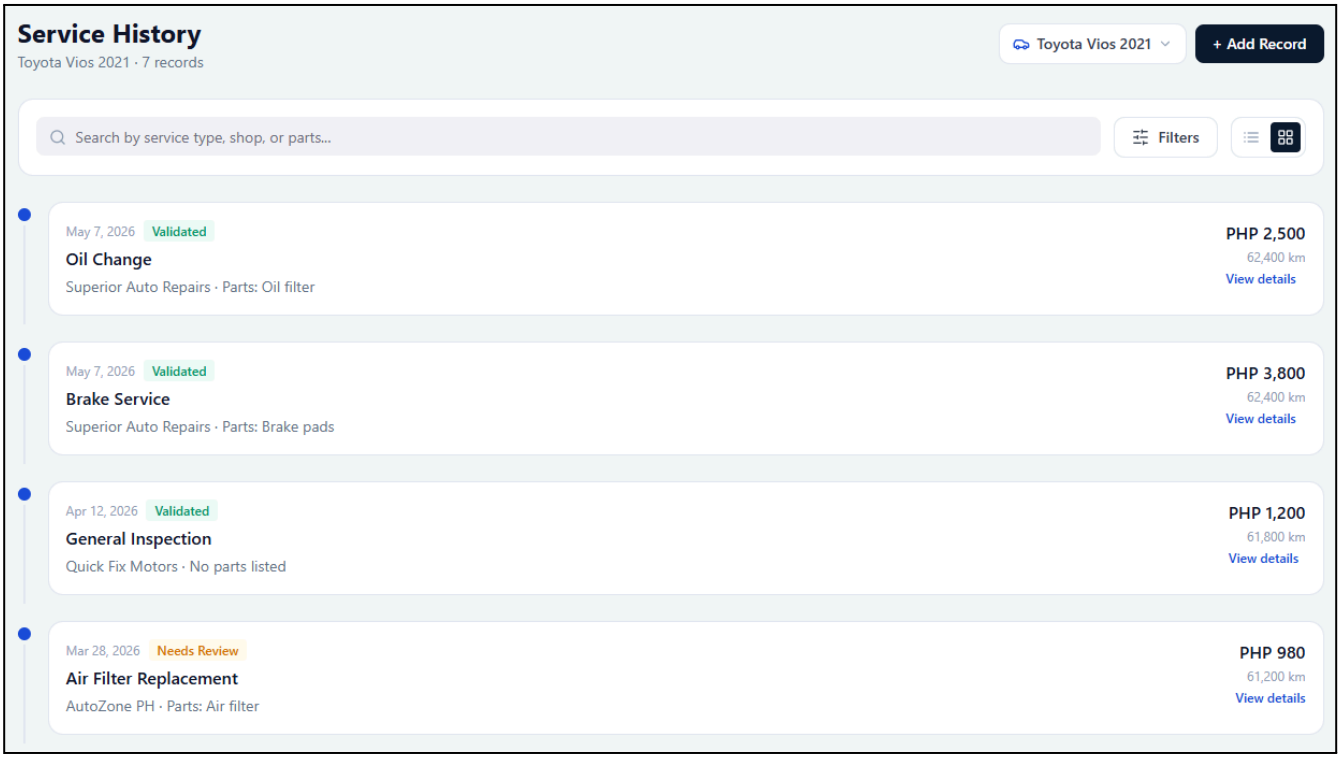

Figure - User Interface Design for Organize Records Chronologically

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceTimelineView	Displays saved service records for the selected vehicle in date order so the owner can scan the maintenance timeline easily.	React page/section component
ServiceRecordTimelineItem	Shows one service record in the timeline with date, service type, cost, shop/mechanic name, and short summary.	Reusable React UI component
DateGroupHeader	Groups records by month, year, or service date to make the chronological history easier to read.	React display component
SortOrderToggle	Allows the owner to switch between latest-first and oldest-first chronological ordering when viewing history.	React control component
HistoryPaginationControl	Loads additional records when the vehicle has many saved service records.	Pagination/load-more component
TimelineEmptyState	Displays a clear message when the selected vehicle has no saved service records yet.	Frontend empty-state component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
HistoryController	Receives API requests for the selected vehicle's service history and returns records in chronological order.	Spring Boot REST controller
ServiceHistoryService	Retrieves service records and sorts them by service date, using saved date or updated date as a secondary order when needed.	Spring service class
VehicleService	Checks that the authenticated user is allowed to view the selected vehicle history before records are returned.	Spring service class
ServiceRecordRepository	Queries Supabase for service records belonging to a selected vehicle profile and supports ordering, limits, and pagination.	Repository/data access component
VehicleRepository	Retrieves vehicle profile information needed to scope the history request to the correct vehicle.	Repository/data access component
ServiceRecord	Stores service date, service type, cost, source, created date, and updated date used for chronological organization.	Domain model/entity class
VehicleProfile	Identifies the vehicle whose service records are being organized and displayed.	Domain model/entity class

- ***Object-Oriented Components***

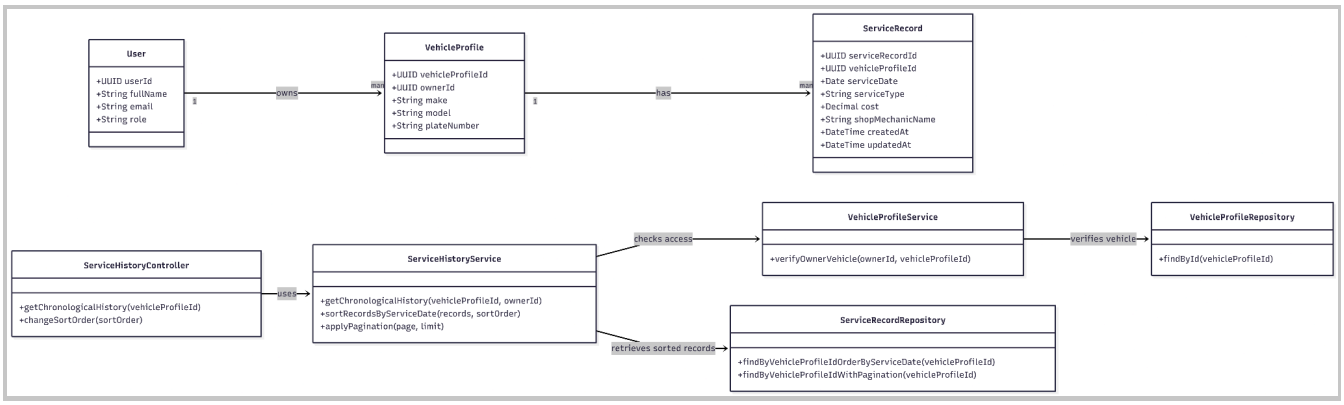

Figure - Class Diagram for Organize Records Chronologically

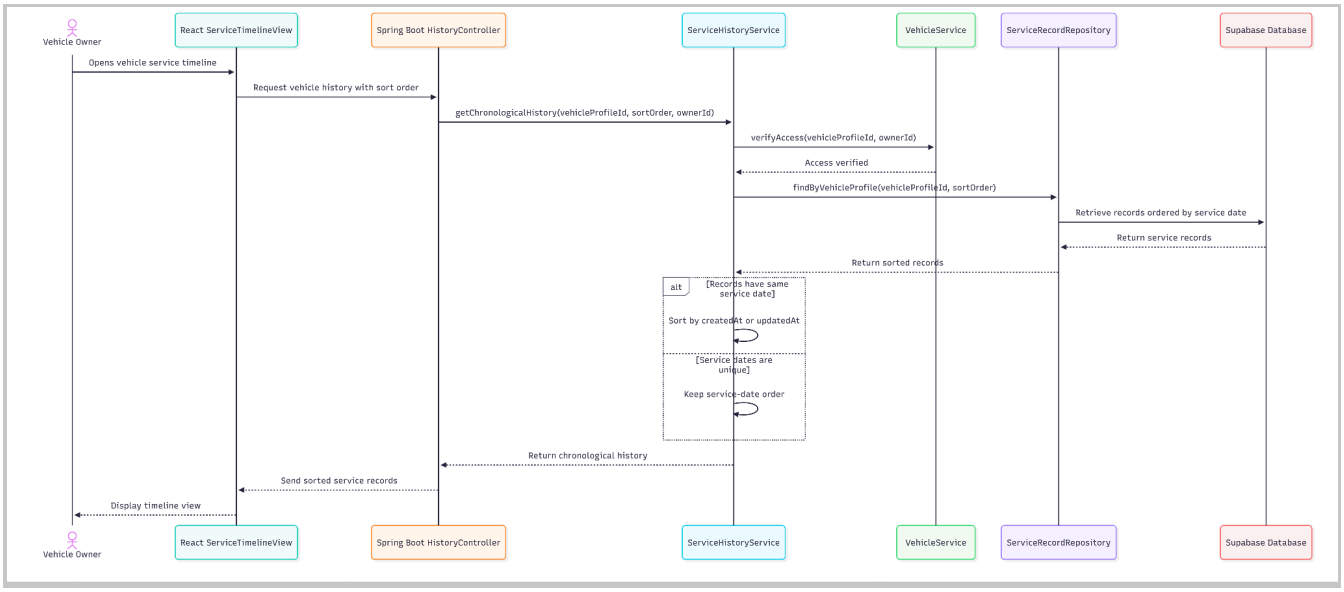

Figure - Sequence Diagram for Organize Records Chronologically

- ***Data Design***

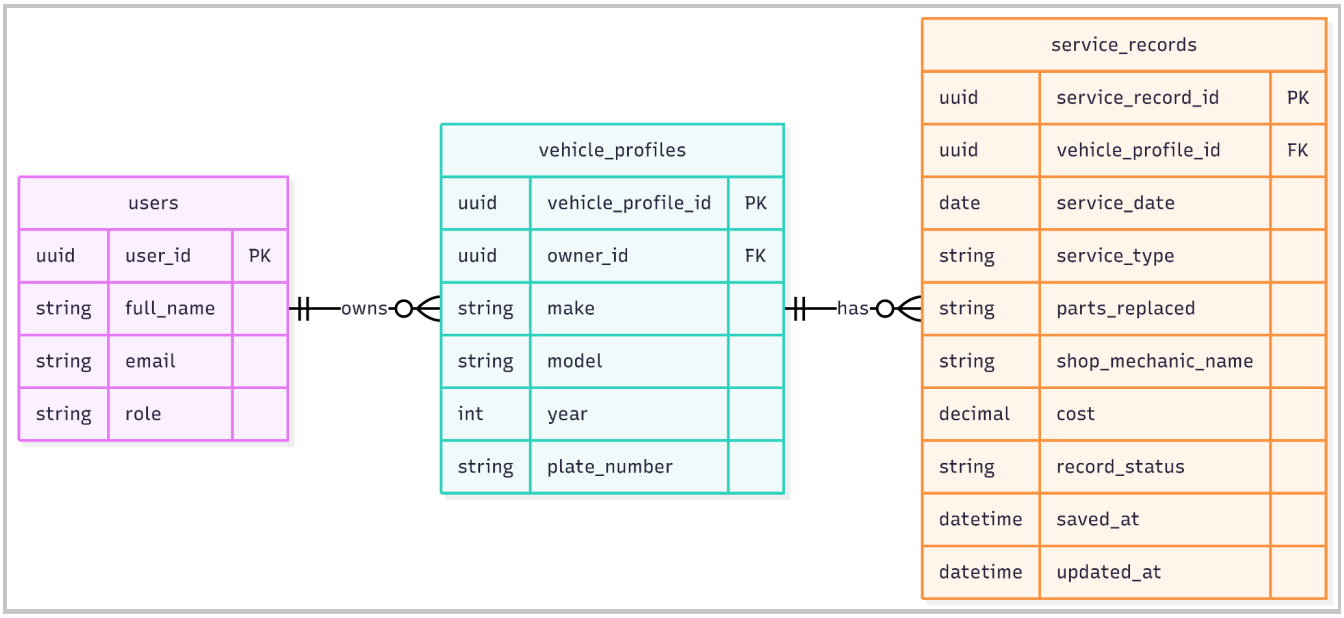

Figure - Entity Relationship Diagram for Organize Records Chronologically

3.3 Categorize Service Records

- ***User Interface Design***

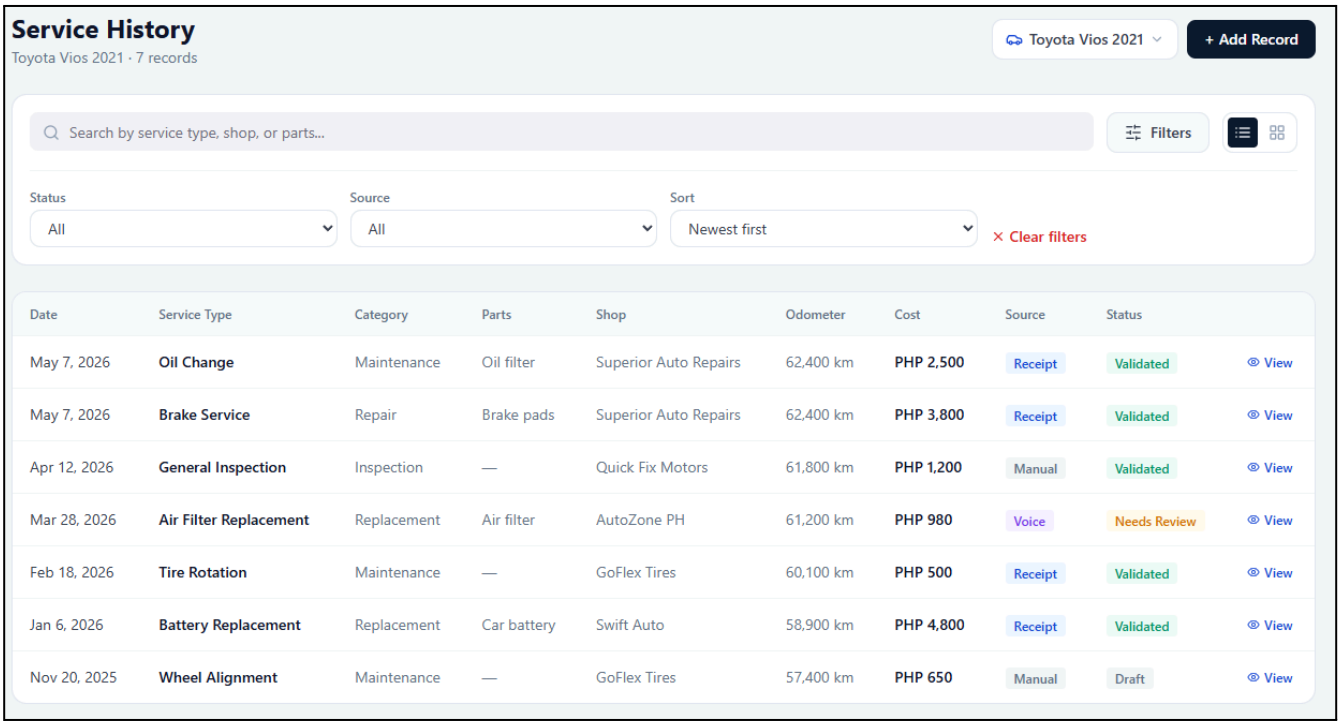

Figure - User Interface Design for Categorize Service Records

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceCategoryFilterPanel	Displays filter controls for browsing records by service type, parts replaced, cost range, and shop/mechanic details.	React filter panel component
ServiceTypeDropdown	Lets the owner filter service history by categories such as maintenance, repair, oil change, brake service, tire service, and other service types.	React dropdown component
CategoryChipList	Shows active category filters as removable chips so the owner can see which filters are currently applied.	Reusable React UI component
PartsSearchInput	Allows the owner to search or filter records based on replaced parts or service keywords.	React search input component
ShopMechanicFilter	Filters service records by shop name, mechanic name, or related service provider details when available.	React filter component
FilteredHistoryResultList	Displays the history records that match the selected categories and search filters.	React list/results component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
HistoryController	Receives category filter and search parameters from the frontend and returns matching service history records.	Spring Boot REST controller
ServiceHistoryService	Applies category and filter rules using fields such as service type, parts replaced, cost, and shop/mechanic details.	Spring service class
ServiceRecordRepository	Queries Supabase for records that match the selected category filters and search conditions.	Repository/data access component
VehicleService	Ensures category searches are scoped only to vehicle profiles owned by the authenticated user.	Spring service class
ServiceRecord	Contains category-related fields such as service type, parts replaced, cost, shop/mechanic name, and remarks.	Domain model/entity class
VehicleProfile	Scopes categorized service records under the correct registered vehicle.	Domain model/entity class
AuditLogService	Optionally logs history filtering/search activity when audit tracking is required by the system.	Spring service class

- ***Object-Oriented Components***

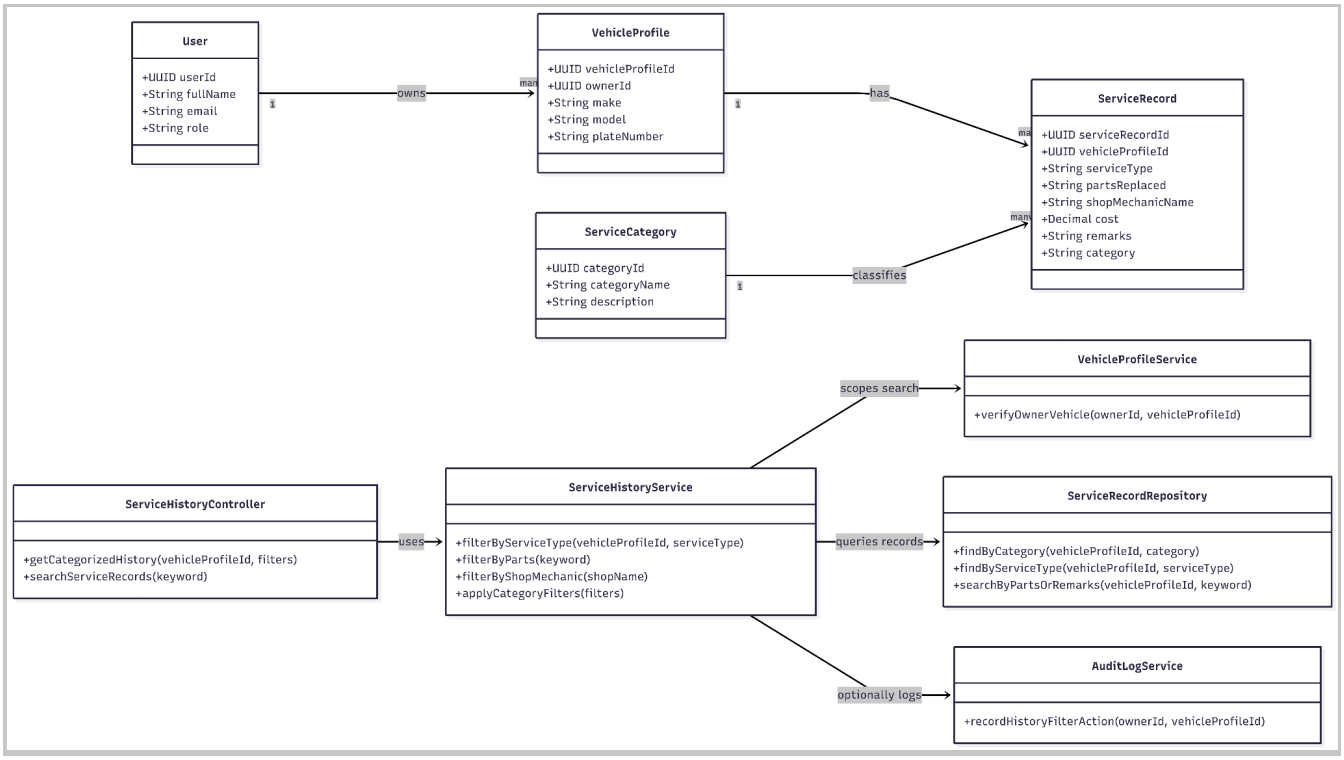

Figure - Class Diagram for Categorize Service Records

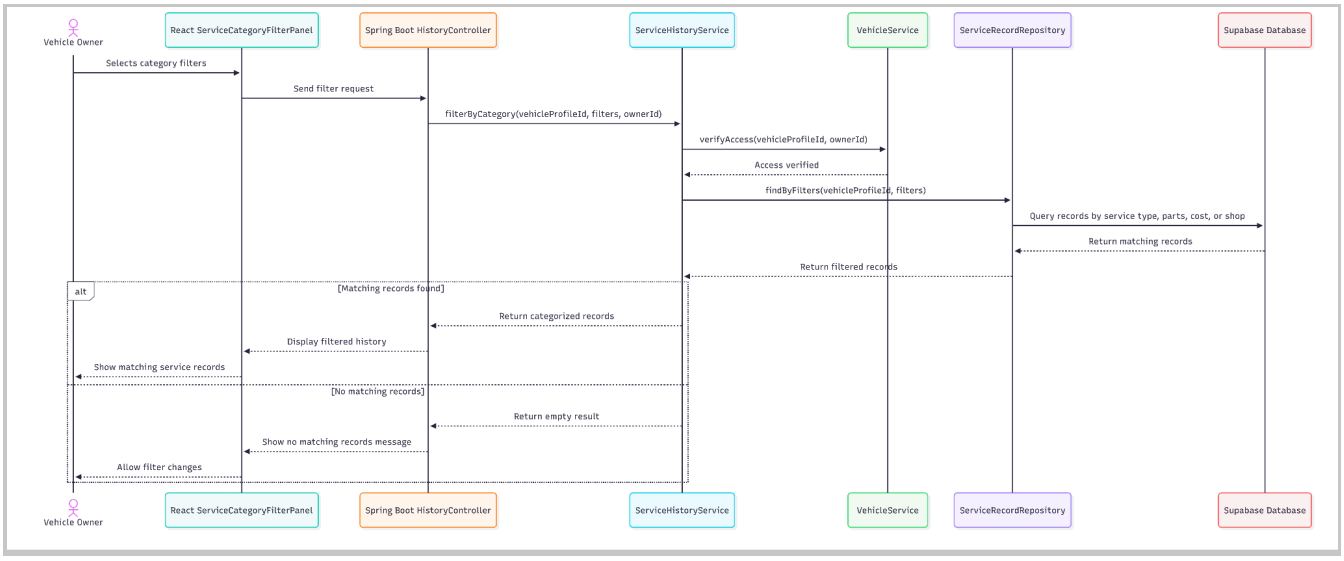

Figure - Sequence Diagram for Categorize Service Records

- ***Data Design***

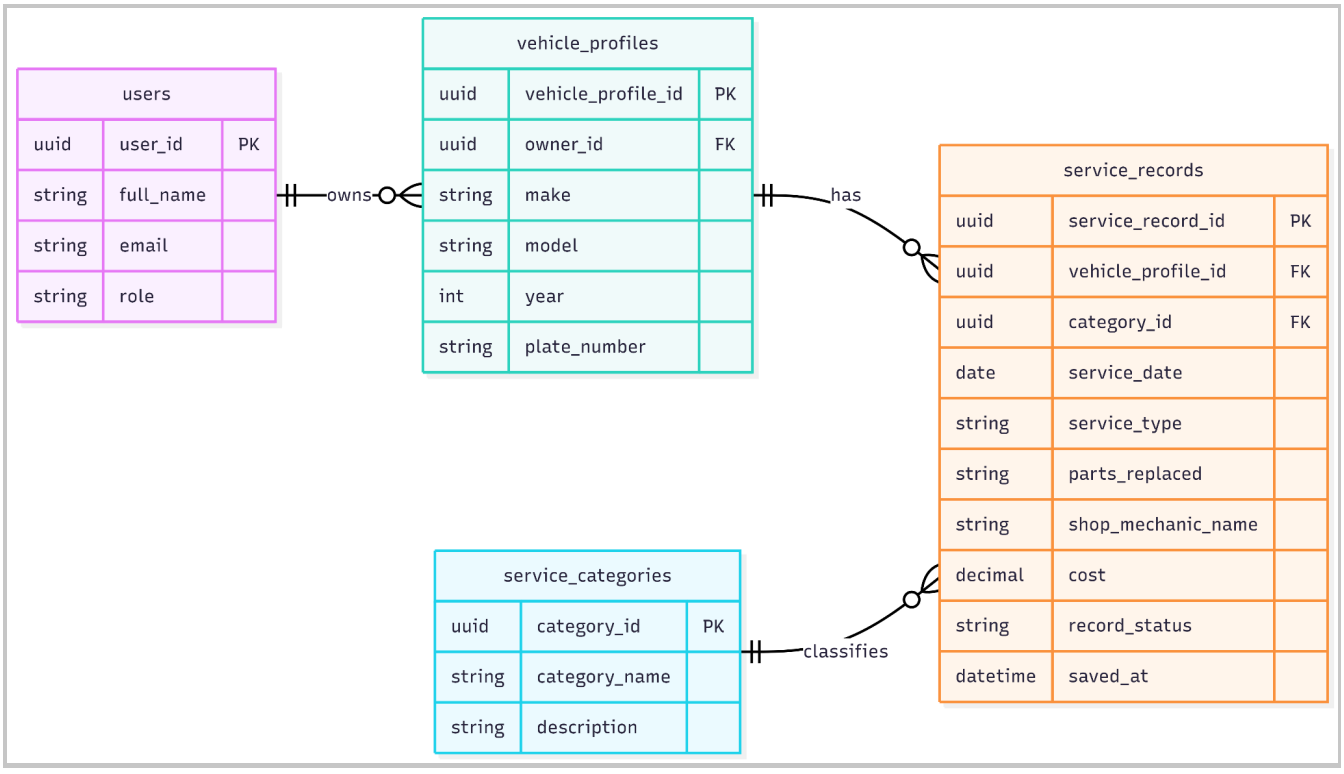

Figure - Entity Relationship Diagram for Categorize Service Records

3.4 View Unified Vehicle Service History

- ***User Interface Design***

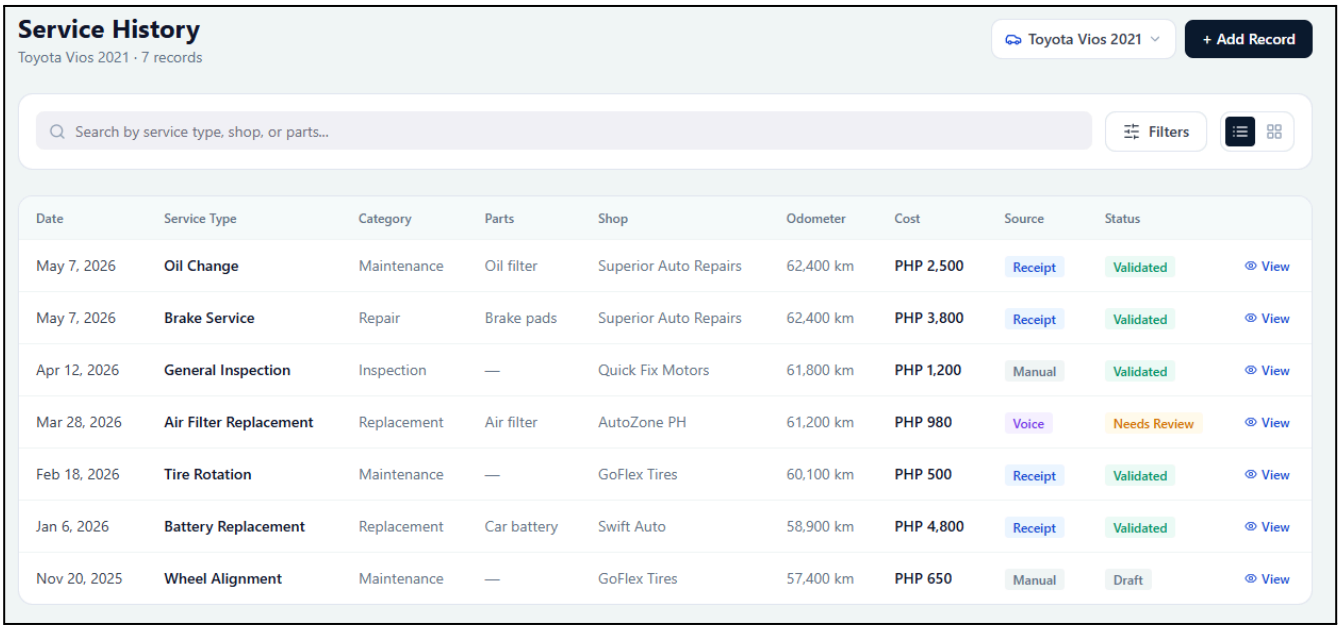

Figure - User Interface Design for View Unified Vehicle Service History

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
VehicleServiceHistoryPage	Shows the complete validated maintenance and repair history of the selected registered vehicle in one centralized page.	React page component
VehicleHistoryHeader	Displays the selected vehicle identity and high-level history information at the top of the page.	React header component
UnifiedServiceRecordList	Lists all validated service records for the selected vehicle using the current sort and filter settings.	React list component
ServiceRecordDetailDrawer	Opens a detailed view of a selected service record without leaving the history page.	React drawer/detail component
HistoryFilterToolbar	Combines search, category filters, date sorting, and clear-filter actions for the unified service history.	React toolbar component
HistoryLoadingErrorState	Handles loading, empty history, access denied, and retry states when service history cannot be loaded.	Frontend state component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
HistoryController	Provides the API endpoint for loading the complete validated service history of a selected vehicle.	Spring Boot REST controller
ServiceHistoryService	Combines ownership checks, record retrieval, chronological ordering, filtering, and pagination for the unified history view.	Spring service class
VehicleService	Validates that the selected vehicle profile belongs to the authenticated owner before history data is returned.	Spring service class
ServiceRecordRepository	Retrieves validated service records from Supabase for the selected vehicle profile.	Repository/data access component
VehicleRepository	Retrieves vehicle details needed for the history page header and access checking.	Repository/data access component
ServiceRecord	Represents each validated maintenance or repair record displayed in the unified history.	Domain model/entity class
VehicleProfile	Represents the registered vehicle whose complete history is being viewed.	Domain model/entity class
AuditLogService	Logs service history access events when user activity needs to be traceable.	Spring service class

- ***Object-Oriented Components***

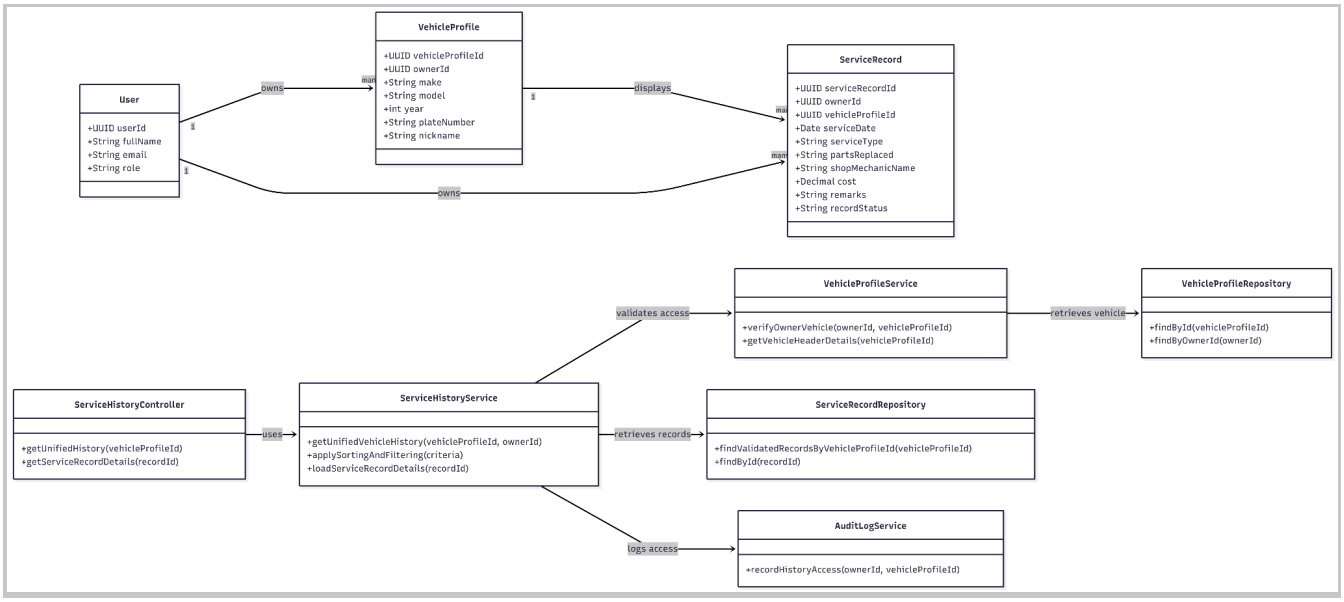

Figure - Class Diagram for View Unified Vehicle Service History

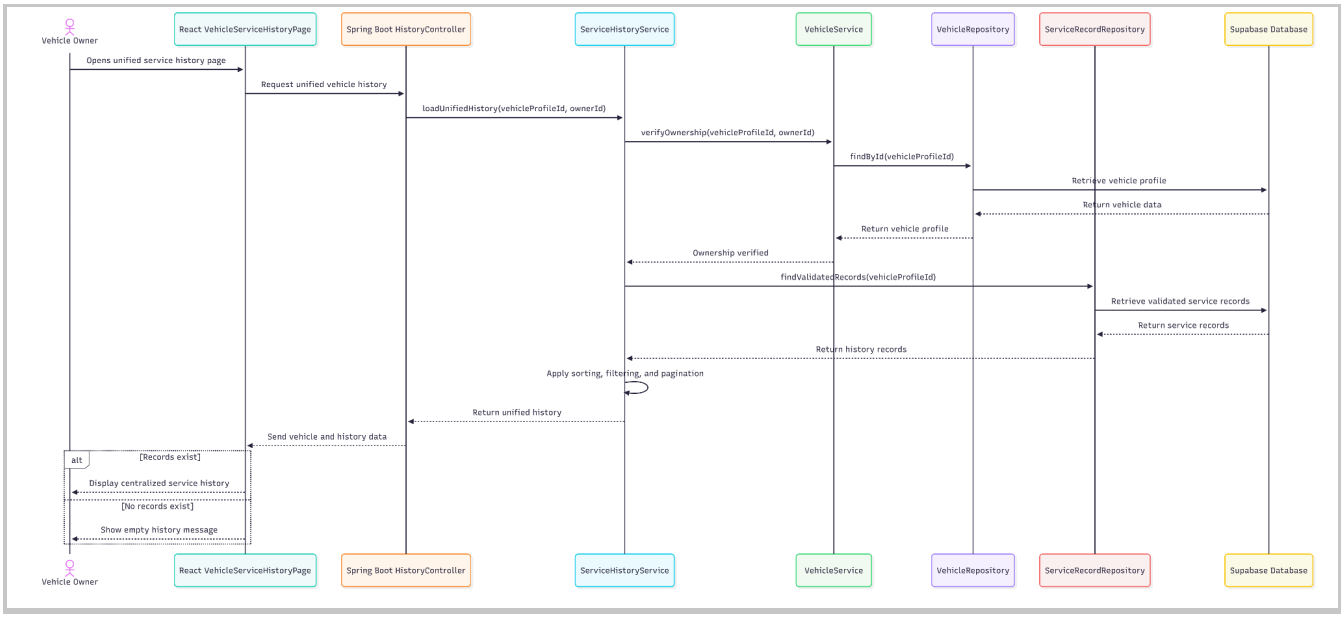

Figure - Sequence Diagram for View Unified Vehicle Service History

- ***Data Design***

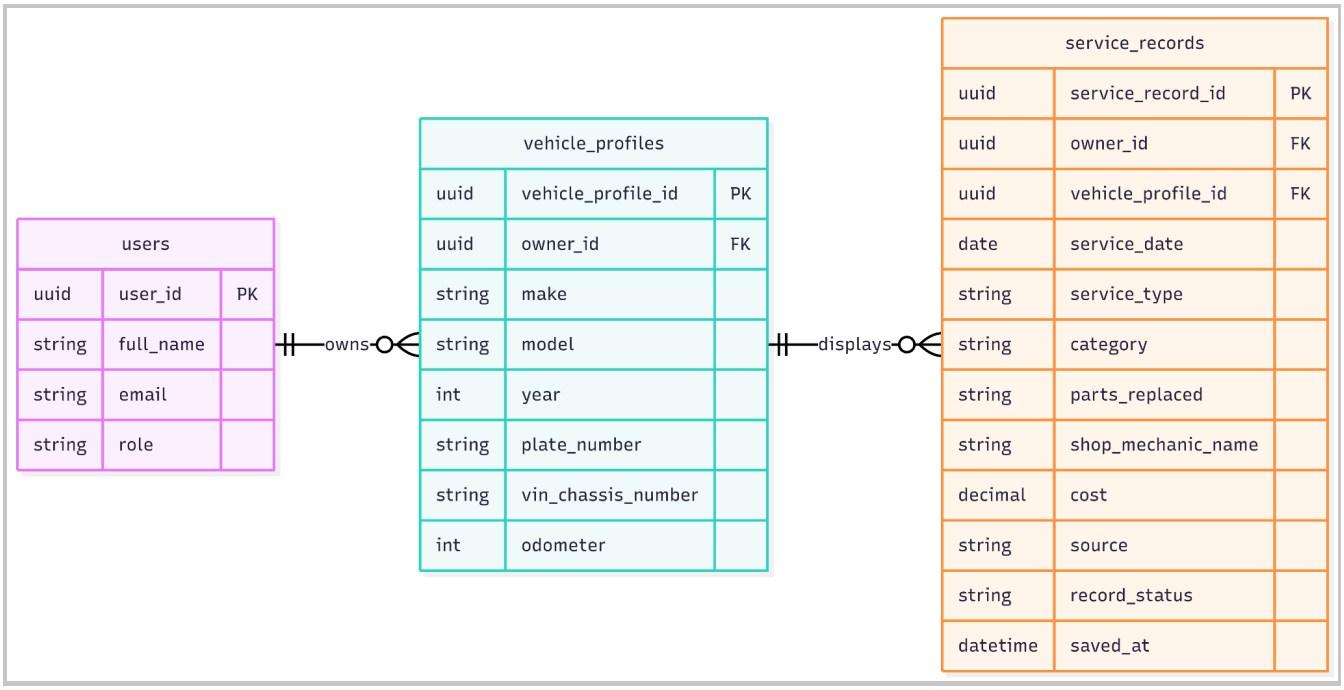

Figure - Entity Relationship Diagram for Categorize Service Records

Module 4: AI-Assisted Service Understanding and Mechanic Handoff

4.1 Show AI-Generated Service Explanation

- **User Interface Design**

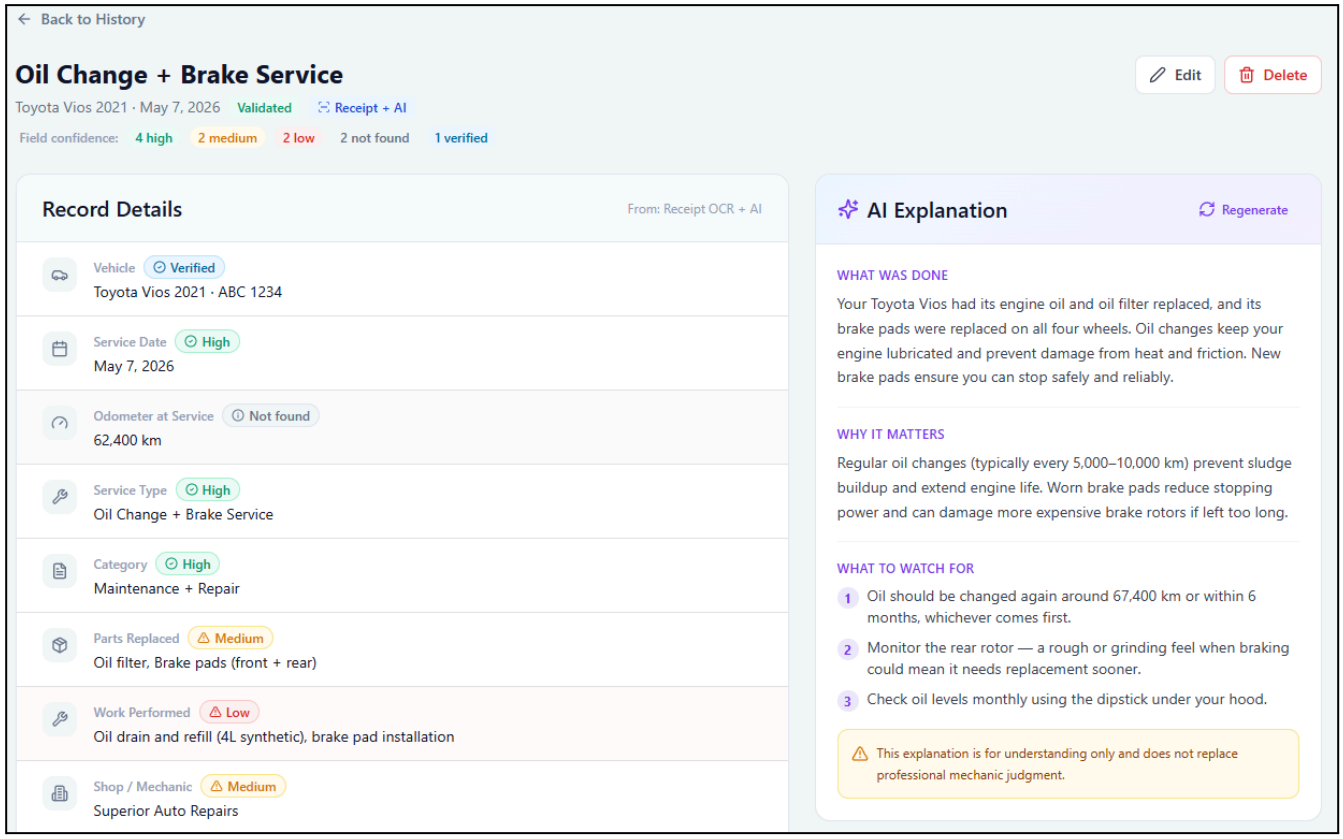

Figure - User Interface Design for Show AI-Generated Service Explanation

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
ServiceRecordDetailPage	Displays a selected validated service record together with its AI-generated explanation.	React page component
AIExplanationPanel	Shows the AI-generated explanation of what was done, why it matters, and what the owner should watch for when available.	React UI component
OriginalRecordSummary	Displays the original saved service record beside the AI explanation to avoid relying only on AI output.	React UI component
GenerateExplanationStatus	Shows loading, unavailable, or completed status while the explanation is being generated or retrieved.	React status component
ExplanationUnavailableMessage	Displays a fallback message if the AI service is unavailable.	Alert/message component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
AIController	Receives requests to retrieve or generate AI explanations for selected service records.	Spring Boot REST controller
AIExplanationService	Generates or retrieves owner-friendly explanations for validated service records.	Spring service class
ServiceRecordService	Retrieves the selected validated service record and verifies that it belongs to the authenticated owner.	Spring service class
AIResultRepository	Stores and retrieves AI-generated explanation results.	Repository/data access component
ServiceRecordRepository	Retrieves the original service record used for explanation generation.	Repository/data access component
AIExplanation	Represents the AI-generated explanation linked to a service record.	Domain model/entity class
ServiceRecord	Represents the validated service record used as the basis for the explanation.	Domain model/entity class
AuditLogService	Records AI explanation generation or retrieval events for traceability.	Spring service class

- ***Object-Oriented Components***

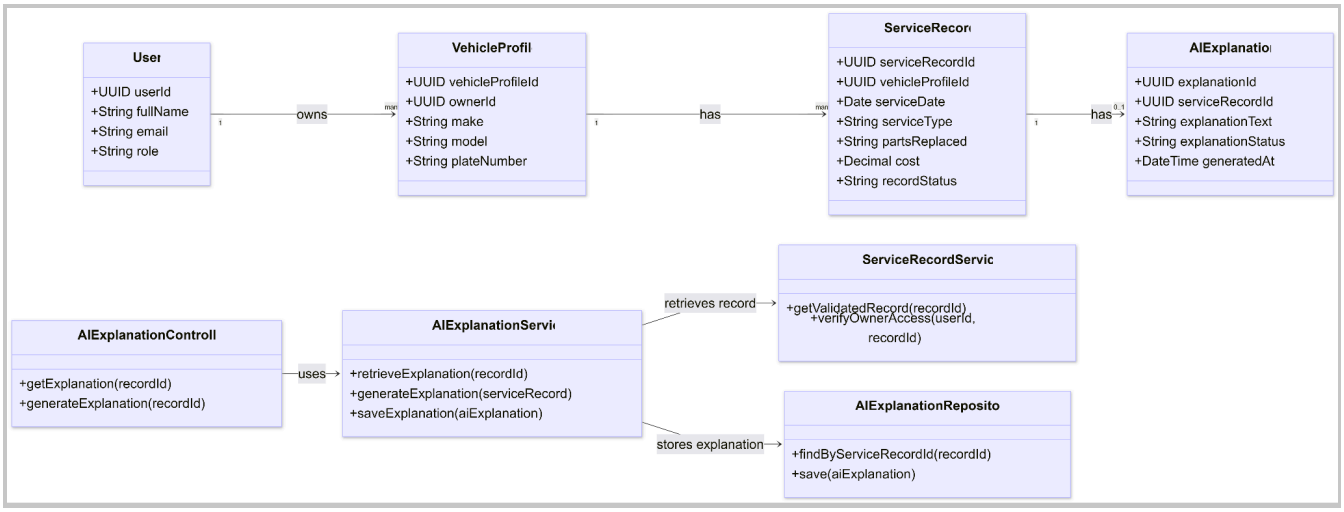

Figure - Class Diagram for Show AI-Generated Service Explanation

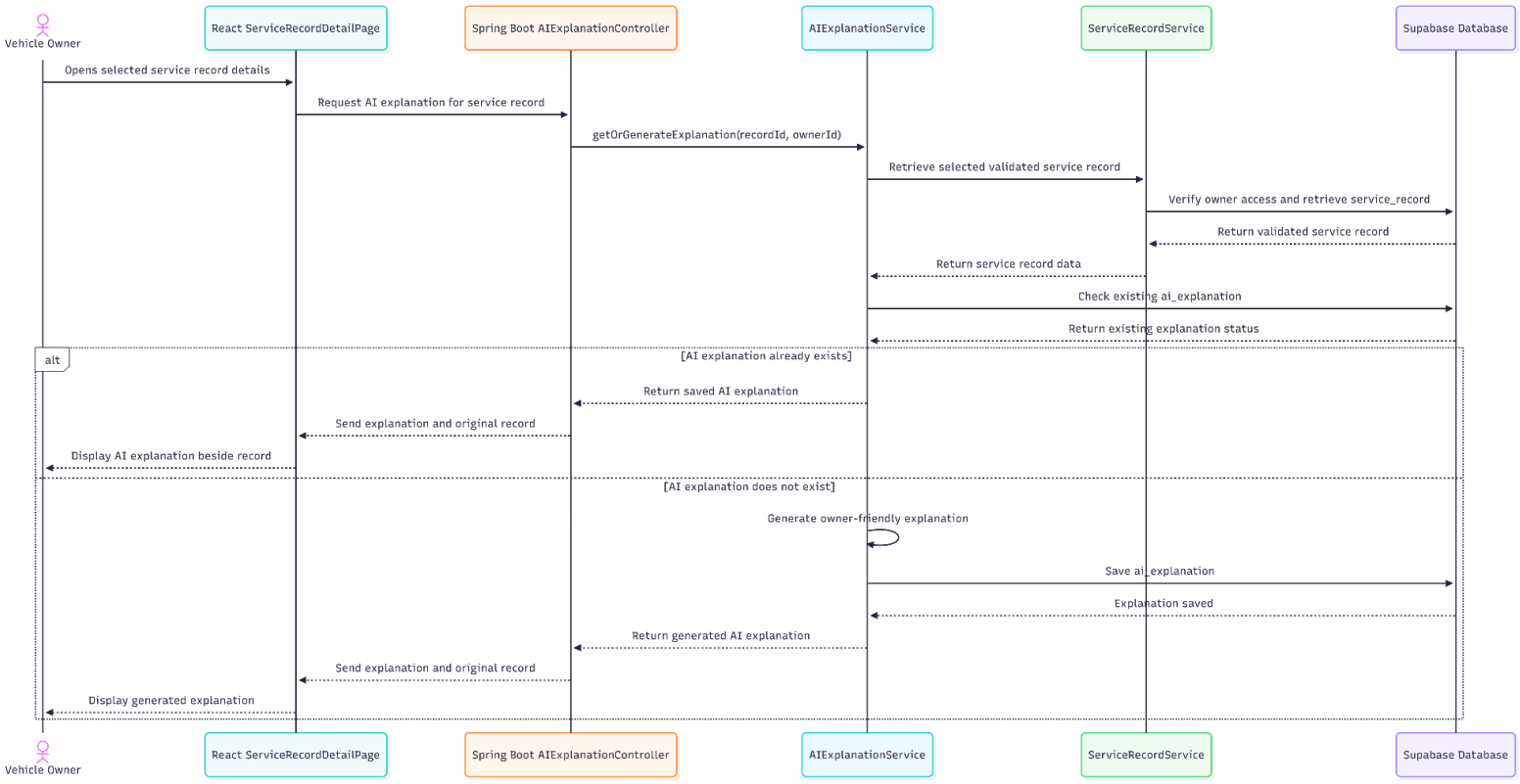

Figure - Sequence Diagram for Show AI-Generated Service Explanation

- ***Data Design***

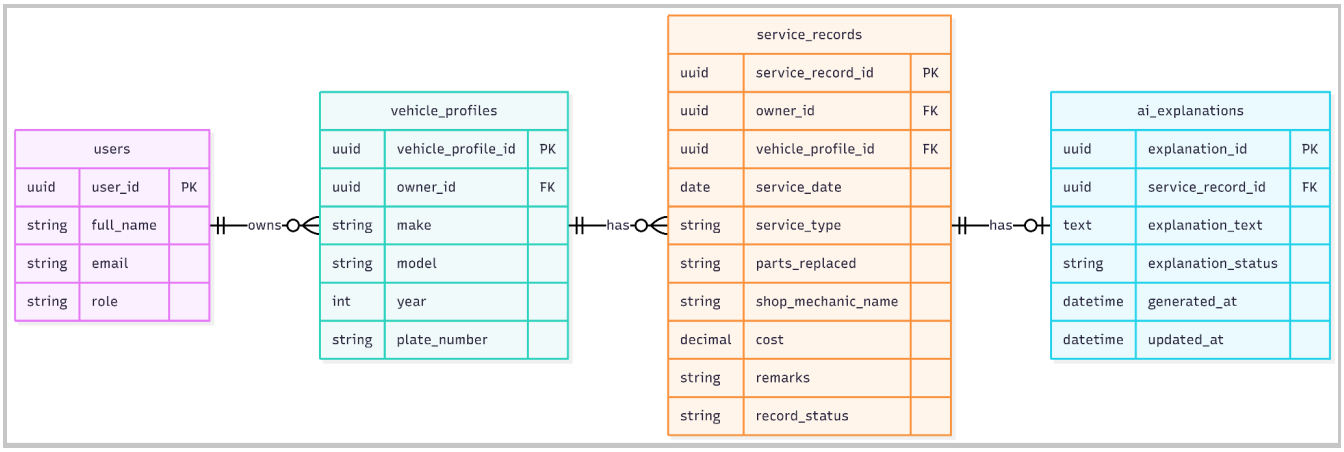

Figure - Entity Relationship Diagram Show AI-Generated Service Explanation

4.2 Generate QR Access Request

- ***User Interface Design***

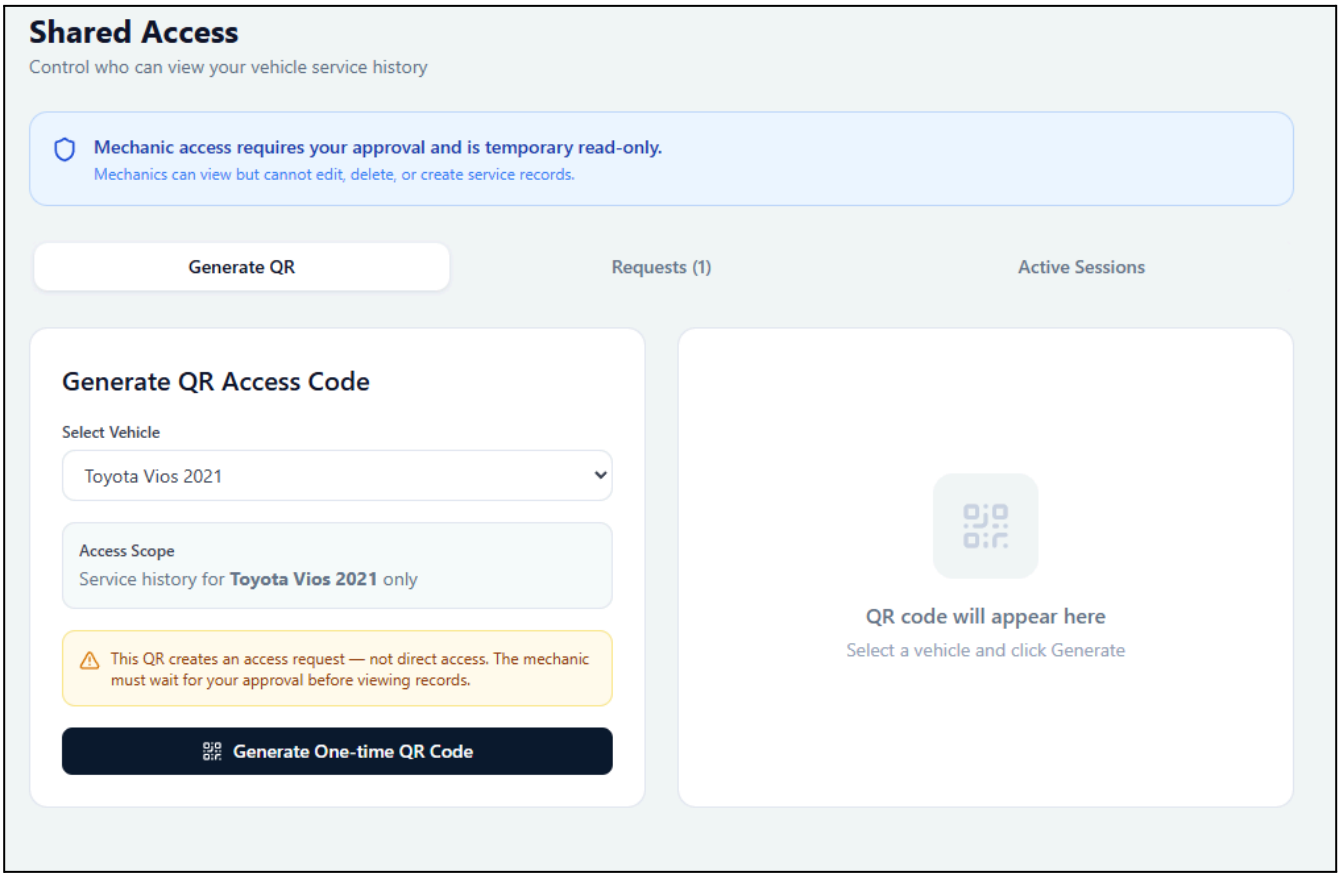

Figure - User Interface Design for Generate QR Access Request

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
QRSharingPage	Displays the interface for generating a QR access request for the selected vehicle service history.	React page component
VehicleShareSelector	Allows the owner to confirm which registered vehicle history will be shared.	React selection component
GenerateQRButton	Starts the QR access request generation process.	Button/action component
QRCodeDisplayPanel	Displays the generated one-time QR code for the mechanic to scan.	React UI component
AccessScopeNotice	Shows that the QR code applies only to the selected vehicle service history.	React notice component
QRExpirationNotice	Informs the owner that the QR request is one-time use and time-limited.	React notice component
QRGenerationStatus	Shows QR generation loading, success, or failure status.	React status component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
QRAccessController	Receives requests to generate one-time QR access requests for a selected vehicle.	Spring Boot REST controller
QRAccessService	Creates the QR access request, generates a unique access token, sets expiration details, and marks the request as one-time use.	Spring service class
VehicleService	Verifies that the selected vehicle belongs to the authenticated owner before QR generation.	Spring service class
QRCodeGenerator	Generates the QR code image or encoded QR content from the access token.	Utility/library wrapper
QRAccessRepository	Stores generated QR access request details.	Repository/data access component
QRAccessRequest	Represents a one-time QR access request linked to a selected vehicle profile.	Domain model/entity class
AuditLogService	Records QR generation events for access traceability.	Spring service class

- ***Object-Oriented Components***

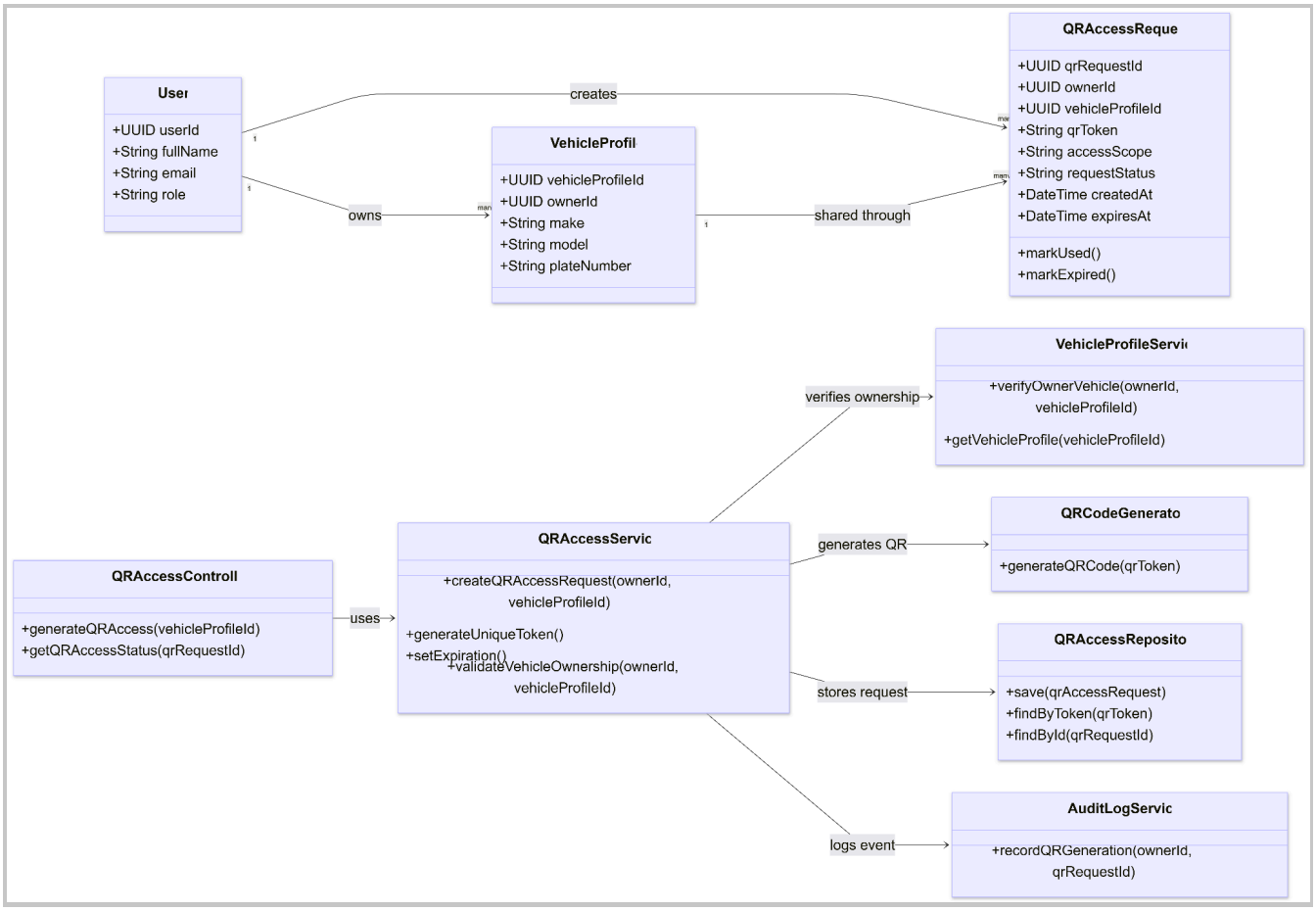

Figure - Class Diagram for Generate QR Access Request

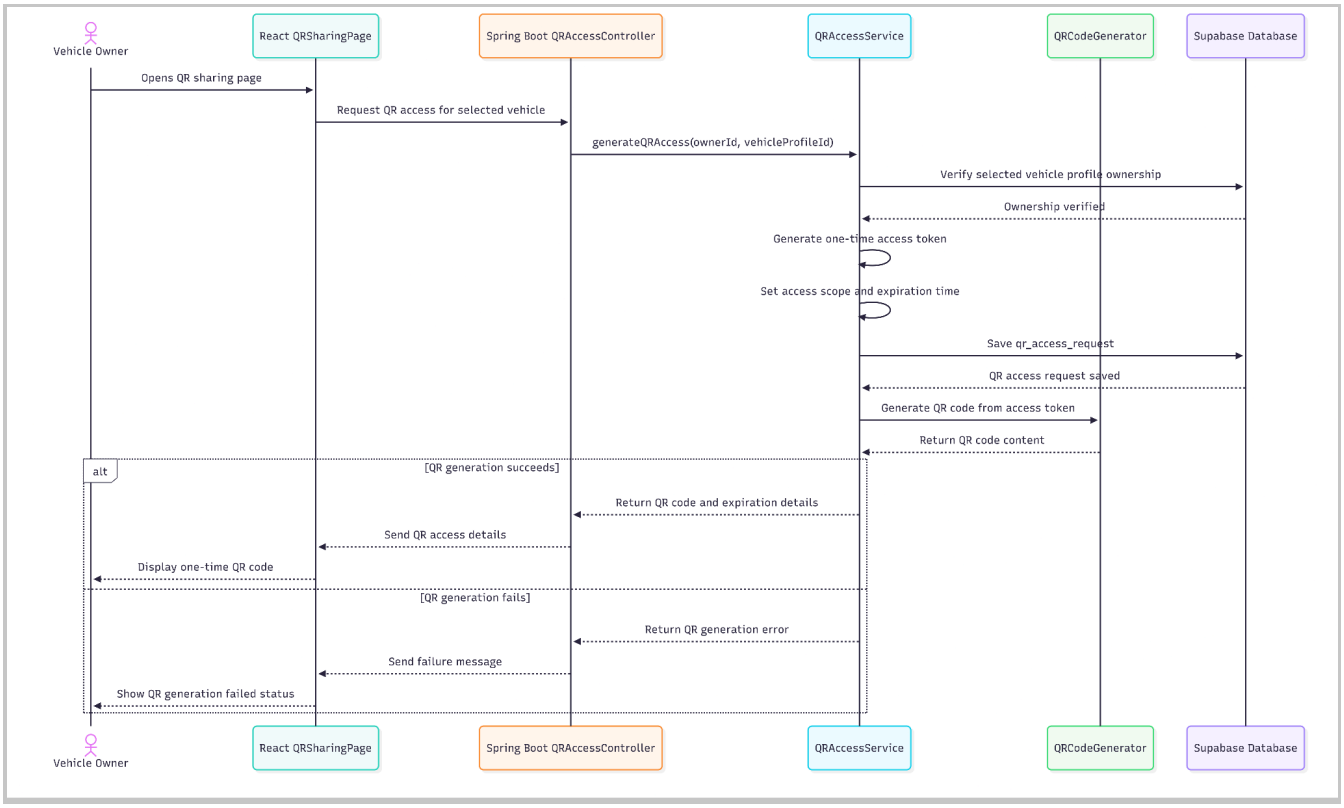

Figure - Sequence Diagram for Generate QR Access Request

- ***Data Design***

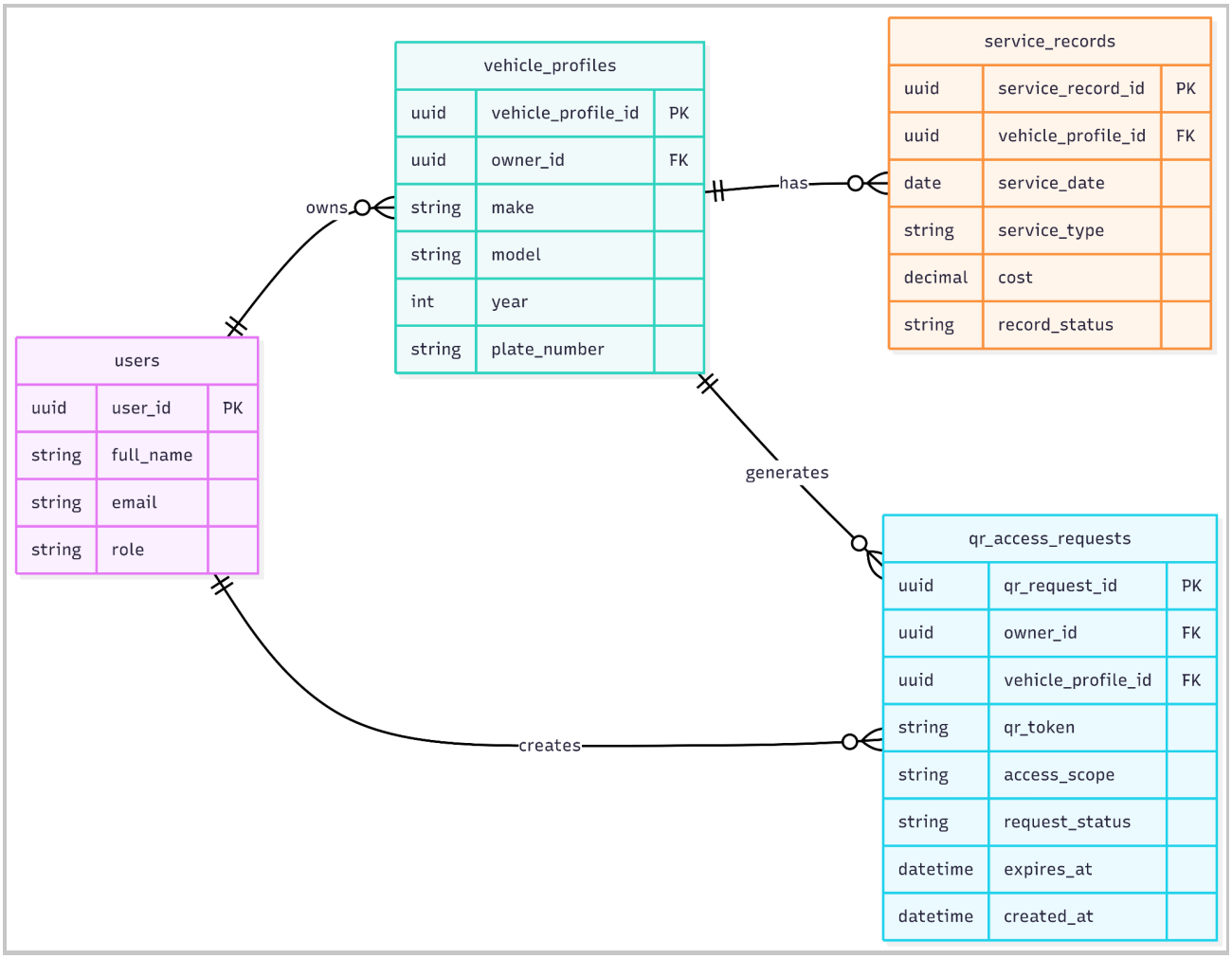

Figure - Entity Relationship Diagram for Generate QR Access Request

4.3 Notify Temporary Access Expiration

- ***User Interface Design***

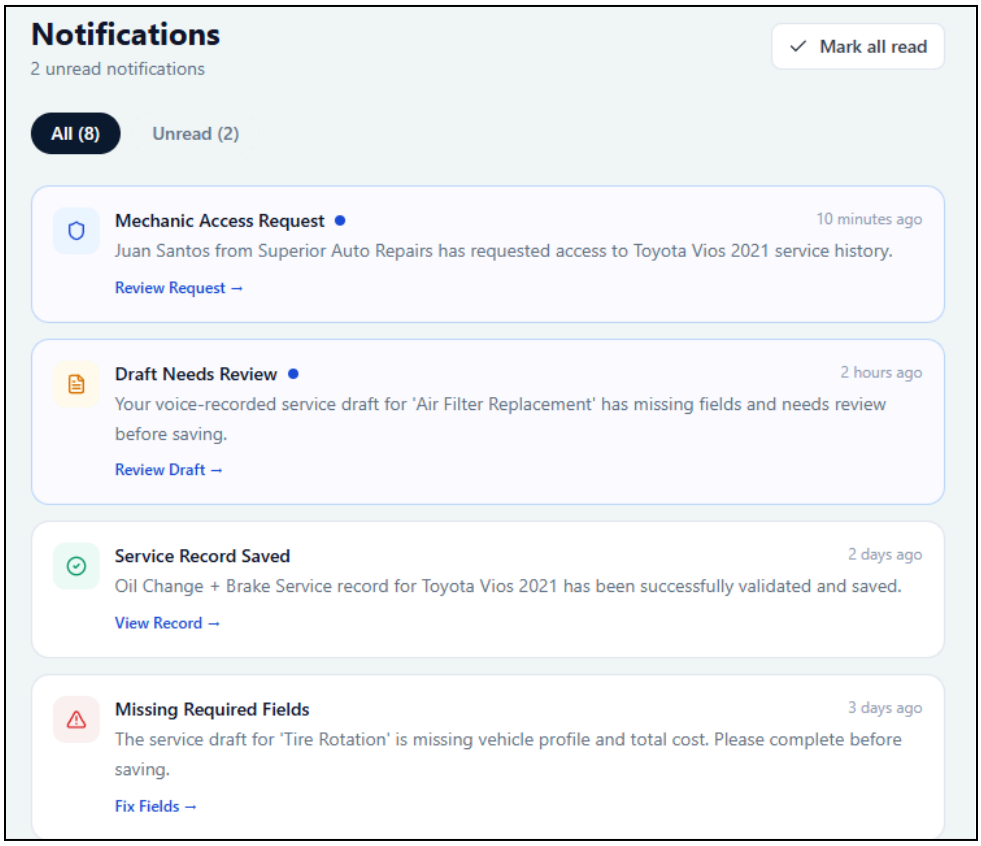

Figure - User Interface Design for Notify Temporary Access Expiration

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
TemporaryAccessNoticePanel	Displays the interface for generating a QR access request for the selected vehicle service history.	React UI component
AccessExpirationInfo	Shows the configured expiration time or expiration message for the current QR access request.	React UI component
SharingPolicyNotice	Explains that mechanics can only request access and cannot view records until the owner approves.	React notice component
QRAccessStatusPanel	Shows the current state of the QR access request, such as active, used, expired, or pending mechanic request.	React status component
RefreshAccessStatusButton	Allows the owner to refresh the current QR access status.	Button/action component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
QRAccessController	Provides QR access status and expiration details to the frontend.	Spring Boot REST controller
QRAccessService	Checks whether the QR access request is active, used, expired, or pending.	Spring service class
MechanicAccessService	Checks temporary mechanic access session status when an approved session exists.	Spring service class
QRAccessRepository	Retrieves QR access request expiration and status data.	Repository/data access component
MechanicAccessRepository	Retrieves temporary mechanic access session details when available.	Repository/data access component
QRAccessRequest	Represents the QR request with expiration time, one-time-use status, and selected vehicle scope.	Domain model/entity class
MechanicAccessSession	Represents approved temporary mechanic access and its expiration time.	Domain model/entity class

- ***Object-Oriented Components***

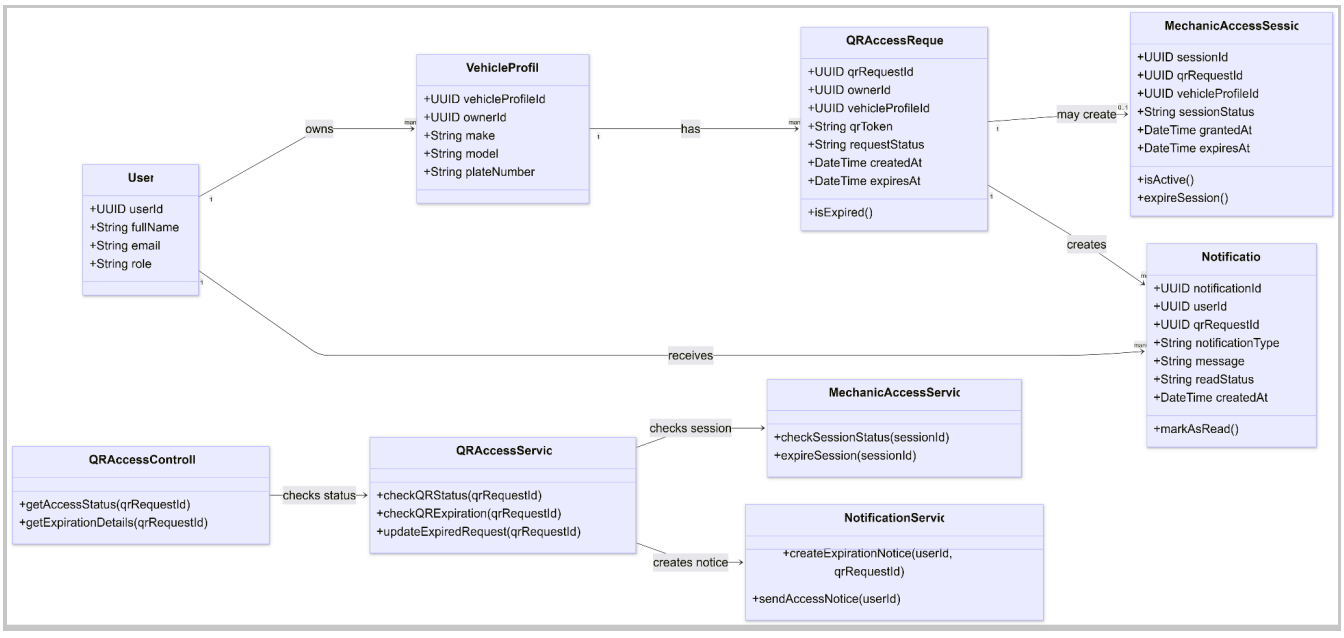

Figure - Class Diagram for Notify Temporary Access Expiration

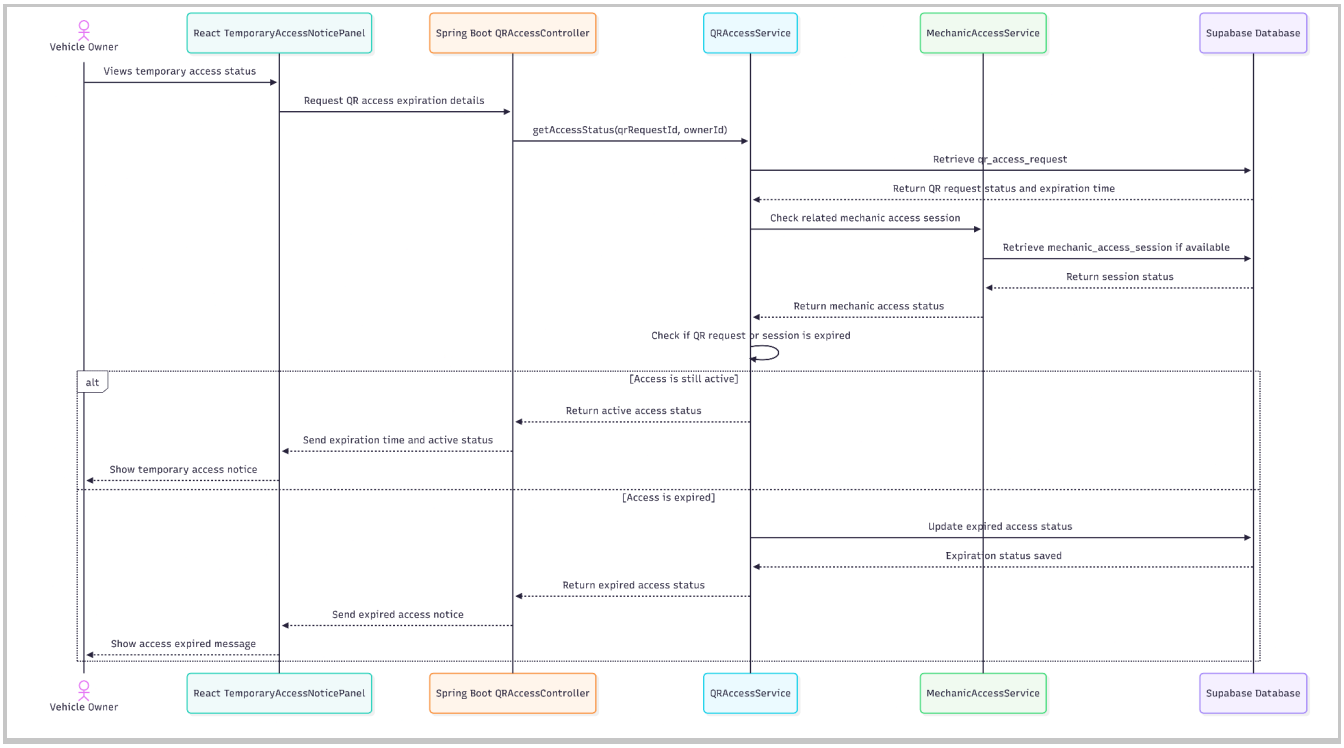

Figure - Sequence Diagram for Notify Temporary Access Expiration

- ***Data Design***

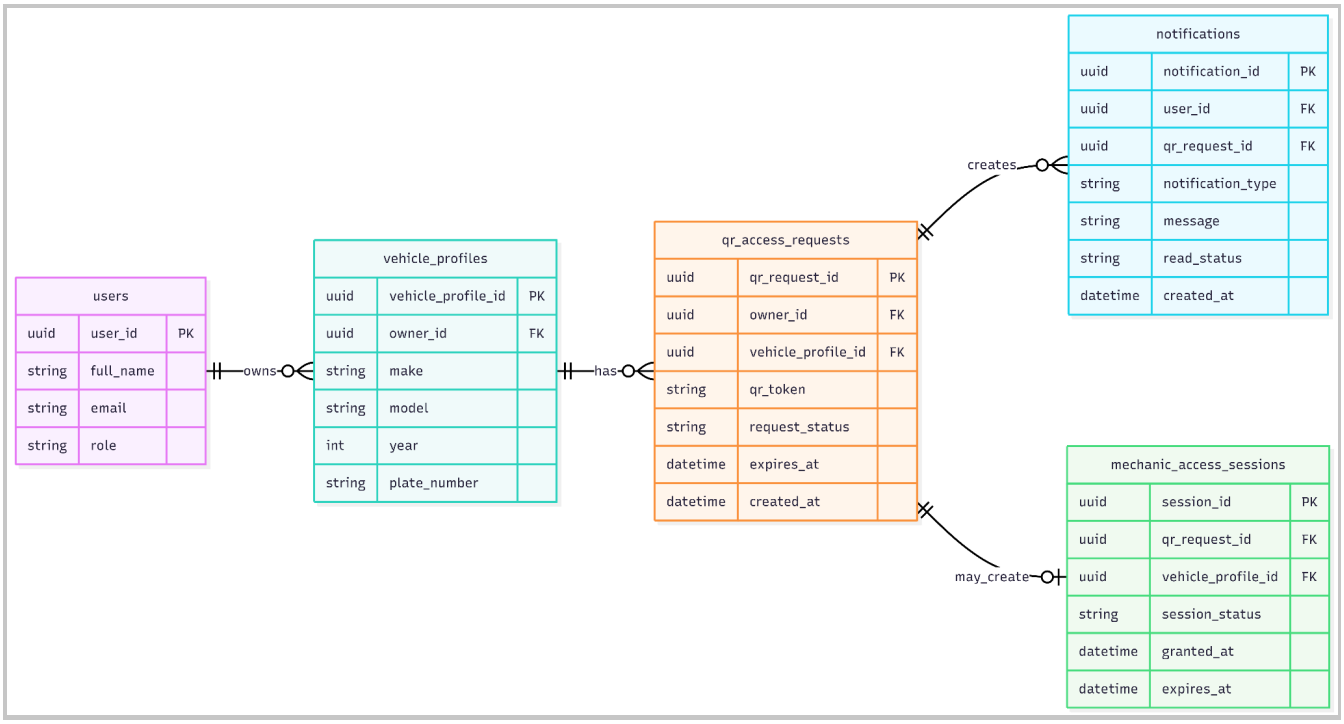

Figure - Entity Relationship Diagram for Generate QR Access Request

4.4 Approve Mechanic Access Request

- ***User Interface Design***

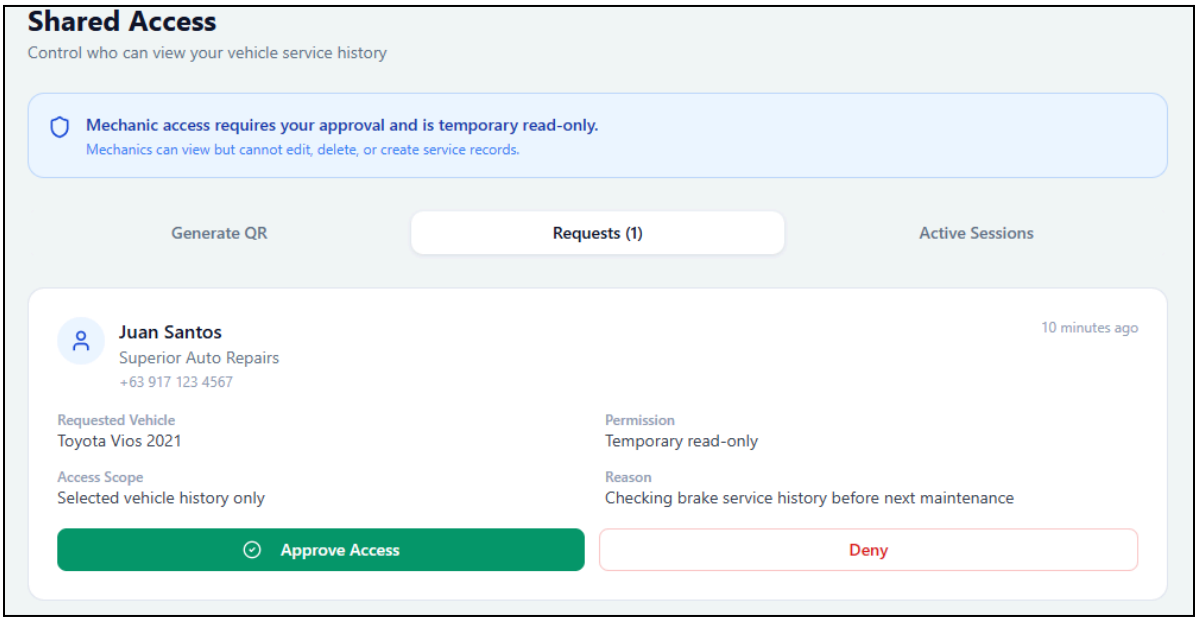

Figure - Approve Mechanic Access Request

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
MechanicAccess RequestPage	Displays incoming mechanic access requests for the owner's selected vehicle history.	React page component
MechanicRequest Card	Shows the configured expiration time or expiration message for the current QR Shows mechanic request details, such as request time, requested vehicle, and request status. request.	Reusable React UI component
AccessScopeSum mary	Displays which vehicle service history the mechanic is requesting to view.	React UI component
ApproveAccessBut ton	Allows the owner to approve the mechanic's request.	Button/action component
DenyAccessButton	Allows the owner to deny the mechanic's request.	Button/action component
ApprovalStatusMe ssage	Displays whether the access request was approved, denied, expired, or failed.	Alert/status component
OwnerApprovalNot ice	Reminds the owner that approving gives temporary read-only access only.	React notice component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
MechanicAccessController	Receives mechanic access requests and owner approval or denial decisions.	Spring Boot REST controller
AccessApprovalService	Handles owner approval or denial and enforces that access cannot be granted without owner approval.	Spring service class
QRAccessService	Verifies that the QR access request is valid, unused, and not expired.	Spring service class
MechanicAccessService	Creates an approved temporary read-only mechanic access session after owner approval.	Spring service class
NotificationService	Notifies the owner of a mechanic request and informs the mechanic of approval or denial status.	Spring service class
QRAccessRepository	Retrieves and updates the QR access request status after mechanic scan or owner decision.	Repository/data access component
MechanicAccessRepository	Stores the mechanic access request and approved access session.	Repository/data access component
QRAccessRequest	Represents the one-time QR access request submitted by the mechanic.	Domain model/entity class
MechanicAccessSession	Represents the approved temporary read-only mechanic access session.	Domain model/entity class
AuditLogService	Records request, approval, denial, and access-related events.	Spring service class

- ***Object-Oriented Components***

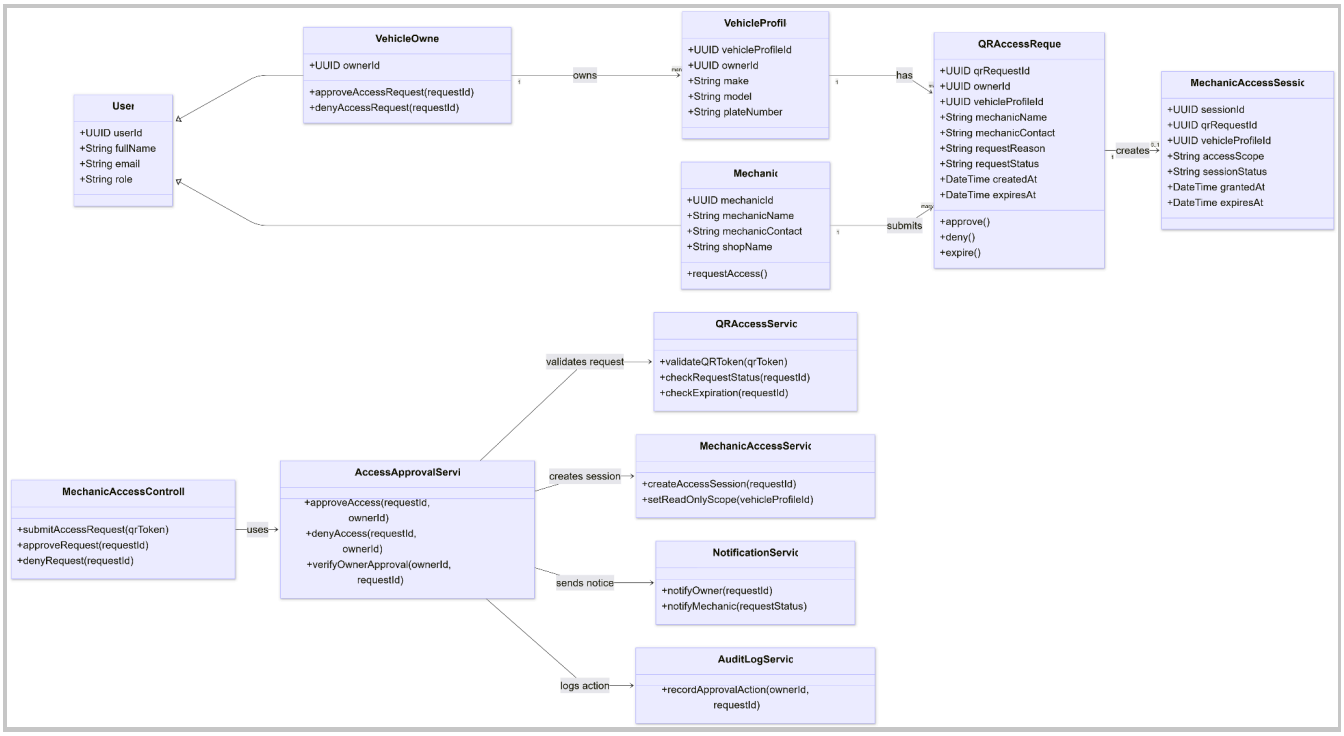

Figure - Class Diagram for Approve Mechanic Access Request

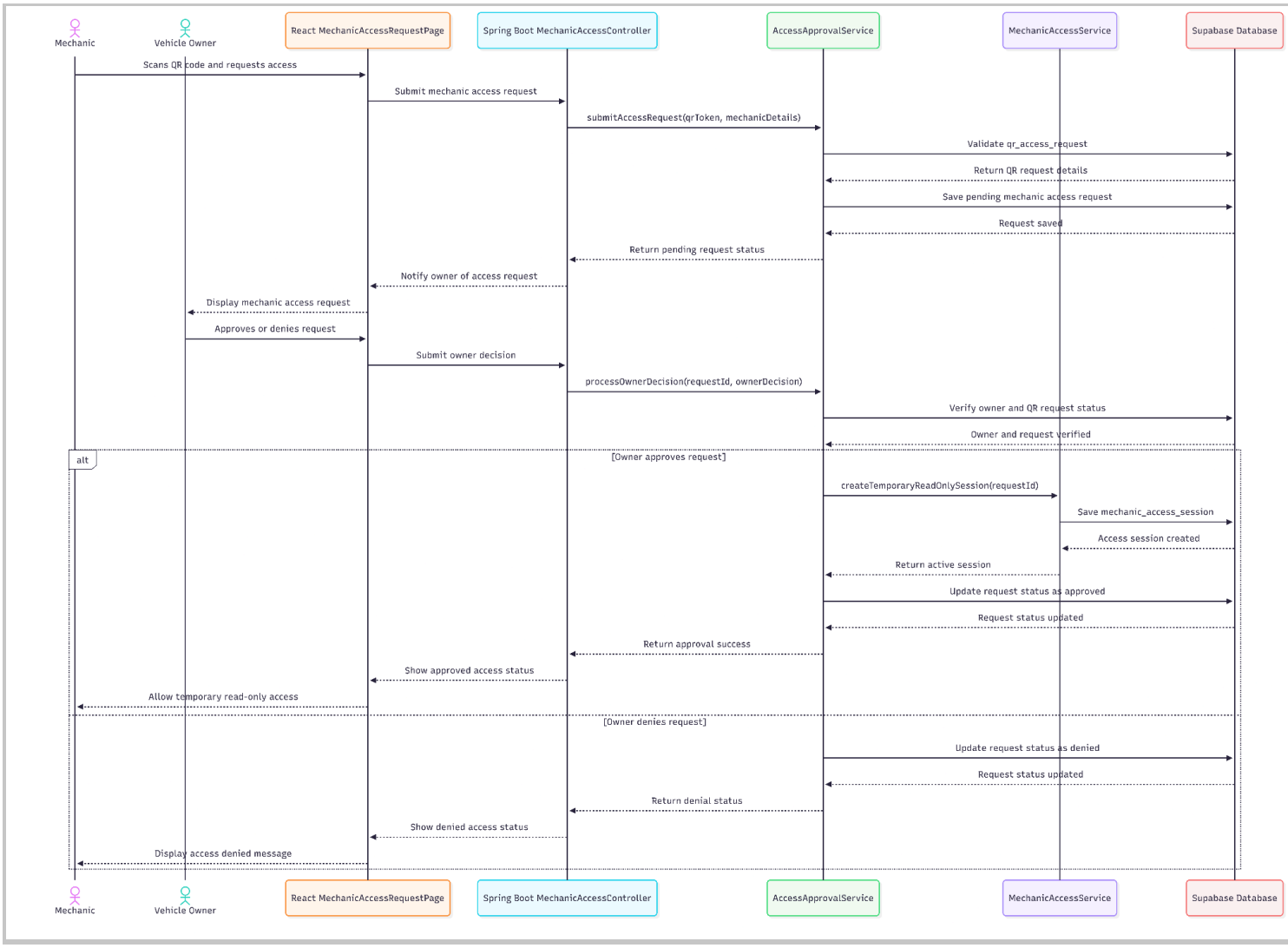

Figure -Sequence Diagram for Approve Mechanic Access Request

- ***Data Design***

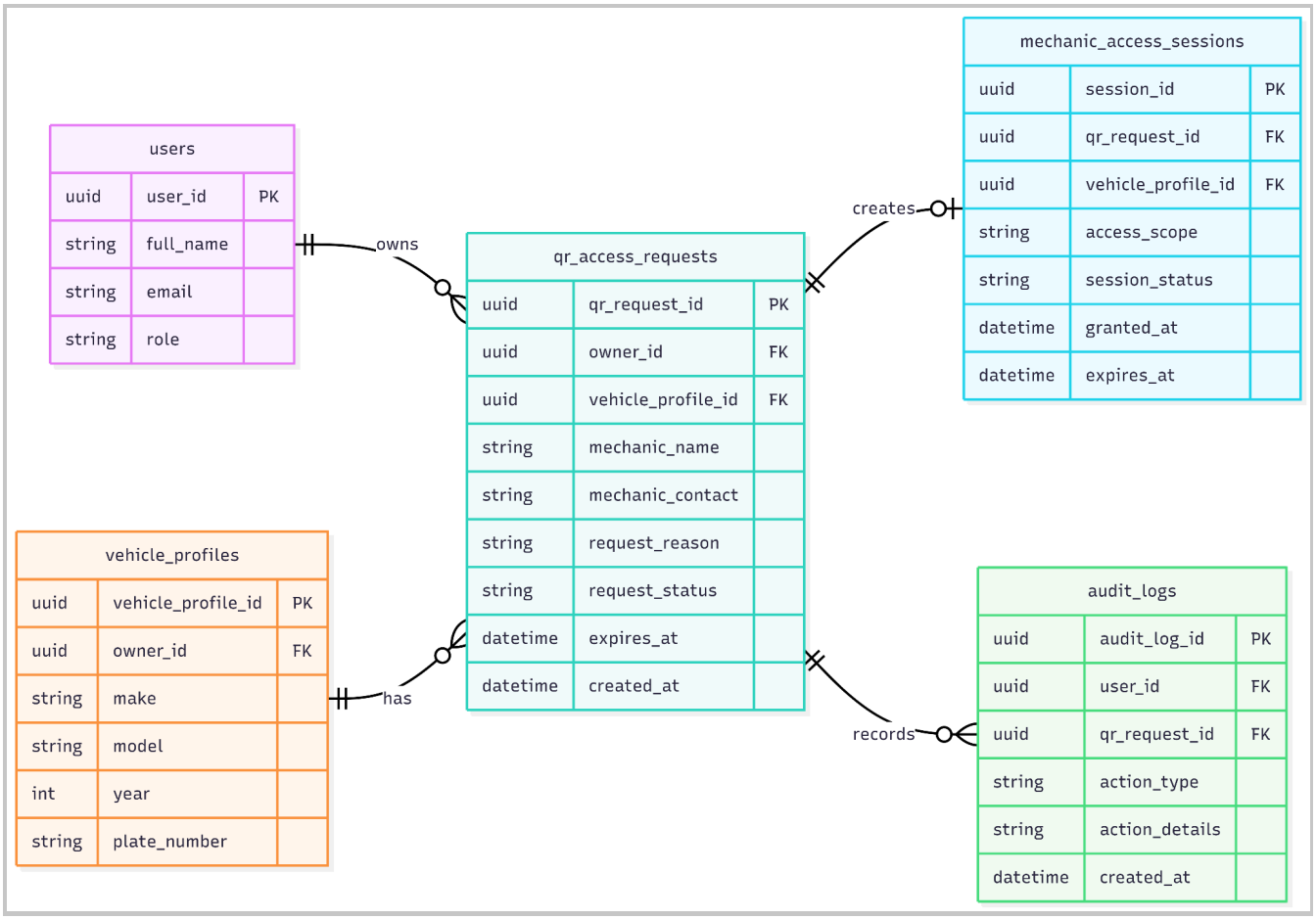

Figure - Entity Relationship Diagram for Approve Mechanic Access Request

4.5 Provide Temporary Read-Only Access

- ***User Interface Design***

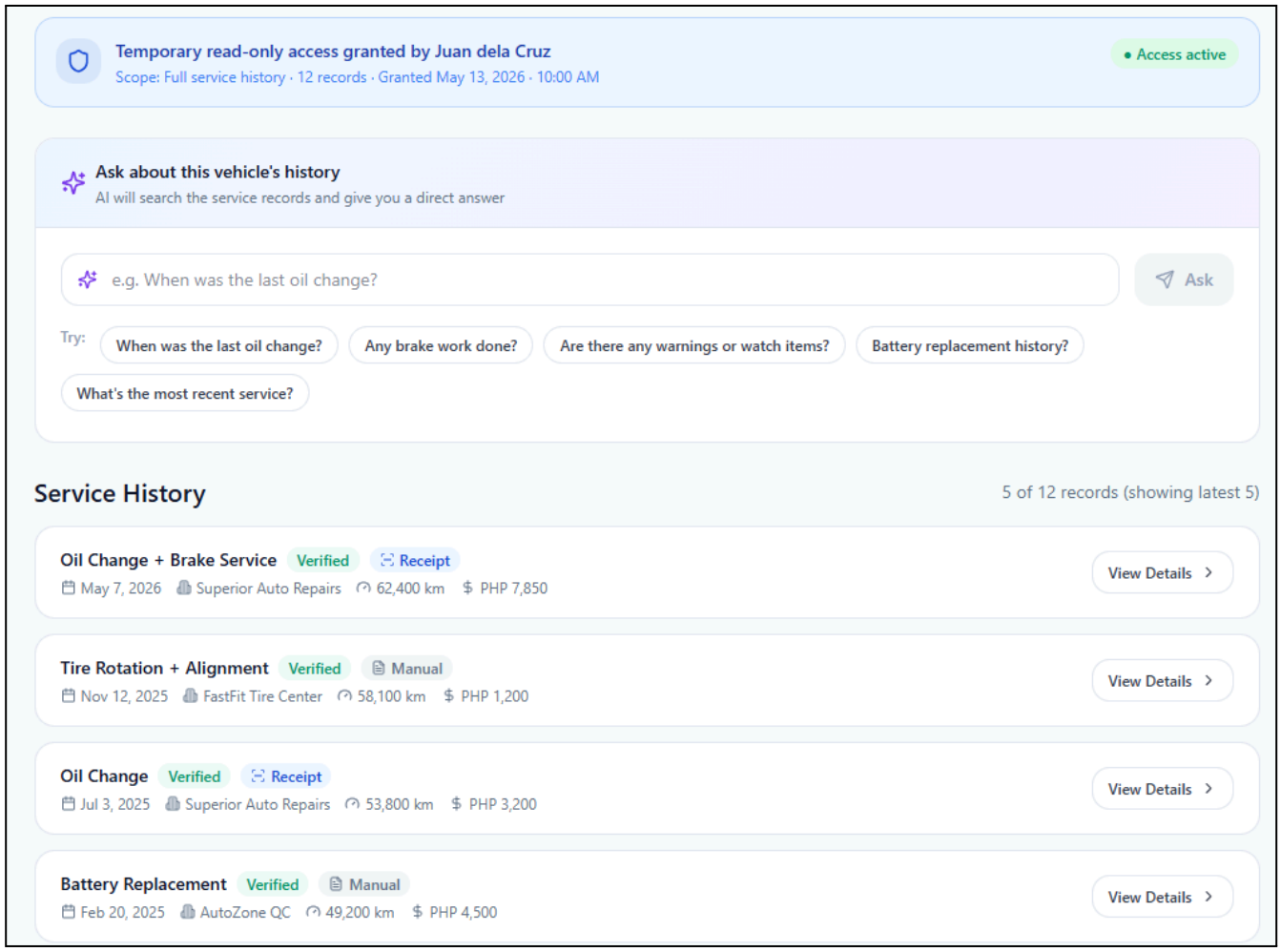

Figure - User Interface Design for Provide Temporary Read-Only Access

- ***Front-end component(s)***

Component Name	Description and Purpose	Component Type / Format
MechanicReadOnlyHistoryPage	Displays the approved vehicle service history to the mechanic in read-only mode.	React page component
ReadOnlyServiceHistoryList	Shows the approved vehicle's validated service records without edit or delete controls.	React list component
ReadOnlyServiceRecordCard	Displays individual service record details in a non-editable format.	Reusable React UI component
AccessTimerBanner	Shows remaining access time or expiration status for the temporary mechanic session.	React status component
ExpiredAccessMessage	Displays a message when the mechanic access session has expired.	Alert/message component
MechanicSearchEntryPoint	Provides access to AI-assisted search only while the approved session is active.	React UI/search component

- ***Back-end component(s)***

Component Name	Description and Purpose	Component Type / Format
MechanicAccessController	Handles requests from mechanics viewing approved temporary service history.	Spring Boot REST controller
MechanicAccessService	Verifies active session status, read-only permission, expiration time, and approved vehicle scope.	Spring service class
ServiceHistoryService	Retrieves the approved vehicle's validated service history for read-only viewing.	Spring service class
MechanicAccessRepository	Retrieves the mechanic access session and checks expiration status.	Repository/data access component
ServiceRecordRepository	Retrieves validated service records for the approved vehicle only.	Repository/data access component
MechanicAccessSession	Represents the temporary read-only access session with expiration and vehicle scope.	Domain model/entity class
ServiceRecord	Represents saved validated service records shown to the mechanic.	Domain model/entity class
AuditLogService	Records mechanic viewing activity for traceability.	Spring service class

- ***Object-Oriented Components***

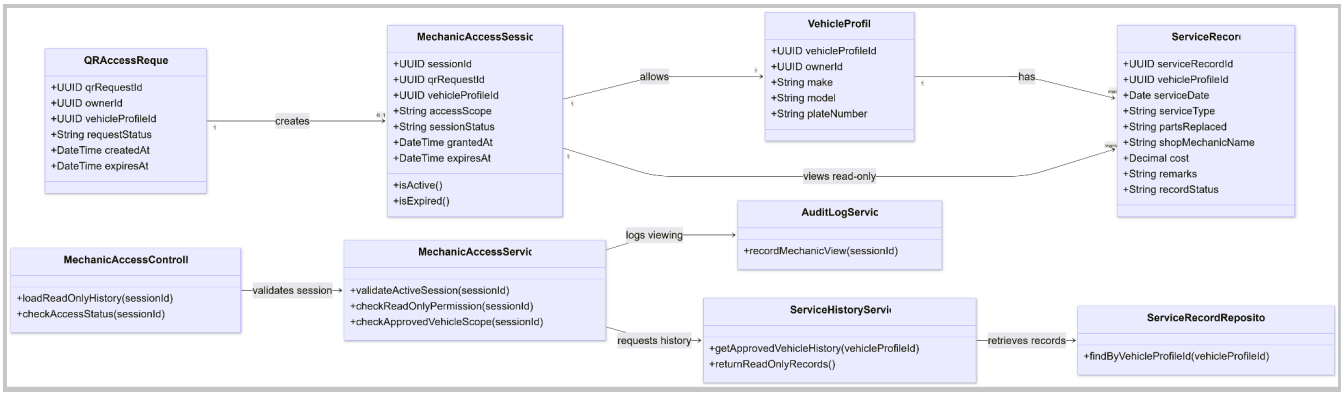

Figure - Class Diagram for Provide Temporary Read-Only Access

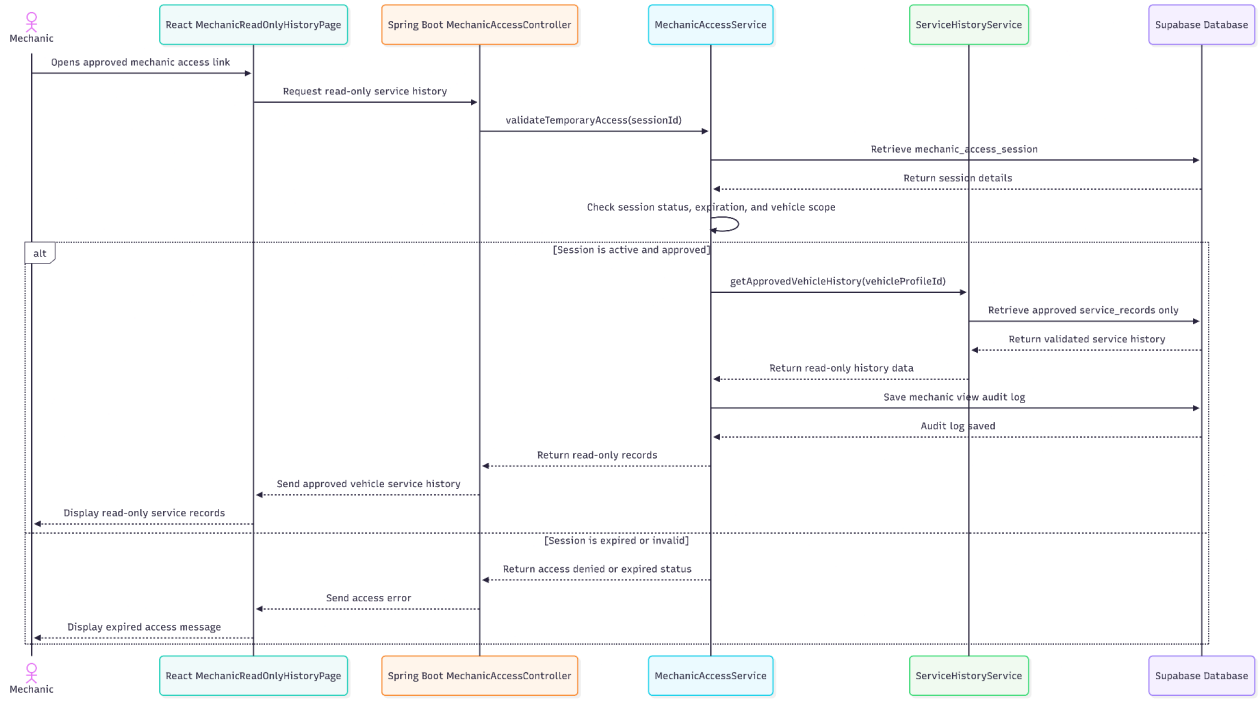

Figure - Sequence Diagram for Provide Temporary Read-Only Access

- ***Data Design***

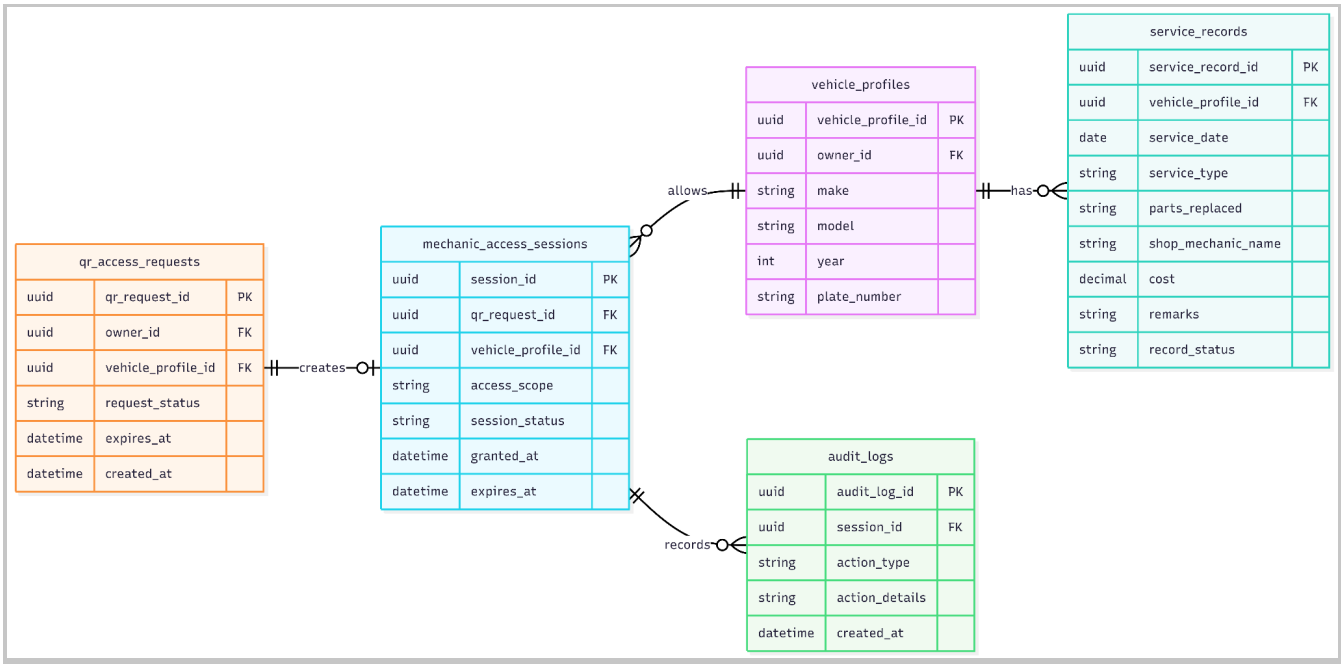

Figure - Entity Relationship Diagram for Provide Temporary Read-Only Access

4.6 Search Shared Records with AI Assistance

- ***User Interface Design***

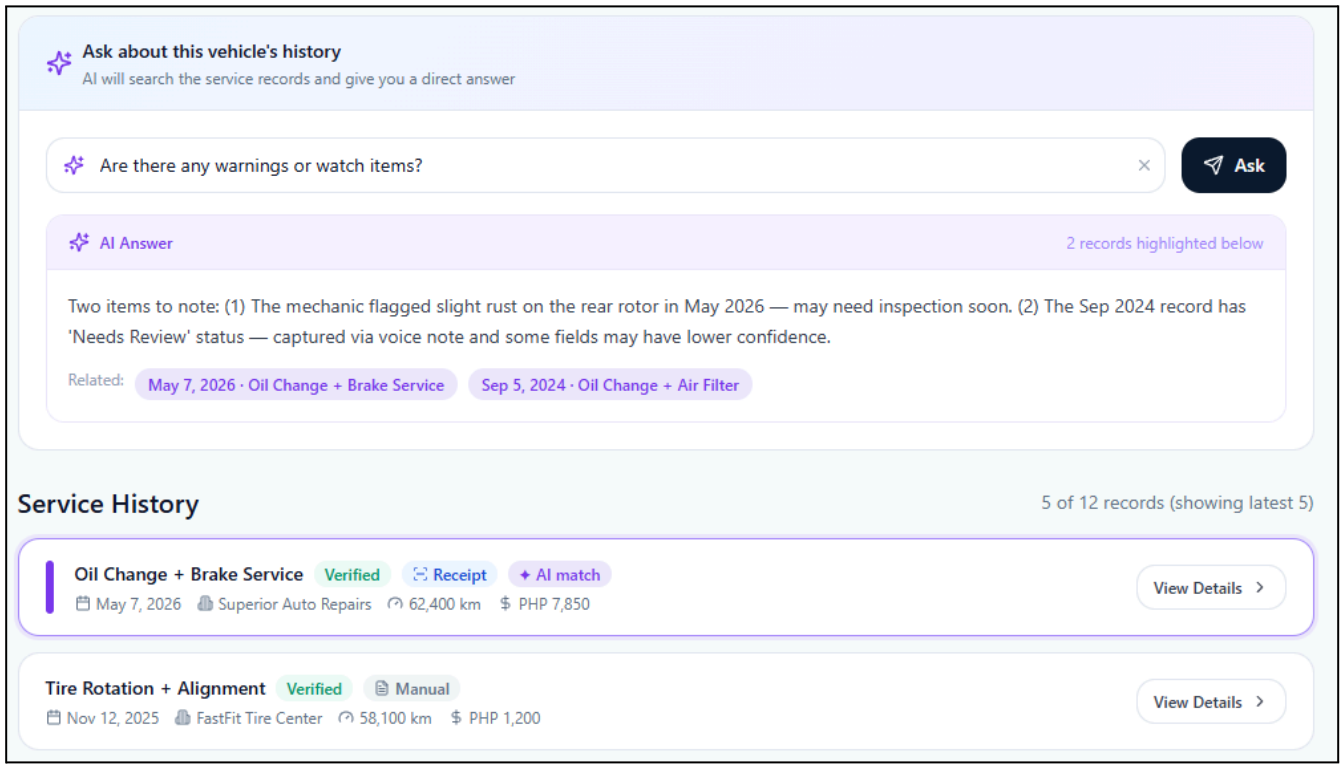

Figure - User Interface Design for Search Shared Records with AI Assistance

- **Front-end component(s)**

Component Name	Description and Purpose	Component Type / Format
MechanicAISearchPanel	Allows the mechanic to enter a service-related question or keyword while viewing approved shared records.	React search component
SearchQueryInput	Accepts natural-language search questions or keywords from the mechanic.	React input component
SearchResultsList	Displays service records related to the mechanic's search query.	React list component
SearchResultRecordCard	Shows each matching service record with relevant service details and explanation snippet when available.	Reusable React UI component
NoSearchResultsMessage	Displays a message when no related approved record is found.	Alert/message component
SearchUnavailableMessage	Displays a fallback message when AI-assisted search is unavailable.	Alert/message component
ManualBrowseFallbackButton	Allows the mechanic to return to manually browsing the approved service history.	Button/navigation component

- **Back-end component(s)**

Component Name	Description and Purpose	Component Type / Format
AIController	Receives AI-assisted mechanic search requests.	Spring Boot REST controller
MechanicSearchService	Processes the mechanic's query and searches only within the approved shared vehicle service history.	Spring service class
MechanicAccessService	Verifies that the mechanic has an active, approved, unexpired access session before search is allowed.	Spring service class
ServiceHistoryService	Provides the approved vehicle's service records as the search scope.	Spring service class
AIExplanationService	May provide record explanation snippets or AI-assisted interpretation for related search results.	Spring service class
ServiceRecordRepository	Retrieves approved vehicle service records used as the searchable data source.	Repository/data access component

MechanicSearchLogRepository	Stores mechanic search queries and result metadata for audit and monitoring.	Repository/data access component
MechanicSearchLog	Represents the mechanic's search query, timestamp, session ID, and result count.	Domain model/entity class
MechanicAccessSession	Represents the approved access session used to limit the search scope.	Domain model/entity class
ServiceRecord	Represents the approved records searched and returned to the mechanic.	Domain model/entity class
AuditLogService	Records AI-assisted search activity for traceability.	Spring service class

- ***Object-Oriented Components***

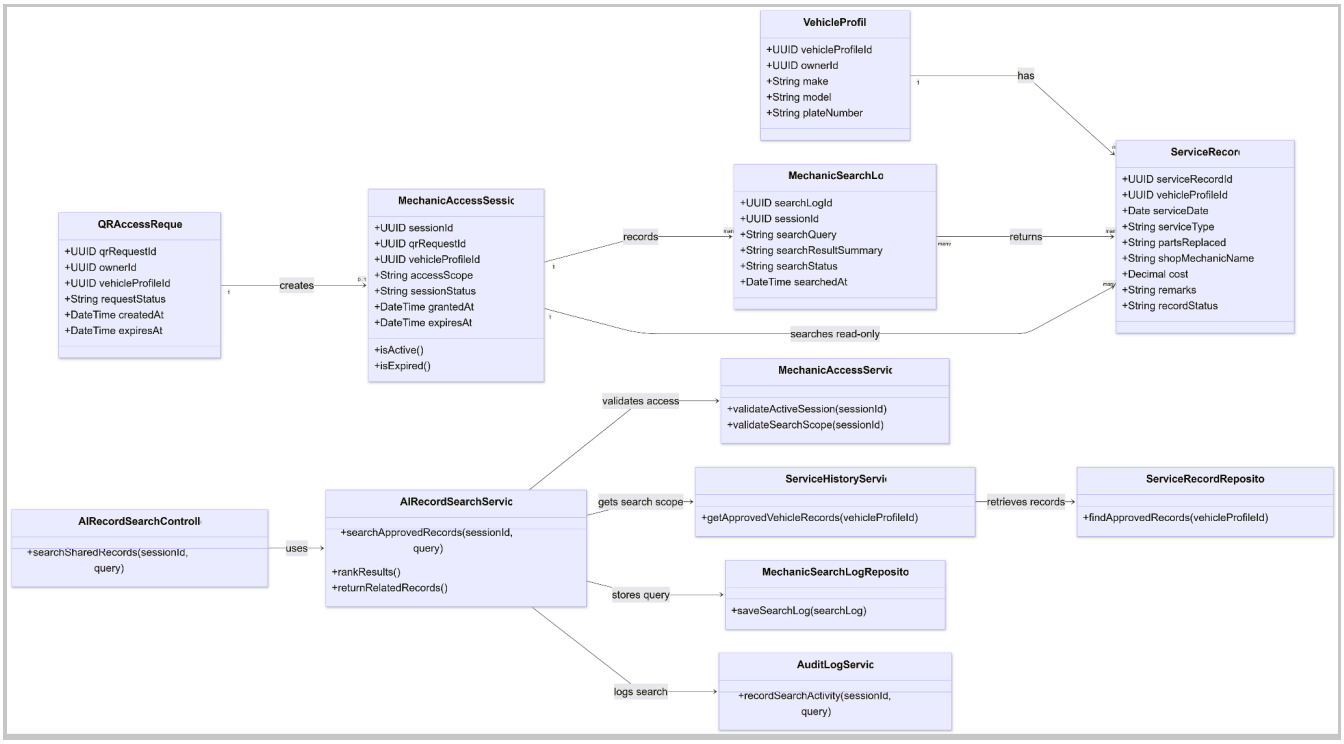

Figure -Class Diagram for Search Shared Records with AI Assistance

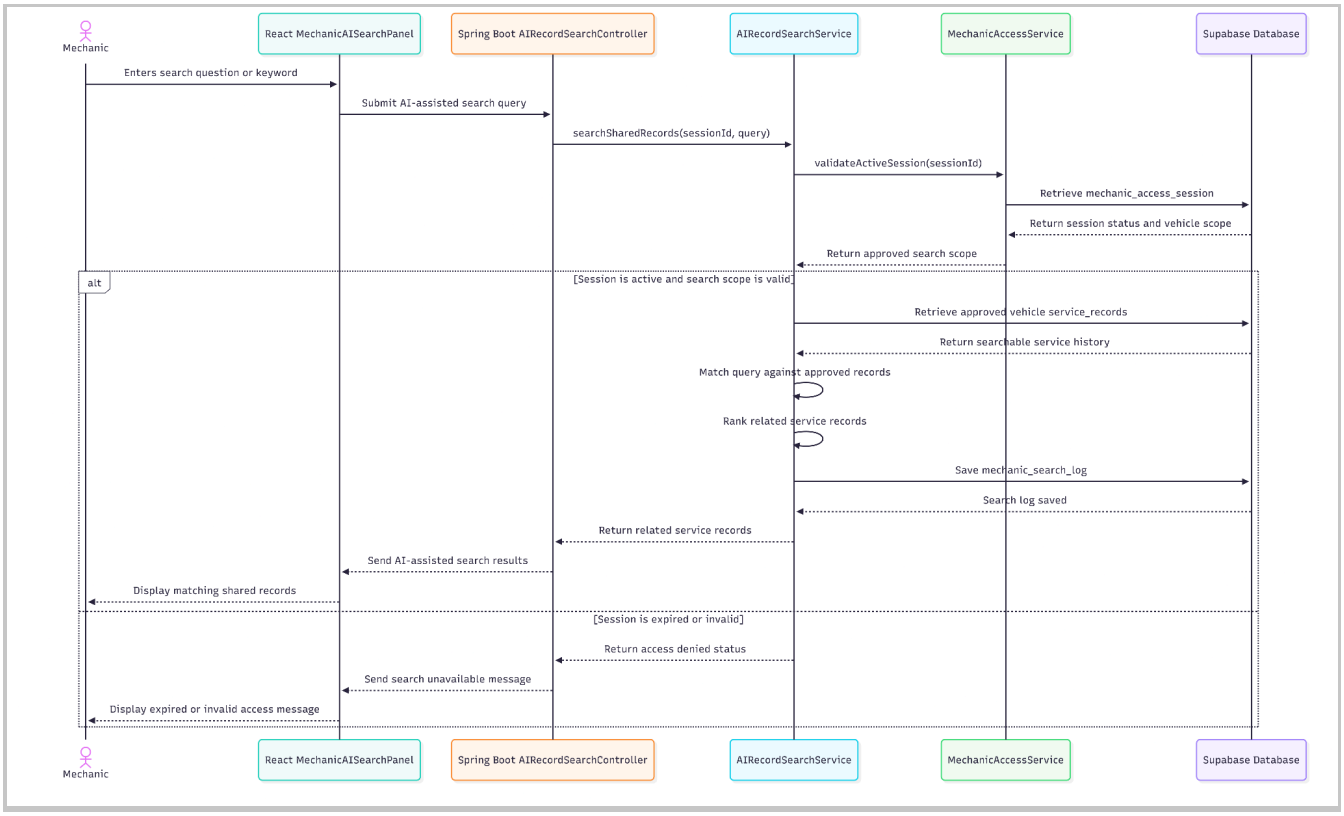

Figure - Sequence Diagram for Search Shared Records with AI Assistance

- ***Data Design***

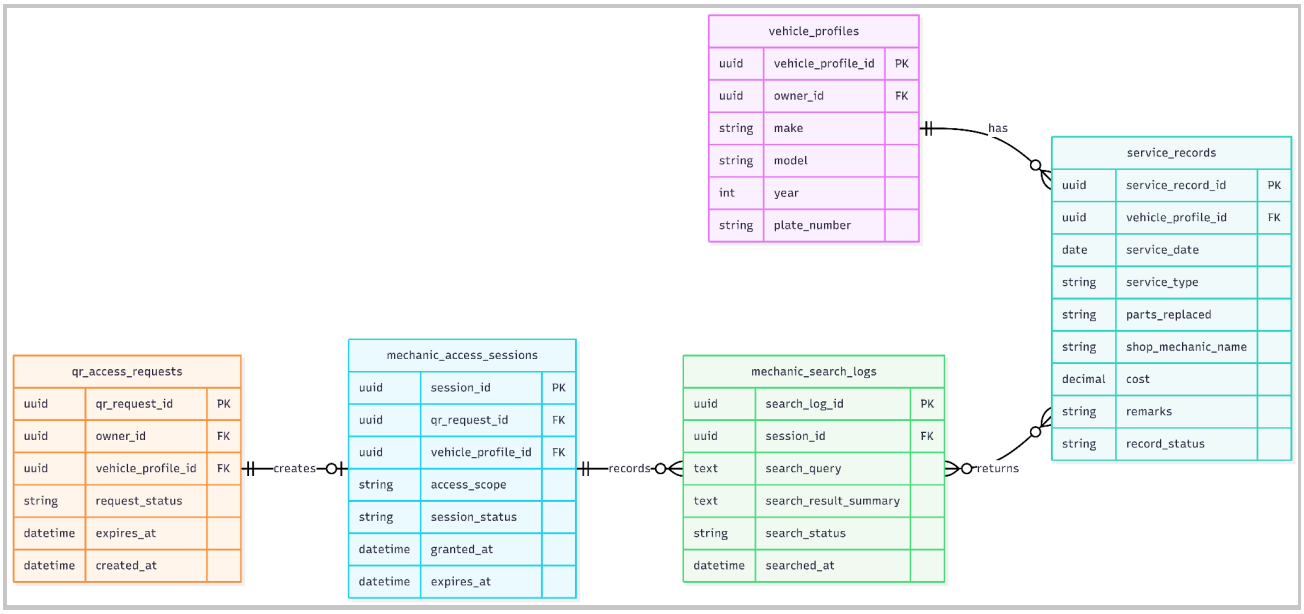

Figure - User Interface Design for Search Shared Records with AI Assistance